Apache Solr: A Practical Engineering Handbook

From Inverted Index to MCP — A Working Engineer's Companion

Aditya Parikh

June 2026

Table of contents

Preface

This handbook is a current, opinionated, code-first walkthrough of Apache Solr 10 — its internals, indexing pipeline, query layer, distributed runtime (SolrCloud), the Model Context Protocol integration, and the broader search-engine landscape it lives in. Every chapter mixes theory with concrete XML, JSON, and SolrJ Java snippets you can run. Diagrams are written in Mermaid and live as text in the source repository.

The book covers Apache Solr 10.0.0 (released 2025) and explicitly calls out differences from Solr 9.x where they affect upgrades. Concepts that have been stable for multiple major versions are presented without version drift; concepts that changed recently are tagged with the JIRA ticket and release in which they landed.

Who this book is for

This book assumes:

- You are a working software engineer. You know Java, HTTP, SQL, and at least one relational database. You have written REST APIs.
- You are not a complete beginner with search. You understand what a search box is and have probably used Elasticsearch or Solr at least casually. If you have never seen an inverted index, Chapter 1 is for you; the rest of the book builds on it.
- You work in a JVM shop. Code samples are Java/SolrJ. The architectural advice is language-neutral, but the runnable code is not.

It is not written for:

- Pure data scientists looking for retrieval-augmented-generation recipes. (Chapter 5 touches relevance engineering, but this is an infrastructure book.)
- Operators of pre-9.x Solr clusters who need an upgrade-only guide. Major-version migration steps are flagged but not exhaustively documented.
- Readers expecting Python, Ruby, or Node.js client examples. The patterns transfer; the code does not.

How to read this book

Chapters 1–4 build on each other and are best read in order. Chapters 5–7 stand alone once that spine is in place.

- Chapter 1 — Why Solr establishes why a B-tree over VARCHAR is the wrong tool for full-text search, walks through the inverted index as it actually lives on disk in a Lucene segment, and ends with a decision framework: Solr vs. SQL vs. a vector database vs. PostgreSQL FTS.
- Chapter 2 — Indexing covers the schema (managed-schema, field types, copyField, dynamicField, docValues, multivalued and dynamic fields), the SolrJ indexing path, partial updates with optimistic concurrency, reindexing strategy, hard vs. soft commits, and segment merging.
- Chapter 3 — Search is the query layer: request handlers, queries vs. filters, the Standard / DisMax / eDisMax parsers, field and range facets (JSON Facet API), BM25 in honest detail, vector and hybrid retrieval, suggesters, highlighting, collapse/expand, and how to debug a slow request.
- Chapter 4 — SolrCloud is the distributed runtime: ZooKeeper, shards, NRT/TLOG/PULL replicas, document routing, leader election, the Collections API, the Apache Solr Operator on Kubernetes, and monitoring.
- Chapter 5 — Relevance Engineering measures and improves what queries return: judgment lists with Quepid and NDCG, the signals plane, learning-to-rank with the ltr module, the relatedness() aggregation, and editorial overrides with the Query Elevation Component.
- Chapter 6 — Solr MCP exposes a Solr collection to an LLM through the Model Context Protocol: server architecture, the four MCP primitives, STDIO vs. HTTP transports, OAuth2 security, observability with the LGTM stack, and how to design tools an LLM can use well.
- Chapter 7 — Alternatives compares Solr to Elasticsearch and OpenSearch in 2026 — licensing history, architecture, API surface, performance reality, migration considerations, and the MCP servers offered by all three engines (§7.9).

Each chapter opens with a callout box listing its prerequisites and what you will learn. For a single topic, use the Index.

Conventions

- Monospaced text denotes code, field names, configuration keys, file paths, JIRA tickets, and shell commands.

- Production tip — xxx. Blue-bar callouts mark hard-won operational advice that contradicts the obvious default. Read these even if you skim the surrounding prose.
- Solr 9 difference — xxx. Green-bar callouts mark behaviour that changed between Solr 9 and Solr 10. Skip these if you are starting fresh on Solr 10.
- Citations. Authoritative references (Apache Solr Reference Guide, JIRA tickets, Lucene Javadoc, published papers) are linked inline on first use and gathered in the References section at the end of the book.
- Figure and table numbering. Figures and tables are numbered per chapter (Figure 6.1, Table 6.3) and cross-referenced inline. The PDF, HTML, and EPUB builds all use the same numbering.

Diagrams

All diagrams are written in Mermaid and rendered at build time. They diff cleanly in version control (no binary blobs), survive Markdown → PDF / HTML / EPUB conversion without hand re-rendering, and you can copy any of them into https://mermaid.live/ to edit and adapt for your own designs.

The diagram style is intentionally consistent:

- Flowcharts (flowchart LR for left-to-right pipelines, flowchart TB for top-down state machines) show data flow and dependency direction.
- Sequence diagrams (sequenceDiagram) show wire-level interactions across nodes — used for leader election, distributed query flow, and update sequences.
- Box labels describe what a thing is; arrow labels describe what crosses the boundary. If a label describes an action ("commit", "fan out", "watch fires"), it lives on the arrow, not the box.
- Bracket conventions: [square brackets] for processes/components, (rounded) for data stores, ((double circles)) for terminal/start states. Same as standard UML/flowchart practice.

When a diagram is dense (e.g., the HNSW graph walk in §3.6), it is intentionally so — the alternative would be three smaller diagrams that don't fit on one page. Read it slowly.

A note on the title

I chose "Practical Engineering Handbook" deliberately. This is not a reference manual — the official Apache Solr Reference Guide already fills that role excellently and is updated with every release. It is also not a relevance-theory book — Relevant Search (Turnbull & Berryman, 2016) and AI-Powered Search (Grainger, Turnbull, Irwin, 2025) cover that territory better than I

could in one chapter. This book is the missing middle: the operational, code-level, "what do I actually type" companion that takes a working JVM engineer from "I have heard of Solr" to "I have shipped Solr to production."

Acknowledgements

This book was written with Claude (Anthropic) as a drafting and review partner. All technical claims have been independently verified against the Apache Solr 10 source, the reference guide, and live testing where applicable. Errors are mine.

Thanks to the Apache Solr committers and the Lucene community for two decades of patient stewardship. Thanks to the authors of Solr in Action, Relevant Search, and AI-Powered Search — this book stands on their shoulders.

Copyright and edition

- Code repository: https://github.com/adityamparikh/solr-handbook-code (planned for v1.0 release)
- Errata and discussion: https://github.com/adityamparikh/solr-handbook-code/issues
- Author contact: aditya.m.parikh@gmail.com

ISBN: not yet assigned (pre-print edition).

Why Solr?

What this chapter covers and why it matters. Before you adopt any search engine, you should be able to defend, in one paragraph, why a B-tree index over VARCHAR columns is not the right tool for the job. This chapter builds that intuition with a concrete TV-shows catalog example, walks through the inverted-index data structure as it actually lives on disk in Lucene, and shows step-by-step how each kind of query (term, phrase, prefix, wildcard, fuzzy, range, boolean) is executed against those files. It ends with a decision framework: Solr vs. SQL vs. a vector database vs. PostgreSQL FTS.

Prerequisites: Working knowledge of SQL and B-tree indexes. No prior Solr experience needed.

You will learn: What an inverted index is, the five data structures inside a Lucene segment, how each Solr query type maps to those structures, the cost model that makes wildcards expensive and term queries cheap, and when Solr is and isn't the right tool.

What you should not skip: §1.3 (the five data structures). Everything in later chapters refers back to it.

1.1 The naive approach: SQL LIKE

Imagine a shows table with 50 million rows — every TV show ever made, every region, every platform. A user types stranger things into your search bar. The naive query is:

```
SELECT id, title, rating
FROM   shows
WHERE  title ILIKE '%stranger things%'
ORDER  BY rating DESC
LIMIT  20;
```

This works for a demo and fails in production for four independent reasons:

1. It cannot use a B-tree index. B-trees on title accelerate prefix matches (title LIKE 'stranger%') but a leading wildcard (%stranger) forces a sequential scan. At 50M rows that is hundreds of milliseconds to many seconds.
2. It has no notion of relevance. A show titled ”Stranger Things” and a show titled ”Stranger in a Strange Land” are ranked identically — both match the substring. ORDER BY rating DESC is a poor proxy for “what the user meant”.
3. It does not understand language. A query for stranger does not match Strangers. kids does not match kid's. sci fi does not match sci-fi. There is no stemming, case folding, possessive stripping, or synonym expansion (so a user typing scifi will not match science fiction).
4. It does not understand multi-term intent. stranger things matches the literal substring but a synopsis containing the words separately — ”a stranger arrives and strange things start happening” — also matches, with no scoring penalty for the gap.

Production search engines solve all four problems with a different data structure: the inverted index.

1.2 The inverted index, conceptually

A B-tree maps row → column value. An inverted index maps term → list of rows containing that term. The asymmetry is the entire point.

Three show documents:

docId	title
1	“Stranger Things Season 4”
2	“The Crown Season 6”
3	“Strange New Worlds”

After lower-casing, tokenizing on whitespace, removing stop words, and stemming (English Porter):

docId	terms
1	stranger, thing, season, 4
2	crown, season, 6
3	strang, new, world

We “invert” this into a term dictionary plus posting lists:

```
term       df   postings (docId → tf, positions)
---------  ---  ----------------------------------------
4          1    1→(1,[4])
6          1    2→(1,[3])
crown      1    2→(1,[1])
new        1    3→(1,[2])
season     2    1→(1,[3]), 2→(1,[2])
strang     1    3→(1,[1])
stranger   1    1→(1,[1])
thing      1    1→(1,[2])
world      1    3→(1,[3])
```

df is document frequency (how many documents contain the term). For each match Lucene stores tf (term frequency in that doc) and optionally positions, payloads, and offsets.

Looking up stranger is an O(log V) operation in the term dictionary (where V is vocabulary size — typically much smaller than the document count); the posting list itself is a sorted, delta-encoded, block-compressed stream of doc IDs. The conjunction stranger AND thing is a leap-frog intersection of two sorted streams, skipping with auxiliary skip-lists. The whole operation is fast even at billions of documents. Notice also that Porter stemming maps both strange and strangers to strang — but not stranger, which stays as its own root (stranger). This is a real quirk of the English Porter stemmer and a recurring source of "why didn't my synonym work?" confusion (covered in §2.4).

1.3 What's actually in a Lucene index

The conceptual picture above is what every "inverted index in 5 minutes" blog post draws. In practice, a Solr index is more than just term → docs. To answer real queries — with ranking, sorting, faceting, phrase matching, and returning fields to the client — Lucene maintains several different data structures side by side. You don't need to know their on-disk layout to be effective with Solr, but you do need to know what's there and why.

Segments: the unit of storage

A Lucene index is a directory of segments. Each segment is a self-contained mini-index, write-once and immutable. New documents land in new segments. Deletes mark old documents as tombstones without rewriting them. A background MergePolicy periodically consolidates many small segments into fewer larger ones, and physically drops tombstoned docs in the process.

The implication: a Solr “index” is really N independent indexes, and every query runs against all of them in parallel and merges the results. That is also why lots of tiny segments hurts — fixed per-segment overhead multiplies.

The five data structures a Solr query touches

A single segment contains (at least) five distinct structures, each optimized for a different access pattern:

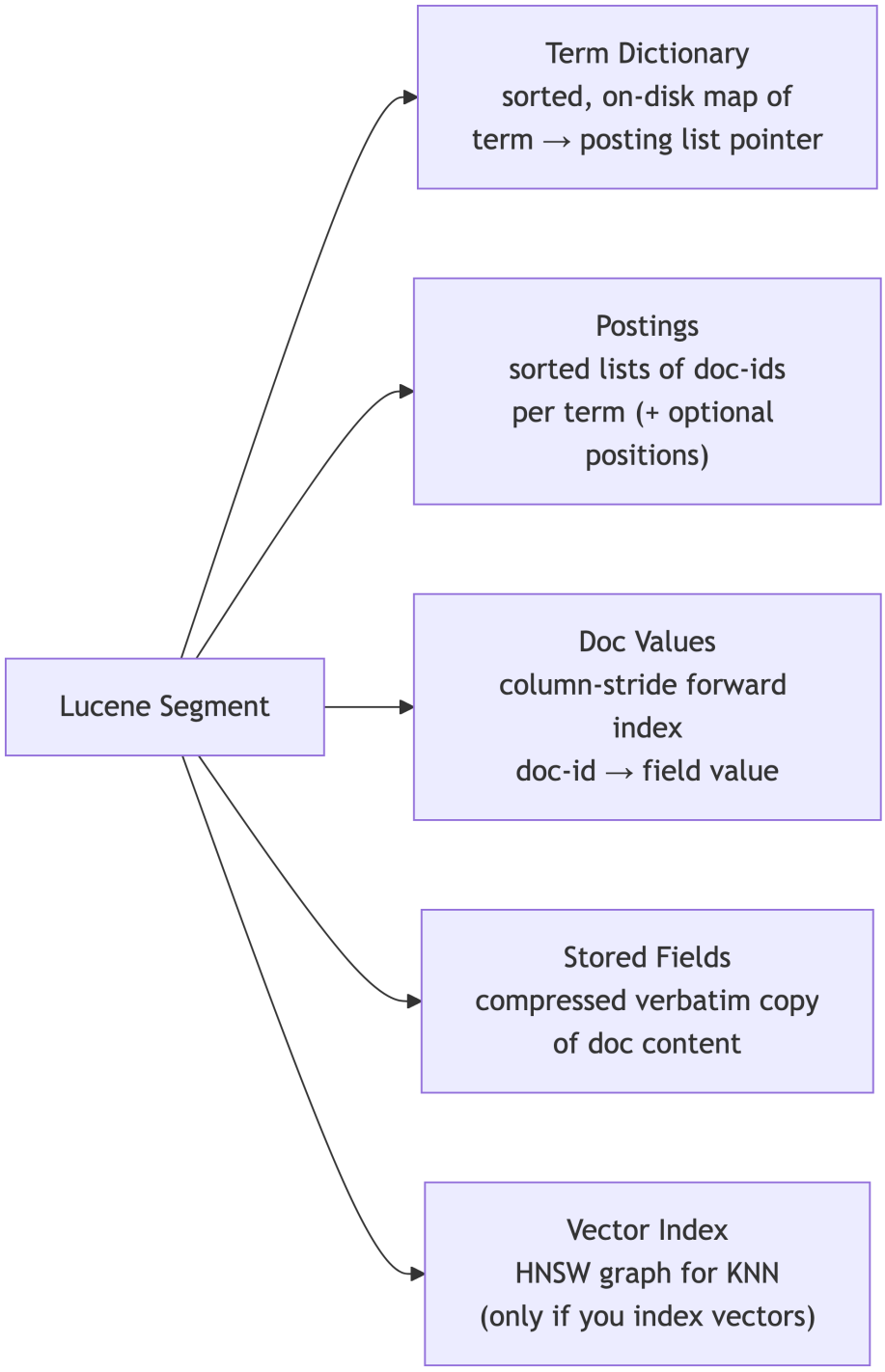

Figure 1: The five distinct data structures inside one immutable Lucene segment and what each one is optimized for.

Table 1: The five data structures inside a Lucene segment, what each one is for, and an example access pattern.

Structure	Used by	Example access
Term dictionary	Almost every query	"Find the entry for stranger and tell me where its posting list is."
Postings	Every text query	"Give me the sorted doc-ids that contain stranger."
Doc values	Sorting, faceting, function queries	"For doc 42, what are its platforms values?"
Stored fields	Returning results to the client	"Return the verbatim title and description of doc 42."
Vector index	Semantic / KNN queries	"Find the 20 vectors closest to this query vector."

The term dictionary is sorted, which is why prefix queries (appl*) are cheap and leading-wildcards (*pple) are expensive — covered in §1.4. The posting list for a term is sorted by doc-id and skip-list-indexed, which is why AND-ing two posting lists ("leap-frog intersection") is fast. Doc values are a column-store — one column per field, laid out doc-id by doc-id — which is why faceting and sorting are cheap when you have docValues=true and brutal when you don't. Stored fields are compressed in chunks, which is why returning huge rows=1000 pages is expensive even when the search itself was fast.

You will see references to specific files (.tim, .doc, .dvd, .fdt, .nvd, .vec etc.) in Solr logs, segment listings, and JIRA tickets. They map directly to the structures above. You almost never need to think about them at that level.

Deletes are soft

A document is "deleted" by flipping a bit in a per-segment live-docs bitset. The doc stays on disk until a merge rewrites the segment. This is why deletes are essentially free at write time and why numDocs (live) and maxDoc (live + deleted) diverge until merging catches up.

Going deeper

If you want the actual file format, encoding tricks (FST term index, PFOR-delta posting compression, skip lists), and Lucene codec internals, the canonical references are:

- Lucene file formats reference — org.apache.lucene.codecs.lucene103 package summary (matches the codec used by Solr 10's Lucene 10.3).
- Sease blog: Exploring Solr Internals — The Lucene Inverted Index — sease.io/2015/07/exploring-solr-internals-lucene.html.
- Michael McCandless's "Changing Bits" blog — long-running deep dives on Lucene internals at blog.mikemccandless.com.
- Jedr Blaszyk: "Exploring Apache Lucene — Part 1: The Index" — j.blaszyk.me/tech-blog/exploring-apache-lucene-index/.

For the rest of this book, knowing that these five structures exist — and what each is for — is enough.

1.4 How each query type actually executes

Now we can walk through the common query types and explain which files they read and why their cost differs.

1.4.1 Term query: q=platforms:Netflix

The simplest case.

1. The query string is run through the field's analyzer chain only if the field has one. A solr.TextField (e.g. text_en) lowercases, tokenizes, and stems. A solr.StrField does none of that — the input bytes are matched against the indexed bytes exactly, case included. So platforms:Netflix on a string field finds documents indexed with "Netflix" but not documents indexed with "netflix" or "NETFLIX". This catches almost every new Solr user once: facet/sort fields are typically StrField, and "why doesn't my platform filter match?" is usually a casing mismatch at index time.
2. .tip lookup → .tim block → posting list pointer.
3. Iterate .doc postings; for each, score with BM25 using .nvd norms; skip if not in .liv.

Cost: linear in the number of matching docs. Cheap.

Production tip — normalize casing at the source. For facet/filter fields, lowercase (or otherwise canonicalize) in the ingestion layer before sending to Solr. Don't try to fix it with a solr.LowerCaseFilterFactory-only field type — that path tokenizes, which breaks exact equality matching used by facets. The right pattern is a StrField mirror populated by your application code or a copyField from a KeywordTokenizerFactory + LowerCaseFilterFactory pipeline. Either way, the bytes you index and the bytes you query must match.

1.4.2 Boolean query: q=stranger AND things, q=stranger OR things, q=stranger NOT love

- AND = leap-frog intersection of posting lists (§1.3.2). Cost grows with the smaller posting list because the smaller one drives the cursor and the larger one gets skipped.
- OR = priority-queue merge of posting lists. Cost grows with the sum of posting-list sizes.
- NOT = AND with the complement; in practice Lucene uses the same iterator framework and just skips matches.

Practical implication: put the most selective clause first if you author the query; the optimizer will reorder when it has stats, but the cheapest disjunction is the one with the smallest combined posting list. In a shows corpus stranger (rare title term) is far more selective than things (common), so the intersection is driven by the much shorter posting list of stranger.

1.4.3 Phrase query: q=title:"stranger things"

Now we need positions.

1. Look up both terms (.tip + .tim) and get their .doc and .pos pointers.
2. Run the AND intersection as before to find candidate docs.
3. For each candidate, decode the .pos entries for both terms in that doc. Look for a position p_stranger and a position p_things such that p_things == p_stranger + 1 (or within slop for "stranger things"~2).
4. Score only the docs that pass the position check.

Cost: AND intersection cost + position-stream decode per candidate. Phrase is meaningfully more expensive than the underlying AND. Implication: index positions only on fields where you need phrase/proximity. The omitPositions="true" field-type attribute on string and most analytics fields drops .pos entirely.

1.4.4 Prefix query: q=title:strang*

Prefix queries leverage the fact that the term dictionary is sorted.

1. Find the first term in the dictionary that starts with strang.
2. Walk forward, collecting terms as long as they start with strang — strange, stranger, strangers, strangely, strangelove, …
3. OR all their posting lists together.

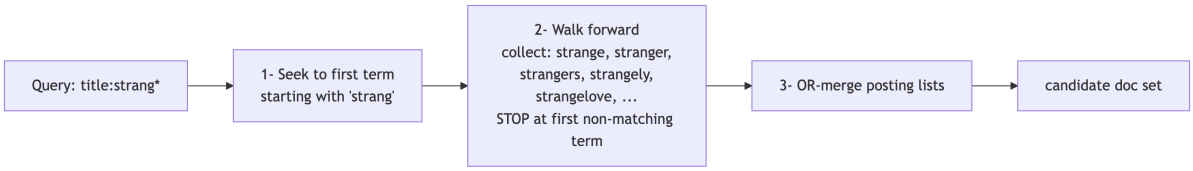

Figure 2: How a trailing-wildcard prefix query walks the sorted term dictionary forward and OR-merges the matching posting lists.

Cost: depends on how many terms share the prefix. strang* might match 50 terms; s* might match 50,000. Lucene auto-rewrites broad prefix queries into a constant-score form to keep this bounded.

> Production tip — the maxBooleanClauses trap. Internally a prefix query expands into an OR of all matched terms. Solr caps the clause count (default 1024, configurable in solrconfig.xml). Very broad prefixes can throw TooManyClauses or silently fall back to a non-scored rewrite. If you intentionally search broad prefixes, either raise the cap or restrict the prefix.

1.4.5 Wildcard query: q=title:t?ings and the leading-wildcard problem

The * (any sequence) and ? (single character) wildcards share the same execution model as prefix queries, but the cost structure depends entirely on where the wildcard is.

Three cases, in order of cost:

Case 1: Trailing wildcard q=title:strang* — already covered above. Seek to the leading literal, scan forward. Cheap.

Case 2: Embedded wildcard q=title:t?ings — Lucene seeks to the longest leading literal (here, t), then scans every term beginning with t and tests each against the wildcard pattern. Same machinery powers regex queries.

Case 3: Leading wildcard q=title:*hings — the pathological case. There is no leading literal at all, so there is nowhere to seek. Lucene has to scan every term in the dictionary and test each against the pattern.

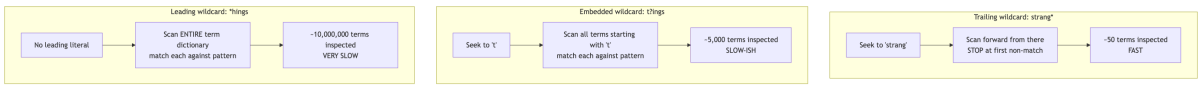

Figure 3: Why wildcard cost depends on the leading literal: trailing wildcards scan ~50 terms, embedded ones thousands, and leading wildcards must scan every term in the dictionary.

Cost summary: a leading literal of even one character (t?ings) keeps wildcard queries manageable on most corpora; a truly leading wildcard (*hings) does not.

The ReversedWildcardFilterFactory fix

If you need leading-wildcard support, the canonical fix is to index every term twice — once normally and once reversed. A leading wildcard against the normal form becomes a trailing wildcard against the reversed form, which is fast.

```
<fieldType name="text_rev" class="solr.TextField" positionIncrementGap="100">
  <analyzer type="index">
    <tokenizer class="solr.StandardTokenizerFactory"/>
    <filter class="solr.LowerCaseFilterFactory"/>
    <!-- Index each term plus its reverse, with a marker character so the
         query parser can tell them apart. Marker defaults to U+0001. -->
    <filter class="solr.ReversedWildcardFilterFactory"
            withOriginal="true"
            maxPosAsterisk="3"
            maxPosQuestion="2"
            maxFractionAsterisk="0.33"/>
  </analyzer>
  <analyzer type="query">
    <tokenizer class="solr.StandardTokenizerFactory"/>
    <filter class="solr.LowerCaseFilterFactory"/>
    <!-- Note: do NOT add ReversedWildcardFilter at query time.
         The query parser detects leading wildcards and rewrites
         them to use the reversed terms automatically. -->
  </analyzer>
</fieldType>
```

What the parameters mean:

- withOriginal=true — index both the original term and its reverse (otherwise normal queries break).
- maxPosAsterisk=3 — only rewrite to use reversed terms if the * is within the first 3 characters. Past that, a normal forward scan is cheap enough.
- maxPosQuestion=2 — same for ?.
- maxFractionAsterisk=0.33 — only rewrite if the * is within the first third of the query string.

At index time, indexing ”things” produces two tokens in the inverted index: things and \u0001sgniht (the marker plus the reverse). At query time, parsing *hings triggers the rewrite: Lucene constructs sgnih* against the reversed terms and runs it as a fast trailing-wildcard query.

Trade-off: index size grows because every term appears twice, and merge throughput drops accordingly. Worth it for fields where leading-wildcard is a real user need; not worth it as a default.

When n-grams are a better answer

ReversedWildcardFilterFactory solves leading wildcards but not arbitrary infix matching. For “match any substring of a show code or short ID” or “type-ahead with infix” use n-gram tokenization instead:

```
<fieldType name=”text_ngram” class=”solr.TextField”>
  <analyzer type=”index”>
    <tokenizer class=”solr.StandardTokenizerFactory”/>
    <filter class=”solr.LowerCaseFilterFactory”/>
    <filter class=”solr.NGramFilterFactory” minGramSize=”3” maxGramSize=”6”/>
  </analyzer>
  <analyzer type=”query”>
    <tokenizer class=”solr.StandardTokenizerFactory”/>
    <filter class=”solr.LowerCaseFilterFactory”/>
    <!-- query-time is NOT n-gram'd: queries are matched as whole tokens
         against the indexed n-grams. -->
  </analyzer>
</fieldType>
```

Indexing ”crown” with min=3 max=6 produces the tokens cro, crow, crown, row, rown, own. A query for row now matches without any wildcard at all — it’s a regular term query. For prefix-only autocomplete use EdgeNGramFilterFactory instead, which only emits prefixes (c, cr, cro, crow, crown) and avoids the explosion of internal n-grams.

Trade-off: n-grams produce 5–10× term volume and similarly grow the index. They eliminate query-time wildcard cost entirely, at the price of larger indexes and slower indexing.

Which approach when

Table 2: Which schema pattern to reach for given the kind of wildcard, substring, or partial-match access you need.

Need	Approach
Trailing wildcard, occasional use	Plain text_* field, no special config
Leading wildcard, occasional use	ReversedWildcardFilterFactory on a dedicated field
Infix matching on short fields (show IDs, episode codes like S04E08)	NGramFilterFactory (min=2, max=4)
Prefix autocomplete / type-ahead	EdgeNGramFilterFactory (min=1, max=20)
Substring search on long text	Reconsider — usually a phrase/proximity query is what the user actually wants
Misspelling tolerance	Spell checker (§3.7) or fuzzy ~, not wildcards

1.4.6 Fuzzy query: q=title:thinngs~2

Fuzzy matches all terms within a small edit distance of thinngs (a few character insertions, deletions, or substitutions — here the extra n is a typo). Internally Lucene uses the term dictionary's sorted structure to enumerate candidate terms efficiently — it does not compare against every term one by one. Useful when a user mistypes breking bad instead of breaking bad.

Cost: dominated by how many terms fall within the edit-distance window. Short query terms with ~2 produce huge candidate sets; longer terms produce small ones. Setting fuzzyPrefixLength=1 or 2 (requiring the first character(s) to match exactly) is the standard way to keep fuzzy queries tractable on large dictionaries.

If you need misspelling tolerance for user input, the spell checker (§3.7) is almost always a better answer than fuzzy queries.

1.4.7 Range query: q=release_year:[2015 TO 2020]

There are two different stories depending on the field type.

For string ranges (e.g., id:[A TO M]): walk the term dictionary forward from the lower bound, OR all postings, stop at the upper bound. Same machinery as prefix.

For numeric Point fields (pint, pfloat, pdate — default in Solr 9+): the data is not in the inverted index at all. Numeric points use a separate indexed structure optimized for range queries. A range query descends to the relevant subtrees and emits matching doc ids directly — much faster than equivalent term ranges, especially for narrow ranges over wide values. This is why the legacy TrieIntField was deprecated in favor of IntPointField.

If you want the data structure behind point fields, search for "Lucene BKD tree".

1.4.8 Function queries: q={!func}recip(ms(NOW,added_at),...)

Function queries don't read the inverted index at all — they read doc values (the column-store described in §1.3). The function tree is evaluated per doc, producing a score. Typically used as a boost rather than a primary query, because they have no candidate-filtering power (every doc "matches" with some score).

1.4.9 Vector (KNN) query: q={!knn f=embedding topK=20}[...]

Vector queries skip the inverted index entirely. They walk the HNSW graph (the vector index from §1.3) to find the top-K nearest neighbors to a query vector. Covered conceptually in Chapter 3.

1.4.10 Summary: what each query type uses

Table 3: Which Lucene data structures each Solr query type reads, and the cost notes that follow from it.

Query type	What it reads	Notes
Term	Term dictionary + postings	Cheap
Boolean (AND/OR/NOT)	Same, for each clause	AND = leap-frog intersection
Phrase / proximity	Adds positions	More expensive than the underlying AND
Prefix	Term dictionary forward scan + postings	strang* cheap, s* expensive
Wildcard	Same, scoped by leading literal	Leading wildcards (*hings) are expensive
Fuzzy	Term dictionary, edit-distance window	Use fuzzyPrefixLength to bound cost
Numeric range	Numeric point index (separate structure)	Very fast
String range	Term dictionary forward scan	Like prefix
Function query (boost)	Doc values only	No candidate filtering
Sort / facet	Doc values	Need docValues=true to be efficient
KNN	HNSW vector index	Skips the inverted index

Internalizing this table is the difference between guessing about Solr performance and reasoning about it.

1.5 BM25 in one paragraph, no math

Each posting contributes a score; the document's score is the sum across query terms. Three intuitive levers control that contribution:

- Term frequency (TF) with saturation. Mentioning "stranger" 50 times in a synopsis is not 50× better than mentioning it once. BM25 saturates contribution as TF grows; the k1 parameter (default 1.2) controls where on the curve saturation happens. Counter-intuitively, smaller k1 means TF saturates faster — the curve flattens sooner, so extra occurrences barely help. Larger k1 keeps rewarding additional mentions for longer. If you read "k1 controls saturation rate," remember that the relationship is inverse to most people's first guess.
- Inverse document frequency (IDF). Rare terms ("mindhunter") count more than common terms ("season"). stranger matching in a corpus where 1% of show titles contain it is far more diagnostic than the matching where 99% do.
- Length normalization. A 4-word title "Stranger Things Season 4" containing "stranger things" is more relevant than a 2,000-word synopsis that happens to contain the phrase. The b parameter (default 0.75, range [0, 1]) controls how aggressively long documents are penalized. b=0 disables length normalization entirely; b=1 applies it at full strength. This is the reason for the per-field-per-doc length byte in .nvd.

We revisit BM25 with the actual scoring breakdown in Chapter 3.

1.6 When to use Solr vs. SQL vs. a vector DB vs. Postgres FTS

Table 4: A decision matrix for picking between Solr, a relational database, a vector database, and Postgres FTS based on the workload.

Use case	Pick
Transactional CRUD; exact lookups; sub-millisecond joins	Relational DB (Postgres/MySQL)
Search inside a single Postgres table, < a few million rows, no faceting needs	Postgres FTS (tsvector + GIN)
Multi-tenant search, faceting, > 50M docs, complex relevance, hybrid lexical+vector	Solr / OpenSearch / Elasticsearch

Use case	Pick
Pure semantic similarity over embeddings, no keyword needs	Dedicated vector DB (Qdrant, Weaviate, Milvus)
Logs, metrics, observability	OpenSearch / Elasticsearch (mature ecosystem)
Large-scale enterprise search with strict OSS licensing and on-prem	Solr

A useful default for greenfield search products in 2026 is: Solr if you want full Apache 2.0 + first-class faceting + on-prem control; OpenSearch if you want a richer observability ecosystem; Postgres FTS if your search needs are modest and you already have Postgres.

1.7 PostgreSQL Full-Text Search: a brief comparison

Postgres ships with real full-text search via tsvector and tsquery:

```
ALTER TABLE shows ADD COLUMN tsv tsvector
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || description)) STORED;
CREATE INDEX shows_tsv_idx ON shows USING GIN (tsv);

SELECT id, title, ts_rank(tsv, q) AS rank
FROM   shows, websearch_to_tsquery('english', 'stranger things') q
WHERE  tsv @@ q
ORDER  BY rank DESC
LIMIT  20;
```

This is genuinely good. You get stemming, stop-words, ranking, prefix matching, and indexed performance. Under the hood the GIN (Generalized Inverted Index) really is an inverted index — the trade-off is operational, not architectural. What Postgres FTS doesn't give you: faceted navigation with sub-second aggregates over hundreds of millions of documents; horizontal sharding tied to query routing; native vector + lexical hybrid retrieval with rank fusion; pluggable analyzer chains with synonyms, ICU folding, shingles; learning-to-rank and feature stores. For "find blog posts that mention X" Postgres FTS is fine. For "power the search bar of a streaming catalog with 50M shows + episodes across 30 genres and 8 platforms" it is the wrong tool.

Exercises

1. Hand-build a posting list. Take three TV-show titles (any genre, any platform) and produce, by hand: the lowercased tokens for each, the term dictionary (sorted unique terms), and the posting lists with positions. Now answer: which terms have df=1? Which would be most diagnostic if a user queried for them?
2. Predict the cost. For each of these queries against a corpus of 50M show records, rank them from cheapest to most expensive without running them, and justify your ordering using §1.4: (a) q=platforms:Netflix, (b) q=title:"stranger things", (c) q=title:strang*, (d) q=title:*hings, (e) q=title:thinngs~2, (f) q=release_year:[2015 TO 2020]. Then, if you have access to a Solr cluster, run them with debug=timing and compare your predictions to reality.
3. Pick the right tool. A team wants to add search over a 2-million-row Postgres episodes table. Acceptance criteria: support stemming, return results in <200 ms, support boolean queries. Do they need Solr? Justify using §1.6 and §1.7. Now change the requirement to "support 50 simultaneous facet aggregations on genre, platform, release year, and language." Same answer?

1.8 Chapter summary

- Search needs a different data structure than transactional storage. The inverted index maps term → docs, which makes multi-term, ranked, language-aware retrieval cheap.
- A Solr index is built from immutable segments. Each segment holds five distinct structures: a sorted term dictionary, postings (sorted doc-id lists per term, with skip lists for fast intersection), doc values (a column-store for sorting/faceting/function queries), stored fields (compressed verbatim copies for retrieval), and a vector index (HNSW graph for KNN).
- Different query types touch different subsets of these structures. Knowing which structure each query reads is the foundation of reasoning about performance.
- Solr's value over LIKE '%...%' is not "faster substring search" — it is fundamentally different semantics (relevance, language, facets) backed by a fundamentally different storage model.
- Postgres FTS is a real option for small-to-medium loads; Solr/OpenSearch/Elasticsearch are the right answer when you need scale, facets, or hybrid vector + lexical retrieval.

Indexing

What this chapter covers and why it matters. Indexing is where most production search problems are actually born — bad schemas, bad analysis chains, bad partial-update strategies, and risky reindexing patterns cause most of the relevance and performance fires you will fight. This chapter starts with the user-facing unit of data in Solr (the collection), then walks through schema design, field types, analyzers, SolrJ bulk indexing, the three flavors of partial update (full replace, atomic, in-place), zero-downtime reindexing patterns, and the commit model.

Prerequisites: Chapter 1 — specifically §1.3 (the five data structures) and §1.4 (which structure each query type reads). You will not get the field-property tradeoffs without it.

You will learn: How to design a Solr schema that you won't regret in six months, how the analyzer chain transforms text at index and query time, how to index from Java with SolrJ at production throughput, how to test against a real Solr with Testcontainers, when to use atomic vs. in-place updates and the trap of silently falling back, how to do zero-downtime reindexes with collection aliases, and how to think about commits.

What you should not skip: §2.8 (partial updates — the single most misunderstood topic in production Solr) and §2.9 (reindexing — when you must, when you don't, and how).

2.0 Running Solr locally

Every example in this chapter assumes a Solr 10 server you can reach at http://localhost:8983. There are three reasonable ways to get one running on a developer laptop; pick the one that matches how you already work.

Install paths

Table 1: Three install paths for a local Solr 10 server — Docker, Homebrew, and the binary archive — and when each one fits.

Method	When to use	Command
Docker	You want a repeatable, isolated runtime — the default recommendation.	docker run -p 8983:8983 apache/solr:10.0.0 solr-precreate shows
Homebrew (macOS)	You want Solr managed alongside your other dev tools.	brew install solr && brew services start solr
Archive	You want to read the on-disk layout, edit solr.in.sh, or run on Linux without Docker.	Download from solr.apache.org/downloads, then bin/solr start -c

The Docker image and the archive both default to SOLR_HOME=/var/solr/data (or the Linux equivalent). Homebrew uses /opt/homebrew/var/lib/solr on Apple Silicon. Know where it is — when an integration test misbehaves, the on-disk index is the source of truth.

Starting Solr

In Solr 10, bin/solr start defaults to SolrCloud mode with an embedded ZooKeeper on port 9983 — perfect for development. Verify the cluster is up:

```
curl -s http://localhost:8983/solr/admin/info/system | jq .lucene.solr-spec-version
# → "10.0.0"
```

If that doesn't return 10.0.0, check solr-8983-console.log in the Solr home directory. For multi-node tests, the Testcontainers pattern in §2.7 is usually simpler than running multiple bin/solr processes by hand.

A tour of the admin UI

Solr ships with a web admin console at http://localhost:8983/solr/. You don't need it — every action it exposes is also a REST call — but in development it shortens many feedback loops. When you first land on the URL you get the Dashboard:

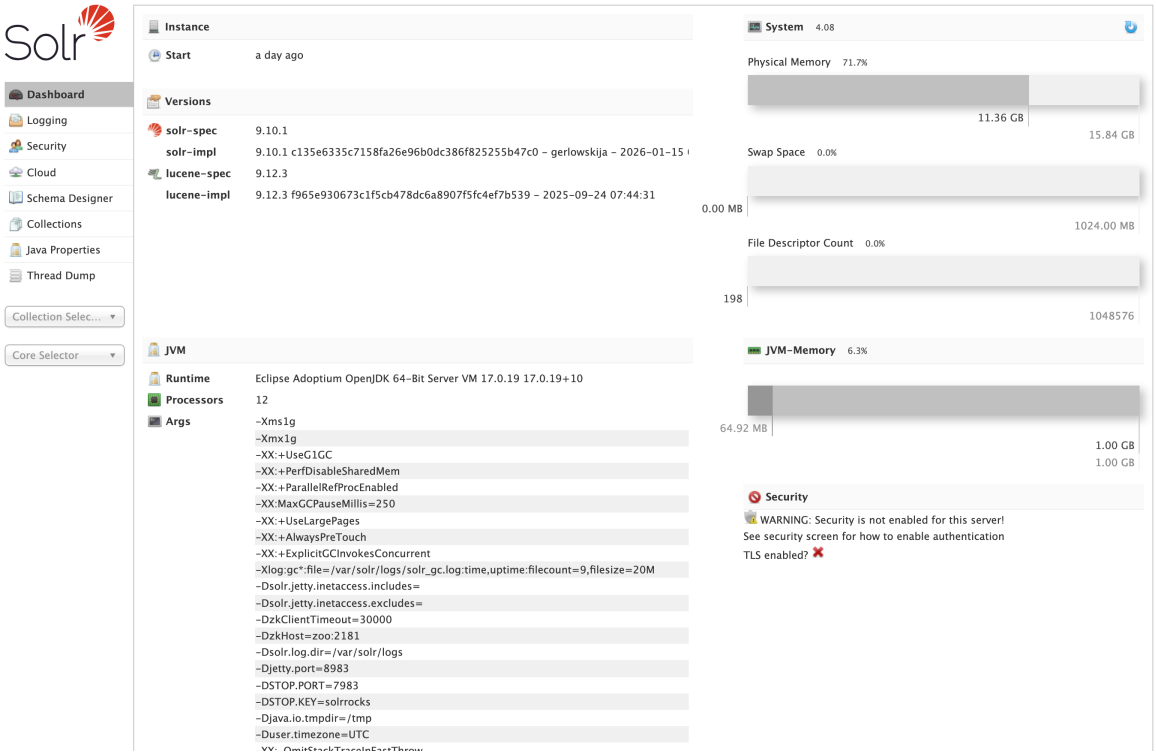

Figure 1: The Solr admin Dashboard — versions, JVM args, system memory, and the security warning that appears on a development node.

The tabs you'll actually use:

- Cloud — A live view of the cluster: nodes, collections, shards, replica states, and ZooKeeper contents. The Graph sub-view (Figure 2) is the fastest way to spot a replica stuck in recovering or down, and the Nodes sub-view exposes per-node CPU, heap, disk, and request rate.

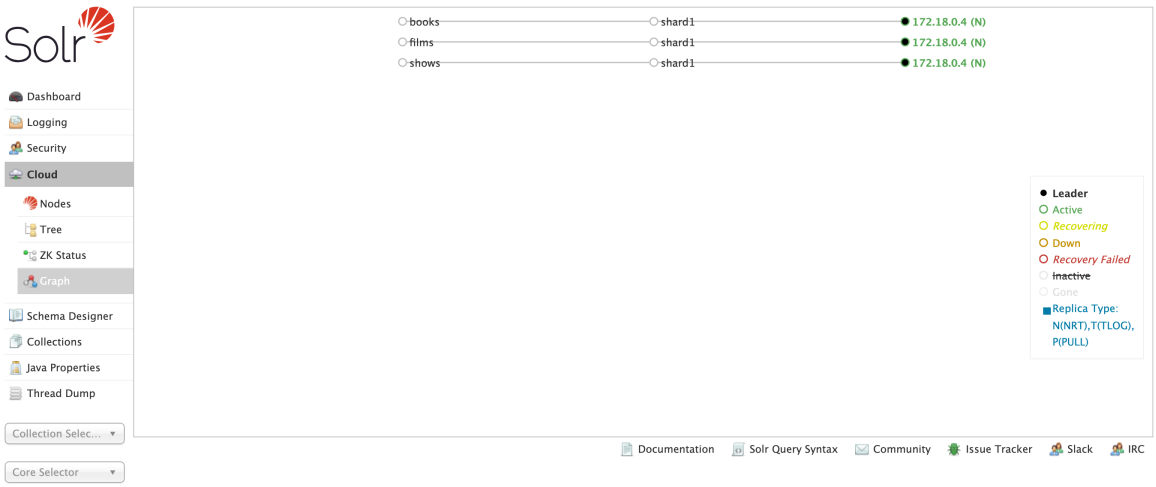

Figure 2: Cloud → Graph: one row per collection, expanded out to shards and replicas, colour-coded by leader / active / recovering / down state. The first thing to look at when something's wrong with the cluster.

- Collections — Create, delete, reload, and inspect collections. A form-driven UI over the Collections API (covered in §4.7).
- Collection selector (left sidebar) — Pick a collection (shows) and several per-collection tools open up:
  - Schema — Browse field definitions and field types, add new fields, see which dynamicField rule matched a given field. Editing here writes through the Schema API into managed-schema.xml.
  - Query — Run an ad-hoc /select request with q, fq, fl, sort, faceting, and debug (Figure 3). The form translates to the URL shown above the result; copy that URL when you want to script the same query.

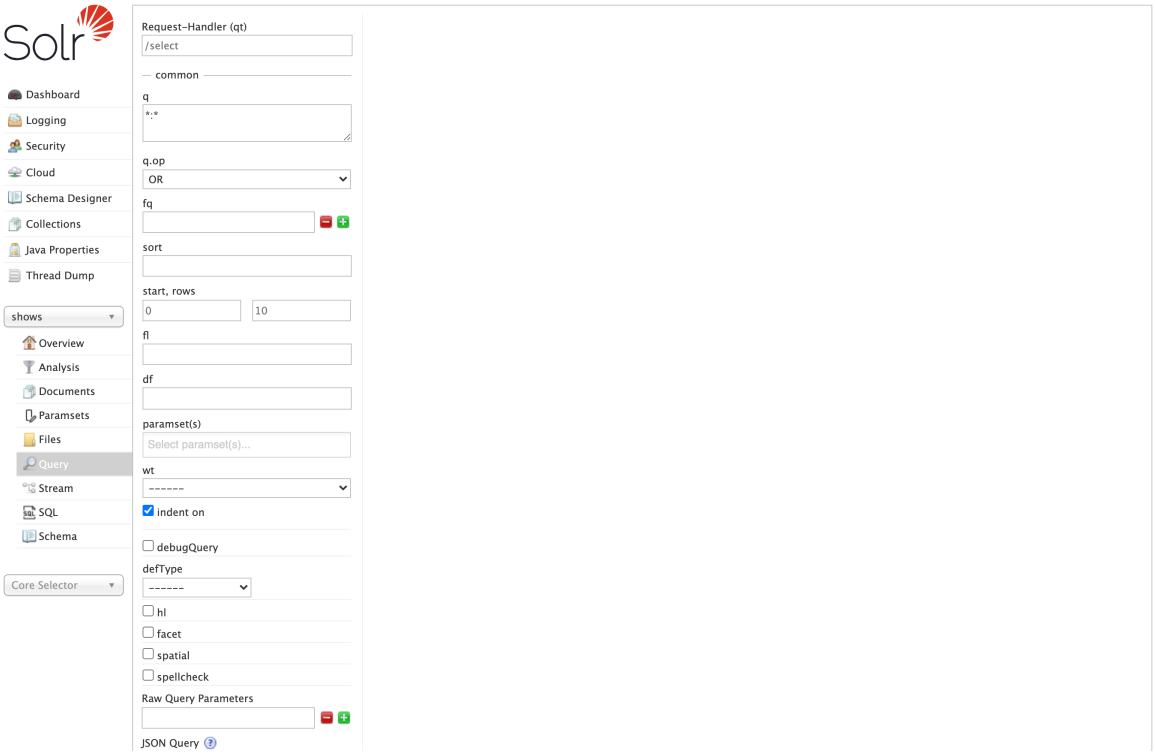

Figure 3: The Query form for the shows collection: separate inputs for q, q.op, fq, sort, start / rows, fl, df, wt, defType, plus checkboxes for hl, facet, spatial, spellcheck, and debugQuery. The generated URL appears above the result pane.

- Analysis — Type a value and see exactly which tokens come out of the index analyzer chain vs. the query analyzer chain (Figure 4). We use this in §2.4 to walk through the text_en field.

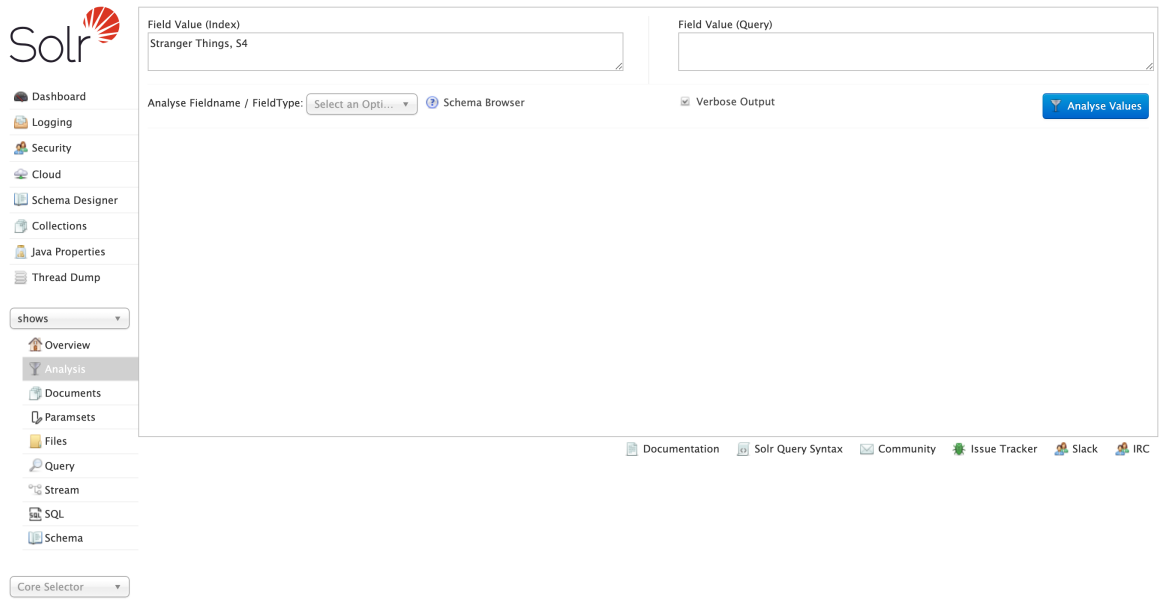

Figure 4: The Analysis tab with the §2.4 walkthrough input typed in. Pick a field name or type from the dropdown, click Analyse Values, and Solr renders the per-stage token output for both the index and query analyzer chains side by side.

- Documents — Add a single document via JSON, XML, or CSV. Handy for one-off testing.

  Production tip — don't expose the admin UI to the public internet. It's a development tool. In production, fence it off with a reverse proxy that requires authentication, or set SOLR_OPTS="-Dsolr.disableAdminUI=true" if you genuinely don't need it. The production observability stack lives in §4.13.

2.1 What is a collection?

A collection is Solr's user-facing unit of data. It is the thing you create, index documents into, query against, alias, back up, and delete. Everything you do as an application developer happens with respect to some collection.

The simplest accurate mental model: a collection is a named, schema-bearing logical index. "Products" is a collection. "User profiles" is a different collection. They have separate schemas, separate documents, and (almost always) separate query traffic.

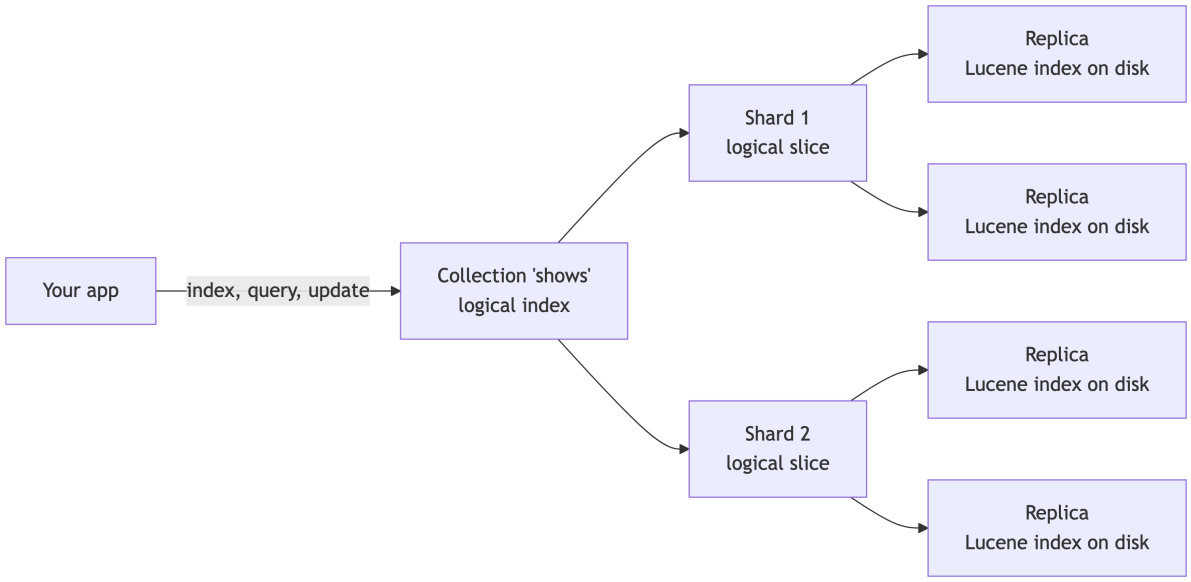

Figure 5: The collection-shard-replica hierarchy: one logical collection maps to multiple shards, each backed by several replica copies of a Lucene index on disk.

What a collection is made of

A collection has four configured pieces:

1. A schema. The field definitions covered in §2.2. The schema lives in a configset.
2. A configset. A bundle of files — managed-schema.xml, solrconfig.xml, synonyms.txt, stopword files — stored in ZooKeeper, shared across the collection's replicas. Configsets can be shared across collections.
3. A number of shards. A shard is a logical horizontal slice of the collection; each document lives in exactly one shard, decided at index time by a router (default: a hash of the document's id). At small scale you have one shard. At large scale you have many.
4. A number of replicas per shard. A replica is a physical copy of a shard's data, hosted on some Solr node. Multiple replicas exist for fault tolerance and read throughput. Each replica is, under the hood, a single self-contained Lucene index — the kind we walked through in Chapter 1.

The chain is therefore: collection → shards → replicas → Lucene index. The collection is the only level you have to think about for routine work; shards and replicas only enter the picture once you scale, and that's Chapter 4's job.

What's in a configset

The configset is the bundle of files that defines how a collection behaves. The same configset can back many collections, and in SolrCloud it lives in ZooKeeper under /configs/<name>/. The files you'll touch in this chapter:

Table 2: The files that live in a SolrCloud configset, what they configure, and the section of this chapter where you'll touch each one.

File	Purpose
managed-schema.xml	Field definitions, field types, copyField, dynamicField. Edited via the Schema API or admin UI (§2.2).
solrconfig.xml	Runtime tuning: request handlers, caches, the commit policy (§2.10), update-processor chain.
synonyms.txt	Synonym rules consumed by SynonymGraphFilterFactory (§2.4). Reload the collection after changing.
stopwords.txt, protwords.txt	Stopwords removed by StopFilter (§2.4); protected words kept verbatim by stemmers.
lang/*.txt	Per-language stopword and stemmer-exception files referenced by language-specific analyzers.

Configset management — uploading with bin/solr zk upconfig or the ConfigSets API, sharing across environments, versioning — is a SolrCloud topic and lives in §4.8.

Cores vs replicas vs collections

A reliable source of confusion. Three terms, distinct levels:

- Collection — the logical thing your application talks to.
- Replica — one copy of one shard of a collection, living on one node.
- Core — the runtime container inside a Solr node that hosts one Lucene index. Each replica is a core on disk. The term is a holdover from Solr's pre-SolrCloud "standalone" days, when you ran cores directly without the collection abstraction.

In SolrCloud you create collections (which creates cores under the hood); in standalone mode you create cores directly. Solr 10 defaults to SolrCloud, so collections are what you'll work with.

A minimal collection

In SolrCloud, you create a collection via the Collections API. From SolrJ:

```
import org.apache.solr.client.solrj.request.CollectionAdminRequest;

CollectionAdminRequest.createCollection("shows", "shows_config", 2, 2)
    .process(client);
// args: collection name, configset name, numShards, numReplicas-per-shard
```

That command tells Solr: "create a collection called shows, use the shows_config configset (which must already exist in ZooKeeper), split it across 2 shards, and put 2 replicas of each shard somewhere in the cluster." Solr handles placement.

Equivalently from bin/solr:

```
bin/solr create -c shows -d shows_config -s 2 -rf 2
```

For a single-node dev cluster, numShards=1 and numReplicas=1 is fine. Production starts at 1 shard × 3 replicas for fault tolerance, and grows from there based on data size and traffic.

When to use one collection vs. many

The single most common collection-design question: should I put this in its own collection or use a field to distinguish it?

Use separate collections when:

- The schemas would meaningfully differ (different field types, different analyzer chains, different language). Shows and user reviews almost always belong in separate collections.
- The query patterns are completely disjoint and you never join across them.
- The retention or lifecycle differs — daily playback events in time-routed collections, shows in a long-lived one.
- The security posture differs (one tenant's data must be physically isolatable).

Use one collection with a discriminator field when:

- The schemas are the same.
- You frequently query across the partitions ("show me all titles available on any platform in this region").
- The cardinality is high (one collection per user is almost never right; one collection per regional catalog sometimes is).

A pragmatic default: start with one collection per “kind of thing” in your domain, and handle the environment dimension with separate Solr clusters (dev / stage / prod) rather than env-suffixed collection names. The collection name stays shows in every environment; the parameter that changes is the cluster URL or ZooKeeper ensemble your client points at. Keeping the name stable means configset references, client code, dashboards, and reindex tooling don’t have to be parameterised per environment, and it removes a class of “wrong collection in this env” mistakes. Reach for the discriminator-field pattern when one-collection-per-thing would explode collection count beyond a few dozen.

What you can do without thinking about shards or replicas

For the rest of Chapter 2, you can treat a collection as a single logical index. Everything we cover — schema design, analyzers, indexing, partial updates, reindexing, commits — works the same whether the collection has 1 shard or 50, 1 replica or 6. The distributed plumbing only becomes relevant in Chapter 4.

2.2 Schema: the contract between your data and your index

A Solr collection has a schema that declares fields (name, type, properties) and field types (text_general, pdate, pint, etc.). Schemas come in two flavors:

- Managed schema (default since Solr 6, the only sensible choice today): edited via the Schema REST API or Admin UI; stored as managed-schema.xml in the configSet in ZooKeeper.
- Classic schema.xml: hand-edited file. Still legal but discouraged.
- Schemaless mode: fields are auto-created from the first document indexed. Convenient for prototyping; avoid in production because field type guessing produces surprises (e.g., ”01234” becomes a number and loses its leading zero).

  Production tip. Start in managed-schema mode with schemaless disabled. Define every field deliberately. Schema drift is a real ops headache; making field creation explicit prevents it.

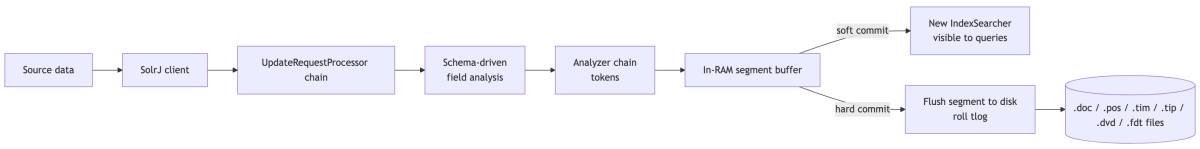

Figure 6: End-to-end indexing pipeline from source data through SolrJ, analysis, and the in-RAM buffer to soft-commit visibility and hard-commit durability on disk.

Document granularity

Before you write a single field, decide what one document represents. For a shows catalog there are three reasonable answers:

Table 3: Three document-granularity choices for a shows catalog — show, episode, or show-with-nested-episodes — with the query patterns each one serves well and the cost each one imposes.

Granularity	Each document is…	Query patterns it serves well	Cost
Show	A whole TV show or movie	"find a show like Stranger Things"; faceting by genre, platform, year; collapse by franchise	One row per show — small index, simple ranking. Episode-level signals must be aggregated upstream of Solr.
Episode	A single episode	"find the episode where X happens"; per-episode descriptions and air dates	10–100× more documents per show; show-level facets need a discriminator field or a roll-up query.
Show with nested episodes	A show document with episode child documents	Both of the above, in a single query	Higher write amplification on updates; the nested-document feature (§2.8) is more expensive than the flat alternatives.

The pull is always toward finer granularity, because each new use case can be answered with a smaller, more specific document. The push back is that finer granularity multiplies your document count, your indexing throughput requirements, and the complexity of any query that has to aggregate back up.

A reasonable default: pick the granularity that matches the unit your search result page renders. If the result is a card showing a whole show, model a show as one document. If the result is an episode card, model an episode. Use nested documents only when the user genuinely needs both views from the same index and you've accepted the operational cost.

This decision is the hardest one to reverse cheaply. Changing field types or analyzer chains requires a reindex (§2.9); changing document granularity requires a reindex and a rewrite of

every consuming query.

2.3 A concrete shows-catalog schema

```
<?xml version="1.0" encoding="UTF-8" ?>
<schema name="shows" version="1.7">

 <!-- ===== Field types ===== -->

 <fieldType name="string"  class="solr.StrField"   sortMissingLast="true"
 ↪  docValues="true"/>
 <fieldType name="boolean" class="solr.BoolField"  sortMissingLast="true"/>
 <fieldType name="pint"    class="solr.IntPointField"    docValues="true"/>
 <fieldType name="plong"   class="solr.LongPointField"   docValues="true"/>
 <fieldType name="pfloat"  class="solr.FloatPointField"  docValues="true"/>
 <fieldType name="pdouble" class="solr.DoublePointField" docValues="true"/>
 <fieldType name="pdate"   class="solr.DatePointField"   docValues="true"/>

 <!-- English text: tokenized, lowercased, stopworded, stemmed -->
 <fieldType name="text_en" class="solr.TextField" positionIncrementGap="100">
   <analyzer type="index">
     <tokenizer class="solr.StandardTokenizerFactory"/>
     <filter class="solr.EnglishPossessiveFilterFactory"/>
     <filter class="solr.LowerCaseFilterFactory"/>
     <filter class="solr.StopFilterFactory" words="lang/stopwords_en.txt"
 ↪   ignoreCase="true"/>
     <filter class="solr.SnowballPorterFilterFactory" language="English"/>
   </analyzer>
   <analyzer type="query">
     <tokenizer class="solr.StandardTokenizerFactory"/>
     <filter class="solr.EnglishPossessiveFilterFactory"/>
     <filter class="solr.LowerCaseFilterFactory"/>
     <filter class="solr.SynonymGraphFilterFactory" synonyms="synonyms.txt"
             ignoreCase="true" expand="true"/>
     <filter class="solr.StopFilterFactory" words="lang/stopwords_en.txt"
 ↪   ignoreCase="true"/>
     <filter class="solr.SnowballPorterFilterFactory" language="English"/>
   </analyzer>
 </fieldType>
```

```
<!-- Dense vector field for semantic search (384-dim sentence embeddings) -->
<fieldType name="knn_vector_384" class="solr.DenseVectorField"
           vectorDimension="384" similarityFunction="cosine"
           knnAlgorithm="hnsw" hnswMaxConnections="16" hnswBeamWidth="100"/>

<!-- ===== Fields ===== -->

<!-- Identity -->
<field name="id"              type="string"  indexed="true"  stored="true"
↪  required="true"/>

<!-- Title in English-analyzed form for relevance scoring, plus a string mirror
     for exact sort/facet (filled via copyField below). -->
<field name="title"           type="text_en" indexed="true"  stored="true"/>
<field name="title_str"       type="string"  indexed="true"  stored="false"
↪  docValues="true"/>

<!-- Original title (non-English shows, e.g. "Squid Game" / "      "). Stored
     as a string so we can show it verbatim and facet on language groupings. -->
<field name="original_title"  type="string"  indexed="true"  stored="true"
↪  docValues="true"/>
<field name="language"        type="string"  indexed="true"  stored="true"
↪  docValues="true"/>

<!-- Long synopsis. Analyzed for relevance and highlighting. -->
<field name="description"     type="text_en" indexed="true"  stored="true"/>

<!-- Catalog facets. All multi-valued + docValues so the JSON Facet API (§3.3)
     can aggregate them without an on-heap field cache. -->
<field name="genres"          type="string"  indexed="true"  stored="true"
↪  docValues="true" multiValued="true"/>
<field name="platforms"       type="string"  indexed="true"  stored="true"
↪  docValues="true" multiValued="true"/>
<field name="creator"         type="string"  indexed="true"  stored="true"
↪  docValues="true" multiValued="true"/>

<!-- Cast: analyzed for searchability ("show me everything with Pedro Pascal")
     AND a string mirror for facet/sort. -->
<field name="cast"            type="text_en" indexed="true"  stored="true"
↪  multiValued="true"/>
```

```
<field name="cast_str"        type="string"  indexed="true"  stored="false"
↪  docValues="true" multiValued="true"/>

<!-- Numeric range/sort fields. -->
<field name="release_year"    type="pint"    indexed="true"  stored="true"/>
<field name="runtime_minutes" type="pint"    indexed="true"  stored="true"/>
<field name="seasons"         type="pint"    indexed="true"  stored="true"/>
<field name="rating"          type="pfloat"  indexed="true"  stored="true"/>
<field name="status"          type="string"  indexed="true"  stored="true"
↪  docValues="true"/>
                          <!-- "ongoing" | "ended" | "cancelled" | "limited" -->
<field name="added_at"        type="pdate"   indexed="true"  stored="true"/>

<!-- Hot counters / scores. These are the in-place-update targets (§2.8.2).
     indexed=false stored=false docValues=true is what unlocks the in-place path. -->
<field name="popularity"      type="pfloat"  indexed="false" stored="false"
↪  docValues="true" multiValued="false"/>
<field name="view_count"      type="plong"   indexed="false" stored="false"
↪  docValues="true" multiValued="false"/>

<!-- Dense vector for semantic similarity ("dark crime drama in Scandinavia"). -->
<field name="embedding"       type="knn_vector_384" indexed="true" stored="true"/>

<!-- _version_ MUST be docValues=true (and indexed=false stored=false) for in-place
     updates to work (§2.8.2). The older "indexed=true stored=true" shape predates
     in-place updates and disables them silently. -->
<field name="_version_"       type="plong"   indexed="false" stored="false"
↪  docValues="true"/>
<field name="_root_"          type="string"  indexed="true"  stored="false"
↪  docValues="false"/>

<uniqueKey>id</uniqueKey>

<!-- copyField: feed title and cast into both their string mirrors (for exact
     facet/sort) and a catch-all text_all field (for default-q search). -->
<copyField source="title"          dest="title_str"  maxChars="256"/>
<copyField source="cast"           dest="cast_str"/>
<copyField source="title"          dest="text_all"/>
<copyField source="original_title" dest="text_all"/>
<copyField source="description"    dest="text_all"/>
```

```
  <copyField source="cast"          dest="text_all"/>
  <copyField source="creator"       dest="text_all"/>

  <field name="text_all" type="text_en" indexed="true" stored="false"
↪   multiValued="true"/>

  <!-- dynamicField: convention for ad-hoc fields outside the main schema. -->
  <dynamicField name="*_s"  type="string"  indexed="true" stored="true"/>
  <dynamicField name="*_i"  type="pint"    indexed="true" stored="true"/>
  <dynamicField name="*_dt" type="pdate"   indexed="true" stored="true"/>

</schema>
```

A canonical seed dataset

Every example in the rest of this book — every query, every facet, every signals row in Chapter 5 — draws from a small fixed set of shows. Pin this table in your head; it makes the examples concrete.

Table 4: The 30-show seed catalog every example in the rest of the book draws from, spanning 8 platforms, ~20 genres, mixed languages, and a mix of ongoing, ended, and cancelled status.

id	title	platforms	genres	release_year	status
ST-001	Stranger Things	Netflix	sci-fi, horror, drama	2016	ongoing
BB-001	Breaking Bad	Netflix	crime, drama, thriller	2008	ended
BCS-001	Better Call Saul	Netflix	crime, drama, legal	2015	ended
TC-001	The Crown	Netflix	drama, historical	2016	ended
WED-001	Wednesday	Netflix	supernatural, comedy, mystery	2022	ongoing
SG-001	Squid Game	Netflix	thriller, drama, korean	2021	ongoing

id	title	platforms	genres	release_year	status
BRG-001	Bridgerton	Netflix	romance, period, drama	2020	ongoing
OZ-001	Ozark	Netflix	crime, drama	2017	ended
TBR-001	The Bear	Hulu	drama, comedy, food	2022	ongoing
OMITB-001	Only Murders in the Building	Hulu	comedy, mystery	2021	ongoing
SH-001	Shōgun	Hulu	drama, historical, japanese	2024	limited
TBY-001	The Boys	Prime Video	superhero, satire, action	2019	ongoing
RCH-001	Reacher	Prime Video	action, thriller, crime	2022	ongoing
FB-001	Fleabag	Prime Video	comedy, drama, british	2016	ended
SVR-001	Severance	Apple TV+	sci-fi, mystery, thriller	2022	ongoing
TL-001	Ted Lasso	Apple TV+	comedy, drama, sports	2020	ended
TM-001	The Mandalorian	Disney+	sci-fi, western, action	2019	ongoing
LK-001	Loki	Disney+	sci-fi, fantasy, drama	2021	ongoing
AND-001	Andor	Disney+	sci-fi, drama, political	2022	ongoing
TLOU-001	The Last of Us	Max	drama, post-apocalyptic, horror	2023	ongoing
HOTD-001	House of the Dragon	Max	fantasy, drama, action	2022	ongoing

id	title	platforms	genres	release_year	status
SUC-001	Succession	Max	drama, satire	2018	ended
WW-001	Westworld	Max	sci-fi, western, drama	2016	cancelled
BM-001	Black Mirror	Netflix	sci-fi, anthology, thriller	2011	ongoing
PB-001	Peaky Blinders	Netflix	crime, period, british	2013	ended
MH-001	Mindhunter	Netflix	crime, psychological, drama	2017	cancelled
WT-001	The Witcher	Netflix	fantasy, action, adventure	2019	ongoing
YS-001	Yellowstone	Peacock	drama, western	2018	ended
TO-001	The Office (US)	Peacock	comedy, sitcom	2005	ended
FLG-001	The Office (UK)	BritBox	comedy, sitcom	2001	ended

That's 30 shows across 8 platforms, ~20 distinct genres, English and non-English titles (Shōgun, Squid Game), running and ended status, and franchise relationships (TM-001, LK-001, AND-001 all share the Star Wars / Marvel cinematic universe — useful for the relatedness() aggregation in §5.5).

Some examples below will refer to these specific IDs. The two pairs TO-001 / FLG-001 (US vs. UK Office) and the MH-001 / BB-001 near-collisions on prefix b* are intentional — they let us demonstrate disambiguation, prefix-query cost, and editorial overrides (§5.6) on realistic data.

Field properties decoded

Table 5: The five schema field properties, what each one enables on a field, and the storage or write cost it adds.

Property	What it does	Cost
indexed=true	Add to inverted index → field is searchable	Write throughput, disk
stored=true	Keep verbatim copy → returnable in fl	Disk; large strings hurt
docValues=true	Column-stride forward index → fast sort/facet/function queries	Disk
multiValued=true	Field can hold many values per doc	Position gap inserted between values
required=true	Doc rejected if field is missing	n/a

Rules of thumb you should internalize:

1. Text you want to search → indexed=true, stored=true (for highlighting), in a text_* field type.
2. Text you want to sort, facet, or filter exactly → string field with docValues=true. Either separate the field, or copyField a tokenized version into a string mirror (title → title_str).
3. Numerics/dates you want to range-query and sort → the solr.*PointField types with docValues=true.
4. Anything you ever return in fl → stored=true or useDocValuesAsStored=true for primitive types.

   Production tip — docValues is almost always worth the disk. Without it, sorting on a text field happens via on-heap field cache reconstruction; with it, sorting is a column read from mmap. Sort, facet, function-query, and grouping all collapse to docValues.

2.4 Analyzers, tokenizers, filters

A text field type has an analyzer chain at index time and (optionally) a different one at query time. The chain is:

Tokenizer → Filter → Filter → ...

For our text_en above, indexing the title ”Stranger Things Season 4” flows:

Figure 7: The text_en analyzer chain transforming 'Stranger Things Season 4' through tokenization, lower-casing, stop filtering, and Porter stemming.

A query for Strange Things goes through the same kind of pipeline (plus synonyms at query time) and reduces to [strang, thing]. Note: Porter stems strange → strang but leaves stranger → stranger. They do not collapse to the same root, which is one reason exact phrase matches are stronger signals than individual-token AND.

> Production tip — keep index and query analyzers symmetric, except for synonym expansion. Doing synonym expansion at index time bakes the synonym set into the index and forces a reindex when you change synonyms.txt. Doing it at query time only requires reloading the collection. Use SynonymGraphFilterFactory (not the legacy SynonymFilterFactory) so multi-word synonyms compose correctly with phrase queries.

Testing analyzer chains with the Analysis form

The fastest way to verify a chain does what you think it does is the admin UI's Analysis tab (introduced in the tour in §2.0). Pick the collection, pick a field name or field type, paste a value into the Index Analyzer box (and optionally a different value into the Query Analyzer box to compare), and click Analyze Values.

For our text_en field, indexing "Stranger Things, S4" produces the following step-by-step breakdown — one row per stage in the chain:

Table 6: Step-by-step output of the text_en analyzer chain when indexing the string "Stranger Things, S4", as seen in the admin UI Analysis tab.

Stage	Output tokens
StandardTokenizer	Stranger, Things, S4 (comma dropped as punctuation)
EnglishPossessiveFilter	Stranger, Things, S4 (no change — no possessive)
LowerCaseFilter	stranger, things, s4
StopFilter	stranger, things, s4 (none of these are stopwords)
SnowballPorter (English)	stranger, thing, s4 (Porter strips the plural -s from things)

A query for ”Strange Things” flows through the same chain (plus any query-time synonym expansion) and reduces to [strang, thing]. Both forms share thing, so documents will match on that token; the strange/stranger divergence is why the exact-phrase boost in §3.2 matters.

When a search fails to return what you expect, the Analysis tab is the first place to look: it shows you what the index actually contains versus what your query actually produced, without having to read tokenized output from a debug=true parameter.

> Production tip — bookmark the Analysis endpoint. The form posts to /solr/<collection>/analysis/field?wt=json&analysis.fieldvalue=<input>&analysis.fieldtype=<type>. Curl that endpoint from CI to assert an analyzer change actually produced the intended tokens — a 10-line regression test for the kind of schema drift that’s painful to debug after the fact.

2.5 copyField and dynamicField

- copyField routes a field’s content into one or more additional fields during indexing. The classic use is a single text_all field that aggregates title, description, cast, creator, etc. (the schema above does this), so that a default q= searches everywhere. Another classic use is shadow string fields for sorting/faceting — note how title → title_str and cast → cast_str in our schema.
- dynamicField matches by suffix/prefix wildcard. *_s means “any field whose name ends in _s is automatically a string”. Useful for multi-tenant or rapidly-evolving data where you don’t want to declare every field. Pattern: network_region_s, imdb_id_s, last_aired_dt.

2.6 SolrJ: indexing from Java

For Solr 10, the Maven coordinates are:

```
<dependency>
  <groupId>org.apache.solr</groupId>
  <artifactId>solr-solrj</artifactId>
  <version>10.0.0</version>
</dependency>
<!-- For Jetty-based clients (HttpJettySolrClient, ConcurrentUpdateJettySolrClient, etc. —
 ↪  also used by the default CloudSolrClient when Jetty is on the classpath) -->
<dependency>
  <groupId>org.apache.solr</groupId>
  <artifactId>solr-solrj-jetty</artifactId>
```

```
  <version>10.0.0</version>
</dependency>
```

The Solr 10 reference guide states verbatim: “If you want to use Jetty-based HTTP clients (HttpJettySolrClient, ConcurrentUpdateJettySolrClient, LBJettySolrClient, or CloudJettySolrClient), you need to add the maven dependency solr-solrj-jetty.” Alternatively, “you can use HttpJdkSolrClient from the base solr-solrj artifact, which uses Java’s built-in HTTP client and has no additional dependencies.” The split into multiple artifacts is the result of SOLR-17161 in the 9.x line; further artifacts include solr-solrj-zookeeper (only needed if you really must speak ZooKeeper from the client) and solr-solrj-streaming (Streaming Expressions).

> Solr 9 difference — single SolrJ artifact. In Solr 9.x (before SOLR-17161 landed late in the cycle), there was a single solr-solrj artifact that pulled in Jetty, ZooKeeper, and streaming transitively. If you’re upgrading a 9.x build script, expect to add solr-solrj-jetty (or switch to HttpJdkSolrClient) and possibly solr-solrj-zookeeper.

> Production tip — Solr 10 SolrJ runs on Java 17. The server requires Java 21 (per the Solr 10 upgrade notes: “Solr 10.0 requires at least Java 21, while SolrJ 10.0 requires at least Java 17”), but the client library only requires Java 17. Java 17 client applications can still talk to a Solr 10 cluster.

Connecting to SolrCloud

In Solr 10, the recommended pattern is to connect via Solr node URLs, not directly to ZooKeeper. The Solr 9.x line deprecated CloudSolrClient.Builder’s ZooKeeper-hosts constructor; the 10.x guidance is to give the client a list of Solr base URLs.

```
import org.apache.solr.client.solrj.impl.CloudSolrClient;
import org.apache.solr.client.solrj.SolrClient;
import java.time.Duration;
import java.util.List;

public class SolrFactory {
    public static SolrClient build() {
        List<String> solrUrls = List.of(
            "http://solr-0.solr-headless.solr.svc.cluster.local:8983/solr",
            "http://solr-1.solr-headless.solr.svc.cluster.local:8983/solr",
            "http://solr-2.solr-headless.solr.svc.cluster.local:8983/solr"
        );
        return new CloudSolrClient.Builder(solrUrls)
```

```
            .withDefaultCollection("shows")
            .withConnectionTimeout(Duration.ofSeconds(10))
            .withRequestTimeout(Duration.ofSeconds(30))
            .build();
    }
}
```

Hold the client as a long-lived singleton in your application — it's thread-safe and expensive to construct. Close it on shutdown.

> Solr 9 difference — ZooKeeper host constructor. In Solr 9.x, new CloudSolrClient.Builder(zkHosts, Optional.of(chroot)) was the common idiom: the client connected directly to ZooKeeper. That constructor was deprecated in 9.x and the Solr 10 recommendation is to pass Solr base URLs as shown above (the client discovers the cluster via the Solr nodes themselves). If you absolutely must speak ZK from the client, add solr-solrj-zookeeper and use the ZK-host constructor — but the URL-list form is the future and is what works in production without the extra dependency.
>
> Also in Solr 10, the Major Changes page notes: "CloudHttp2SolrClient.Builder has moved to CloudSolrClient; users should generally have no need to refer to CloudHttp2SolrClient or any class/member with 'http2' in it. The builder will check if Jetty HttpClient is available and use that, otherwise fallback on a JDK based HttpClient." For new code in Solr 10, prefer CloudSolrClient.Builder directly. The CloudHttp2SolrClient symbol still exists for source compatibility but is effectively an alias.

Indexing a single document

```
import org.apache.solr.common.SolrInputDocument;
import org.apache.solr.client.solrj.SolrClient;

SolrInputDocument doc = new SolrInputDocument();
doc.addField("id",          "ST-001");
doc.addField("title",       "Stranger Things");
doc.addField("description",
    "When a young boy vanishes, a small town uncovers a mystery involving "
  + "secret government experiments, supernatural forces, and one strange little girl.");
doc.addField("genres",      List.of("sci-fi", "horror", "drama"));
```

```
doc.addField("platforms",       List.of("Netflix"));
doc.addField("creator",         List.of("The Duffer Brothers"));
doc.addField("cast",            List.of("Millie Bobby Brown", "Finn Wolfhard",
                                 "Winona Ryder", "David Harbour"));
doc.addField("release_year",    2016);
doc.addField("runtime_minutes", 51);
doc.addField("seasons",         4);
doc.addField("rating",          8.7f);
doc.addField("status",          "ongoing");
doc.addField("added_at",        java.time.Instant.now().toString());
doc.addField("embedding",       List.of(0.012f, -0.183f, /* ... 384 dims ... */ 0.044f));
// popularity and view_count are in-place-updated by the signals pipeline (Chapter 5);
// don't set them at initial-index time.

client.add("shows", doc);
client.commit("shows"); // For demos only; see commit-strategies section.
```

Bulk indexing throughput

For real ingestion pipelines do not call client.add(doc) in a loop per document. Two patterns dominate:

1. Batched add(Collection<SolrInputDocument>) on CloudSolrClient — gives you back-pressure and exceptions.
2. ConcurrentUpdateJettySolrClient (Solr 10) — fire-and-forget with internal queues and parallel HTTP. Highest throughput; you must implement an error handler because exceptions surface asynchronously.

```
import org.apache.solr.client.solrj.impl.ConcurrentUpdateJettySolrClient;

try (ConcurrentUpdateJettySolrClient client =
        new ConcurrentUpdateJettySolrClient.Builder(
                "http://solr-0:8983/solr/shows")
            .withQueueSize(10_000)
            .withThreadCount(8)
            .build()) {

    List<SolrInputDocument> batch = new ArrayList<>(1_000);
    for (Show s : shows) {
        batch.add(toSolrDoc(s));
```

```
        if (batch.size() == 1_000) {
            client.add(batch);
            batch.clear();
        }
    }
    if (!batch.isEmpty()) client.add(batch);
    client.blockUntilFinished(); // wait for the internal queue to drain
}
```

> Solr 9 difference — ConcurrentUpdateHttp2SolrClient. In Solr 9.x, the equivalent class was ConcurrentUpdateHttp2SolrClient (note the Http2 in the name). Solr 10 (SOLR-18005) refactored this: the old class was made abstract and renamed ConcurrentUpdateBaseSolrClient, with ConcurrentUpdateJettySolrClient added as the new Jetty-backed concrete implementation. If you're upgrading from 9, replace the symbol name; the API surface is essentially the same.

For the typical case (a Java service indexing through SolrCloud) prefer CloudSolrClient batches — it routes documents to the correct shard leader directly, avoiding the inter-node hop that ConcurrentUpdateJettySolrClient against a single endpoint introduces.

> Production tip — set withQueueSize deliberately. Too small and threads stall on the bounded queue. Too large and a node failure leaves thousands of in-flight docs to recover. 5k–20k is a typical sweet spot. Monitor with the Solr metrics endpoint (UPDATE.updateHandler.adds).

Atomic and partial updates: a teaser

SolrJ also supports atomic updates — modify a single field without resending the whole document — and in-place updates, which skip reindexing entirely. These are important enough to deserve their own section: see §2.8 below.

2.7 Integration testing with Testcontainers

What it is

Testcontainers is a Java library that spins up real Docker containers inside your JUnit or TestNG tests and tears them down automatically when the test suite finishes. The solr module provides a SolrContainer that starts the official apache/solr Docker image, exposes the Solr port, and gives you a ready-to-use SolrClient — the same client you use in production.

> Why not an embedded Solr? There is no supported embedded Solr in Solr 10. The old EmbeddedSolrServer still compiles but is undocumented, untested for correctness, and missing SolrCloud semantics. The Solr project's own recommendation is to test against a real Solr instance. Testcontainers is how you do that without a shared dev cluster.

The official Docker image

The official image is apache/solr on Docker Hub. Tags follow Solr releases: apache/solr:10.0.0, apache/solr:9.8, apache/solr:latest. The Solr Admin UI is available on port 8983 — useful for poking at a running container while debugging a test.

> Admin UI for debugging. When a test is failing and you want to inspect the index, add a Thread.sleep(300_000) breakpoint, connect to http://localhost:<mappedPort>/solr, and use the Analysis tab (to trace how a string is tokenized), the Query tab (to run ad-hoc queries), and the Schema tab (to inspect field definitions). It's the fastest way to diagnose analyzer chain or schema issues without writing more test code. See the Solr Admin UI reference for the full tour.

Maven dependency

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>solr</artifactId>
  <version>1.20.4</version>
  <scope>test</scope>
</dependency>
<!-- JUnit 5 integration -->
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>1.20.4</version>
  <scope>test</scope>
</dependency>
```

A minimal integration test

```
import org.apache.solr.client.solrj.SolrClient;
import org.apache.solr.client.solrj.impl.CloudSolrClient;
import org.apache.solr.client.solrj.request.CollectionAdminRequest;
import org.apache.solr.client.solrj.response.QueryResponse;
import org.apache.solr.client.solrj.request.SolrQuery;
import org.apache.solr.common.SolrInputDocument;
import org.junit.jupiter.api.*;
import org.testcontainers.containers.SolrContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;

import java.util.List;

@Testcontainers
class ProductIndexTest {

    @Container
    static SolrContainer solr = new SolrContainer(
            DockerImageName.parse("apache/solr:10.0.0"))
        .withCollection("shows");    // creates the collection on startup

    private SolrClient client;

    @BeforeEach
    void setUp() throws Exception {
        // SolrContainer exposes the standard Solr port on a random host port
        String url = "http://" + solr.getHost() + ":" + solr.getSolrPort() + "/solr";
        client = new CloudSolrClient.Builder(List.of(url))
            .withDefaultCollection("shows")
            .build();
    }

    @AfterEach
    void tearDown() throws Exception {
        client.close();
    }

    @Test
    void indexAndQuery() throws Exception {
```

```
        // Index a document
        SolrInputDocument doc = new SolrInputDocument();
        doc.addField("id",           "ST-001");
        doc.addField("title",        "Stranger Things");
        doc.addField("platforms",    "Netflix");
        doc.addField("release_year", 2016);
        client.add(doc);
        client.commit();

        // Query it back
        QueryResponse resp = client.query(
            new SolrQuery("title:Stranger").setRows(10));

        Assertions.assertEquals(1, resp.getResults().getNumFound());
        Assertions.assertEquals("ST-001",
            resp.getResults().get(0).getFieldValue("id"));
    }
}
```

A few things worth noting:

- @Container + static — a static container is shared across all tests in the class and started once. Drop static if you need a clean Solr per test (slower but fully isolated).
- withCollection("shows") — creates a collection using Solr's default configset. To use a custom configset, mount it into the container with .withSolrConfiguration("configsetName", path) — see the Testcontainers Solr docs.
- getSolrPort() returns the mapped host port, which is random. Testcontainers handles the Docker port mapping; you never hardcode 8983 in tests.
- Commit is required. The container runs with the same commit defaults as a real Solr. Either call client.commit() explicitly after indexing in tests, or configure the container's solrconfig.xml with a short autoSoftCommit interval.

  Production tip — one container per test class, shared. Container startup is the expensive part (3–8 seconds for Solr). Use @Container static and @TestMethodOrder to keep the container alive across all tests. If test isolation requires a clean index, call client.deleteByQuery("shows", "*:*") + client.commit() in @BeforeEach rather than restarting the container.

  Production tip — pin the image version. apache/solr:latest will silently pull a newer Solr on the next docker pull, which can break tests when APIs change. Pin

to apache/solr:10.0.0 (or whatever minor you're targeting) in CI, and update deliberately.

2.8 Partial updates: atomic, in-place, and when they fall back

Once your index is bootstrapped, almost all write traffic in a production Solr deployment falls into one of three buckets:

1. Full document replacement. You send the entire document; Solr discards the old version and reindexes it.
2. Atomic update. You send only the fields you want to change, marked with modifier maps (set, add, inc, …). Under the hood Solr still reindexes the whole document — you just didn't have to assemble it on the client side.
3. In-place update. Same syntax as an atomic update, but on fields that meet a strict set of constraints Solr rewrites only the docValues column. The document is not reindexed. This is the only path that is genuinely "partial" at the Lucene level.

Optimistic concurrency (the _version_ field) is orthogonal to all three — you can layer it on any update.

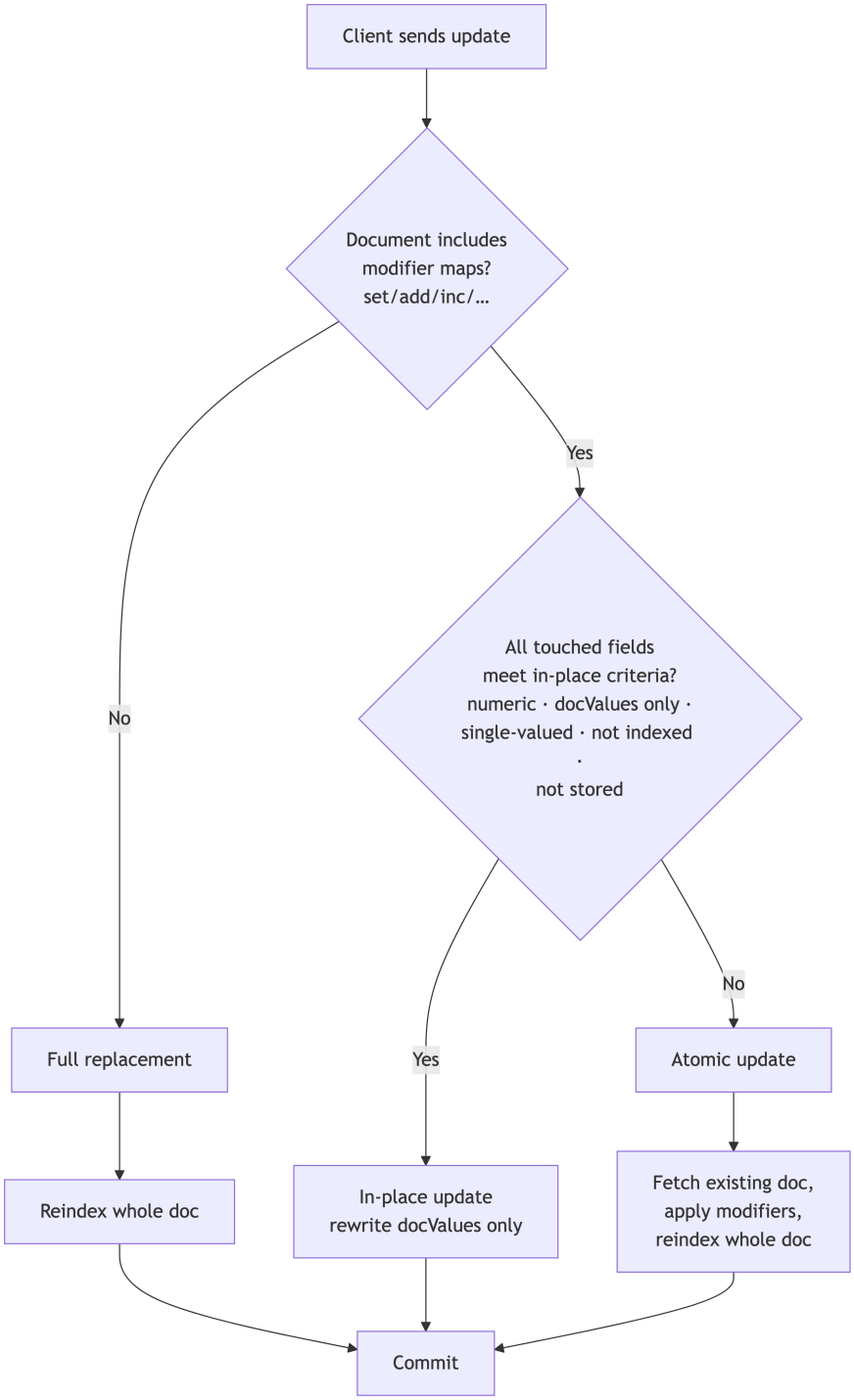

Figure 8: Decision tree for partial-update dispatch: how Solr chooses between full replacement, in-place docValues rewrite, and a fallback atomic reindex.

Production tip — atomic cheap. The single most common mistake is thinking atomic updates are partial at the Lucene level. They aren't. If a 50-field document gets an inc on one counter, Solr rebuilds and reindexes all 50 fields, runs every

analyzer, and replaces the underlying Lucene document. For genuinely hot fields, plan for in-place.

2.8.1 Atomic updates

Supported modifiers

Table 7: The six atomic-update modifiers Solr supports, the field shape each requires, and what the operation does to the existing values.

Modifier	Field shape	Behavior
set	any	Replace value(s). null or [] removes the field.
add	multi-valued	Append value(s).
add-distinct	multi-valued	Append value(s) only if not already present.
remove	multi-valued	Remove all occurrences of the given value(s).
removeregex	multi-valued	Remove values matching a regex.
inc	numeric, single-val.	Increment (negative argument decrements).

These are the only six. Anything else — rename a field, concatenate strings, derive a value — has to be done client-side via a read-modify-write with a full replace.

Schema requirements

From the Solr 10 reference guide: every "real" field your application populates must be stored="true" OR docValues="true". Solr reconstructs the document from one of those. The exception is copyField destinations, which must not be stored and must not behave as if stored (useDocValuesAsStored="false"), otherwise on each atomic update Solr would double-write — once from the source-field copy and once from the existing stored destination value — producing duplicate or stale tokens.

If any required field violates these rules, the entire atomic update fails. There is no partial recovery.

How atomic updates actually work

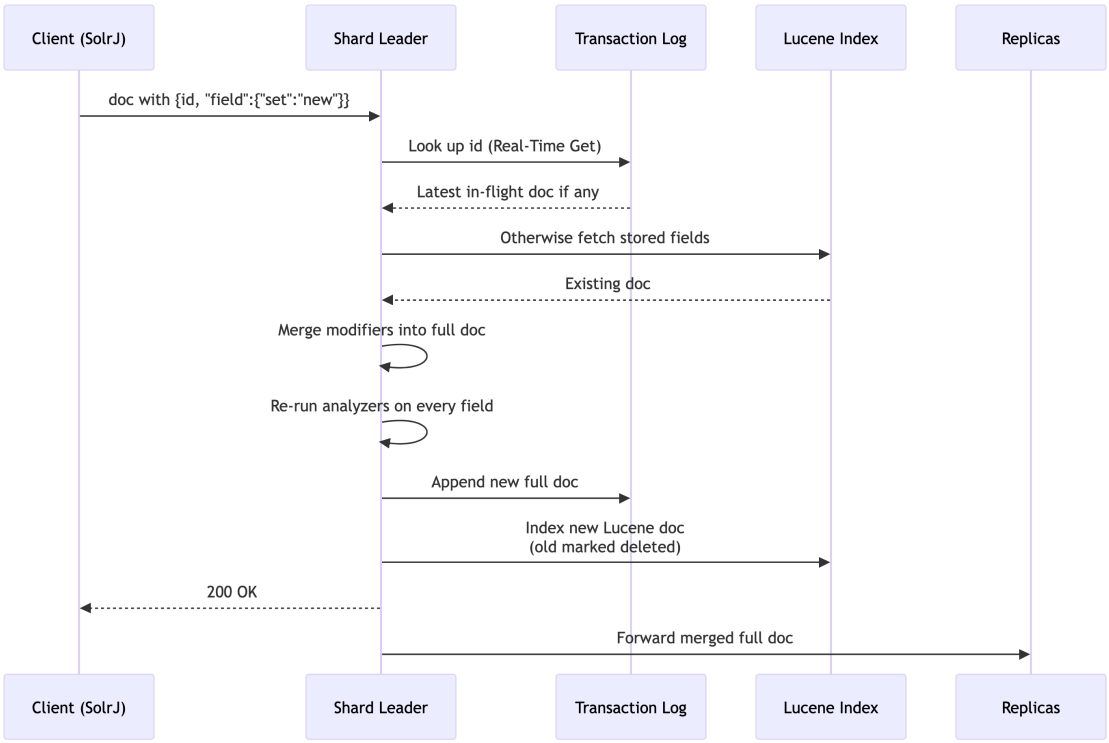

Figure 9: How an atomic update actually executes on the shard leader: read existing doc, merge modifiers, re-run analyzers, reindex the whole Lucene document, then forward to replicas.

Three things follow from this:

1. Atomic partial at the Lucene level. Lucene has no concept of mutating a doc; the new doc is added and the old one is logically deleted. Segment merges reclaim the space later.
2. All analyzers run again on every field. Heavy stem chains or synonym graph filters cost the same whether you set one field or twenty.
3. The merge happens on the shard leader before forwarding to replicas. Replicas see the merged full document, not the modifier instructions.

Practical SolrJ helper

Wrap the modifier-map ceremony so app code stays readable:

```
public final class AtomicOp {
    public static Map<String,Object> set(Object v)          { return
     ↪  Collections.singletonMap("set", v); }
    public static Map<String,Object> setNull()              { return
     ↪  Collections.singletonMap("set", null); }
    public static Map<String,Object> inc(Number delta)      { return Map.of("inc", delta); }
    public static Map<String,Object> add(Object v)          { return Map.of("add", v); }
    public static Map<String,Object> addDistinct(Object v) { return Map.of("add-distinct",
     ↪  v); }
    public static Map<String,Object> remove(Object v)       { return Map.of("remove", v); }
    public static Map<String,Object> removeRegex(String r) { return Map.of("removeregex",
     ↪  r); }
}
```

A typical update reads cleanly:

```
SolrInputDocument doc = new SolrInputDocument();
doc.addField("id", "ST-001");
doc.addField("rating",      AtomicOp.set(8.9f));              // bump rating after new season
doc.addField("view_count", AtomicOp.inc(1));                  // hot counter (§2.8.2)
doc.addField("platforms",  AtomicOp.addDistinct("Hulu"));     // newly licensed to Hulu
doc.addField("status",      AtomicOp.set("ended"));           // show wrapped
client.add("shows", doc);
```

Common gotchas

- copyField and destinations. If you ever stored data directly into a copyField destination (a common bootstrap mistake), atomic updates will silently lose it — only what came in via copyField sources is reconstructed. Make destinations stored="false".
- set on multi-valued. Accepts a list, replaces the entire list. set(null) or set([]) clears it.
- inc on a missing field. Treated as starting from zero. Safe for "first view" counters.
- Boolean / date fields. set works; inc does not. To toggle a boolean you must read it and write the negation.

2.8.2 In-place updates: the genuinely partial path

What they are

An in-place update is what you actually want for a counter, popularity score, rating, or "last seen" timestamp. The Solr 10 reference guide is direct: "only the fields to be updated are affected and the rest of the documents are not reindexed internally. Hence, the efficiency of updating in-place is unaffected by the size of the documents that are updated."

Concretely: Solr rewrites the docValues column for the affected field on the live segment, without producing a new Lucene document. No analyzer run, no copyField re-evaluation, no full document round-trip from the transaction log.

When Solr picks the in-place path

Solr decides at request time, per field. All of the following must hold:

1. The field is numeric (e.g. pint, plong, pfloat, pdouble, pdate).
2. The field has indexed="false", stored="false", multiValued="false", docValues="true".
3. The _version_ field itself has the same shape (the default in modern Solr).
4. Any copyField destinations from the updated field also satisfy the above.
5. The modifier is set or inc.

If any of those is violated, Solr silently falls back to a regular atomic update — same response, very different cost. That silent fallback is the trap.

> Production tip — fail fast with update.partial.requireInPlace=true. Solr 10 lets you set this request parameter; if the update can't be done in-place the request fails instead of silently falling back. Use this on hot counter paths in your CI tests — the day someone "helpfully" adds stored="true" to your counter field, the build breaks instead of production performance.

Schema pattern for counters

```
<field name="view_count"  type="plong"  indexed="false" stored="false" docValues="true"
 ↪  multiValued="false"/>
<field name="popularity"  type="pfloat" indexed="false" stored="false" docValues="true"
 ↪  multiValued="false"/>
<field name="last_watched" type="pdate" indexed="false" stored="false"
 ↪  docValues="true" multiValued="false"/>
```

These are sortable, facetable, and usable in function queries (boost=popularity). They are not matchable in text queries. Almost always the right trade-off for view-count and recency-style data on a shows catalog.

SolrJ example with the safety net

```
import org.apache.solr.client.solrj.request.UpdateRequest;

public void bumpViewCount(SolrClient client, String docId) throws Exception {
    SolrInputDocument doc = new SolrInputDocument();
    doc.addField("id", docId);
    doc.addField("view_count", AtomicOp.inc(1));

    UpdateRequest req = new UpdateRequest();
    req.add(doc);
    req.setParam("update.partial.requireInPlace", "true"); // Solr 10: fail fast
    req.process(client, "shows");
}
```

When to reach for in-place updates

Good fits: view counts, click counts, "likes", popularity scores from a batch job, ratings, "last seen" timestamps, A/B test bucket scores. Bad fits: anything you want to query as a term, anything multi-valued, strings, fields that participate in copyField to a text field.

In-place updates are O(1) in document size. Atomic updates are O(total field count × analyzer cost). On a 30-field document with stem and synonym filters the difference is one to two orders of magnitude in CPU and tlog volume.

2.8.3 Optimistic concurrency, used well

Optimistic concurrency rides on the _version_ field, which is automatically written on every document. The semantics:

Table 8: How the value supplied for _version_ on an update controls optimistic-concurrency semantics, from must-not-exist through exact-version-match.

Value of _version_ on the update	Meaning
> 1 (e.g. 1632740…)	Must exist; the version must match exactly.
1	Must exist, version doesn't matter.
0 (default if omitted)	Don't care: insert or replace.
< 0 (e.g. -1)	Must not exist.

A version mismatch returns HTTP 409.

Read-modify-write in SolrJ

```
import org.apache.solr.common.params.CommonParams;

public boolean updateRatingWithRetry(SolrClient client, String id, float newRating, int
↪   maxRetries)
      throws Exception {
  for (int attempt = 0; attempt < maxRetries; attempt++) {
    // 1. Read current version via Real-Time Get (NOT /select — would return stale data).
    SolrDocument current = client.getById("shows", id);
    if (current == null) return false;
    long version = (Long) current.getFieldValue("_version_");

    // 2. Build partial update with the version constraint.
    SolrInputDocument doc = new SolrInputDocument();
    doc.addField("id", id);
    doc.addField("rating", AtomicOp.set(newRating));
    doc.addField(CommonParams.VERSION_FIELD, version);    // "_version_"

    try {
      client.add("shows", doc);
      return true;
    } catch (SolrException e) {
```

```
            if (e.code() == 409) continue;   // lost race, retry
            throw e;
        }
    }
    return false;
}
```

Three things matter here:

- Always use getById() (Real-Time Get, hits /get) for the read. A /select query can return the pre-commit state and you'll race against your own writes.
- Cap retries at 3–5. If you can't win in 5 tries, you're in a write storm — back off or queue.
- Combine freely with atomic or in-place updates. Optimistic concurrency is just a constraint on the version field; it doesn't care what the rest of the update looks like.

"Insert only" and "update only"

For an idempotent insert endpoint:

```
doc.addField(CommonParams.VERSION_FIELD, -1);   // must NOT exist; 409 on duplicate
```

For an "edit but only if it still exists" endpoint:

```
doc.addField(CommonParams.VERSION_FIELD, 1);    // must exist; 409 if deleted
```

These three levers — -1, 1, N — cover most "create vs update vs replace" REST semantics without an explicit lookup-then-write transaction.

2.8.4 Partial updates on nested documents (briefly)

Nested documents are supported by partial updates, with two rules:

1. The schema needs _root_ as an indexed field (Solr auto-populates it with the root document's ID). For partial updates to children, _root_ must also be stored="true" or docValues="true".
2. The update document must include _root_ set to the parent's ID. Without it, Solr treats the child ID as a root ID and may overwrite an unrelated root document.

```
// If you'd modeled episodes as child docs of shows (we don't in this book —
// shows-only schema — but as an example):
SolrInputDocument child = new SolrInputDocument();
child.addField("id",      "ST-001!S04E01");          // child id
child.addField("_root_", "ST-001");                  // mandatory: parent's id
child.addField("view_count_l", AtomicOp.inc(1));

UpdateRequest req = new UpdateRequest();
req.add(child);
req.process(client, "shows");
```

Caveats: in-place updates do not work on child documents (they go through the atomic path, which conceptually rewrites the entire tree). And delete-by-id only works on root documents — to delete a child, use delete-by-query or an atomic remove on the parent's children pseudo-field.

2.9 Reindexing: when, why, and how

Reindexing is the operation everyone wishes they didn't have to do, and the operation that distinguishes shops that ship Solr changes confidently from shops that don't. The Solr 10 reference guide states it cleanly: "Lucene does not use a schema, schema is a Solr-only concept. When you change Solr's schema, the Lucene index is not modified in any way." The schema is a rulebook applied at ingest time — if the rulebook changes, you must reapply it to every document.

2.9.1 When you must reindex

A full reindex is required when any of the following changes:

1. A field's type changes (e.g. string → text_general).
2. The index-time analyzer chain on an existing field type changes — tokenizer, filter ordering, stemmer language, new char filter.
3. You add a new analyzed/indexed text field. Existing documents have no terms for it.
4. You change tokenizers or stemmer settings.
5. You change Similarity per field. Per-field similarity is part of indexed norms.
6. You add docValues="true" to an existing field. DocValues are written at index time; flipping the property does nothing to existing segments.
7. A major Solr version upgrade (e.g. 9.x → 10.x). Lucene only supports reading the current and previous major versions.

8. You change luceneMatchVersion in solrconfig.xml — analysis behavior at the Lucene layer can shift silently.
9. You change the codecFactory.
10. You add a new copyField source/destination. Old documents won't have tokens propagated; only new and updated docs will.
11. Any change to the index-time UpdateRequestProcessorChain (URPs that touch field values, language detection, signature/dedupe).

2.9.2 When you do not need to reindex

- Atomic and in-place updates (per §2.8) — these aren't schema changes.
- Adding a new field that isn't a copyField target. Existing documents simply lack a value. Queries against the new field will only match documents indexed after the schema change, which is usually the desired "from now on" behavior.
- Anything in solrconfig.xml that doesn't change what gets stored: cache sizes, request handlers, query parsers, default query parameters, autowarming counts, circuit breakers, rate limiters.
- Cluster topology changes: adding nodes, adding replicas, splitting shards, balancing.
- Changes to synonyms.txt or stopwords.txt IF those filters are only configured in the query-time analyzer chain. This is the single most useful pattern in this whole chapter.

Why synonyms belong at query time

Configure synonyms in the query-time analyzer only:

```
<fieldType name="text_general" class="solr.TextField" positionIncrementGap="100">
  <analyzer type="index">
    <tokenizer class="solr.StandardTokenizerFactory"/>
    <filter class="solr.StopFilterFactory" ignoreCase="true" words="stopwords.txt"/>
    <filter class="solr.LowerCaseFilterFactory"/>
  </analyzer>
  <analyzer type="query">
    <tokenizer class="solr.StandardTokenizerFactory"/>
    <filter class="solr.StopFilterFactory" ignoreCase="true" words="stopwords.txt"/>
    <filter class="solr.SynonymGraphFilterFactory" synonyms="synonyms.txt"
↪     expand="true"/>
    <filter class="solr.LowerCaseFilterFactory"/>
  </analyzer>
</fieldType>
```

Synonyms change frequently — new show abbreviations, new colloquial titles, marketing decisions. The cost of reindexing the whole corpus every time someone adds ”got game of thrones” or ”the office office us” is prohibitive. Query-time expansion lets you edit synonyms.txt, reload the collection, and the new mappings take effect immediately. The reference guide confirms: “If separate analysis chains are defined for query and indexing events for a field and you change only the query-time analysis chain, reindexing is not necessary.”

2.9.3 Zero-downtime reindex: the blue/green alias swap

The right pattern in production is to point search and write traffic at a collection alias, not a collection name. To reindex:

1. Create a new collection with the new schema/config.
2. Bulk-ingest into it.
3. Atomically point the alias at the new collection (the old mapping is replaced in place).
4. Validate.
5. Keep the old collection around for rollback, then delete after a grace period.

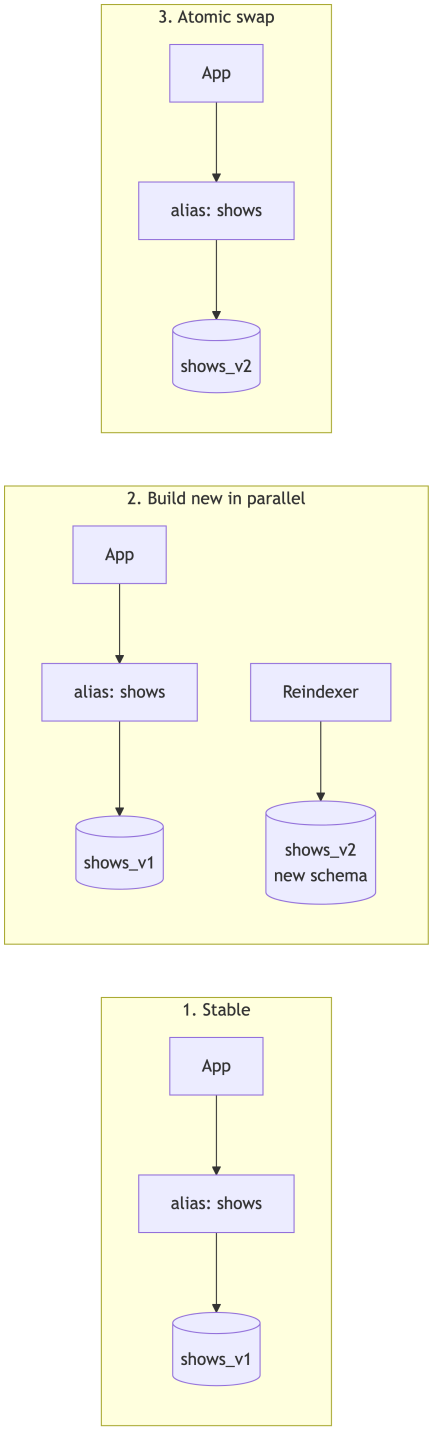

Figure 10: Zero-downtime blue/green reindex: build the new collection in parallel, then atomically repoint the alias so the application never changes its target.

Application code never changes — it uses the alias shows for reads, writes, atomic updates, everything.

There is no separate MODIFYALIAS command for standard aliases. Re-issuing CREATEALIAS with the same alias name is the swap operation, and it's atomic.

SolrJ code for the swap

```
import org.apache.solr.client.solrj.request.CollectionAdminRequest;

public final class AliasSwap {
    public static void createCollection(SolrClient client, String name, String configset,
                                        int numShards, int numReplicas) throws Exception {
        CollectionAdminRequest.createCollection(name, configset, numShards, numReplicas)
            .process(client);
    }

    /** Idempotent. CREATEALIAS replaces the alias→collection mapping if the alias exists. */
    public static void pointAliasAt(SolrClient client, String alias, String collection) throws Exception {
        CollectionAdminRequest.createAlias(alias, collection).process(client);
    }

    public static void deleteCollection(SolrClient client, String name) throws Exception {
        CollectionAdminRequest.deleteCollection(name).process(client);
    }
}
```

Full orchestration:

```
public void blueGreenReindex(SolrClient client) throws Exception {
    String alias  = "shows";
    String oldCol = "shows_v1";
    String newCol = "shows_v2";

    AliasSwap.createCollection(client, newCol, "shows_conf_v2", 4, 2);
    BulkReindexer.reindex(client, oldCol, newCol);          // §2.9.4
    Verifier.verify(client, oldCol, newCol);                // §2.9.5 - abort on mismatch
    AliasSwap.pointAliasAt(client, alias, newCol);          // atomic swap
    // Keep oldCol around for at least 24h, then:
    // AliasSwap.deleteCollection(client, oldCol);
```

```
}
```

> Production tip — never delete the old collection on the same day as the swap. The swap is instantaneous, but if you discover a relevance regression in production you want a one-line rollback (createAlias("shows", "shows_v1")), not a re-ingest.

REINDEXCOLLECTION — the built-in option

Solr ships a REINDEXCOLLECTION command that does steps 1, 2, and the alias swap from the server side:

```
CollectionAdminRequest.reindexCollection("shows_v1")
    .setTarget("shows_v2")
    .setConfigName("shows_conf_v2")
    .setBatchSize(500)
    .process(client);
```

It creates a temporary collection with the new schema, copies the source documents via their stored fields, runs them through the new analyzer chain, and aliases the original name to the new collection. The Javadoc carries an unambiguous warning: "Reindexing is potentially a lossy operation — some indexed data that is not available as stored fields may be irretrievably lost."

In production we recommend reindexing from the system of record whenever possible:

- Anything indexed but not stored is gone forever once you accept the swap.
- You can apply schema-aware transformations during the copy.
- You exercise the actual production ingest path, which catches latent bugs.

Use REINDEXCOLLECTION for index-only shape changes (more shards, different similarity, new docValues fields on already-stored fields). Reach for a custom indexer for schema and content changes.

2.9.4 Bulk reindex with SolrJ

The pattern: read the source with cursorMark, optionally transform, batch-write the target with bounded parallelism.

cursorMark is the only correct way to read at scale. Conventional start/rows pagination degrades quadratically as start grows; cursorMark is constant-time per page. It requires a total-ordering sort, conventionally on the uniqueKey.

```
import java.util.*;
import java.util.concurrent.*;
import org.apache.solr.client.solrj.SolrQuery;
import org.apache.solr.client.solrj.response.QueryResponse;
import org.apache.solr.common.SolrDocument;
import org.apache.solr.common.SolrInputDocument;
import org.apache.solr.common.params.CursorMarkParams;

public final class BulkReindexer {

    public static void reindex(SolrClient cloud, String src, String dst) throws Exception {
        SolrQuery q = new SolrQuery("*:*")
            .setRows(1000)
            .addSort("id", SolrQuery.ORDER.asc);    // MUST sort on uniqueKey

        String cursor = CursorMarkParams.CURSOR_MARK_START;
        ExecutorService pool = Executors.newFixedThreadPool(4);
        List<Future<?>> inFlight = new ArrayList<>();

        try {
            while (true) {
                q.set(CursorMarkParams.CURSOR_MARK_PARAM, cursor);
                QueryResponse resp = cloud.query(src, q);

                List<SolrInputDocument> batch = new ArrayList<>(resp.getResults().size());
                for (SolrDocument d : resp.getResults()) batch.add(transform(d));

                // Bounded in-flight batches = backpressure.
                while (inFlight.size() >= 8) inFlight.remove(0).get();
                inFlight.add(pool.submit(() -> { cloud.add(dst, batch); return null; }));

                String next = resp.getNextCursorMark();
                if (cursor.equals(next)) break;
                cursor = next;
            }
            for (Future<?> f : inFlight) f.get();
            cloud.commit(dst);
        } finally {
            pool.shutdown();
            pool.awaitTermination(1, TimeUnit.MINUTES);
```

```
        }
    }

    private static SolrInputDocument transform(SolrDocument src) {
        SolrInputDocument out = new SolrInputDocument();
        for (String field : src.getFieldNames()) {
            if ("_version_".equals(field)) continue;   // ALWAYS strip _version_
            out.addField(field, src.getFieldValue(field));
        }
        return out;
    }
}
```

Things that will bite you if you ignore them:

- Always strip _version_. Copying the source _version_ into the target makes Solr think the document already exists at that version and rejects it. version=0 (omitted) is what you want.
- Sort on uniqueKey. cursorMark requires a total-ordering sort.
- Commit only at the end. autoSoftCommit in solrconfig.xml handles visibility during the run; you don't want to force hard commits per batch.
- Bounded in-flight batches are your backpressure. rows=1000 × inFlight 8 = roughly 8,000 in-flight documents at any moment, which is a reasonable starting point for a 3–4 node cluster.

2.9.5 Verification

A reindex is not done when the indexer says "done". It's done when you've proved the new collection is at least as good as the old one.

1. Document counts. numDocs of the source should equal numDocs of the target (unless your transform legitimately drops documents):

```
long total = client.query(coll, new SolrQuery("*:*").setRows(0))
    .getResults().getNumFound();
```

2. Top-N overlap on a head-query set. For your top 50 head queries, compare top-10 result IDs between old and new. Identical isn't required — relevance is allowed to improve — but a big shift on a benign change is a smell. Common targets: overlap@10 0.7 for a "minor" schema

change, 0.5 for a deliberate relevance change, < 0.3 is almost always a bug (most often a forgotten copyField).

3. Facet histograms. For categorical fields, compare facet=true&facet.field=X&facet.limit=-1 term-by-term. Anything missing in the new collection is suspicious.

4. Field coverage. For each field, run field:[* TO *] and confirm counts are in the right ballpark. A field that was non-zero in the old collection and is suddenly zero in the new is a copyField that didn't get re-declared or a required="true" that should have been added.

2.9.6 The decision tree

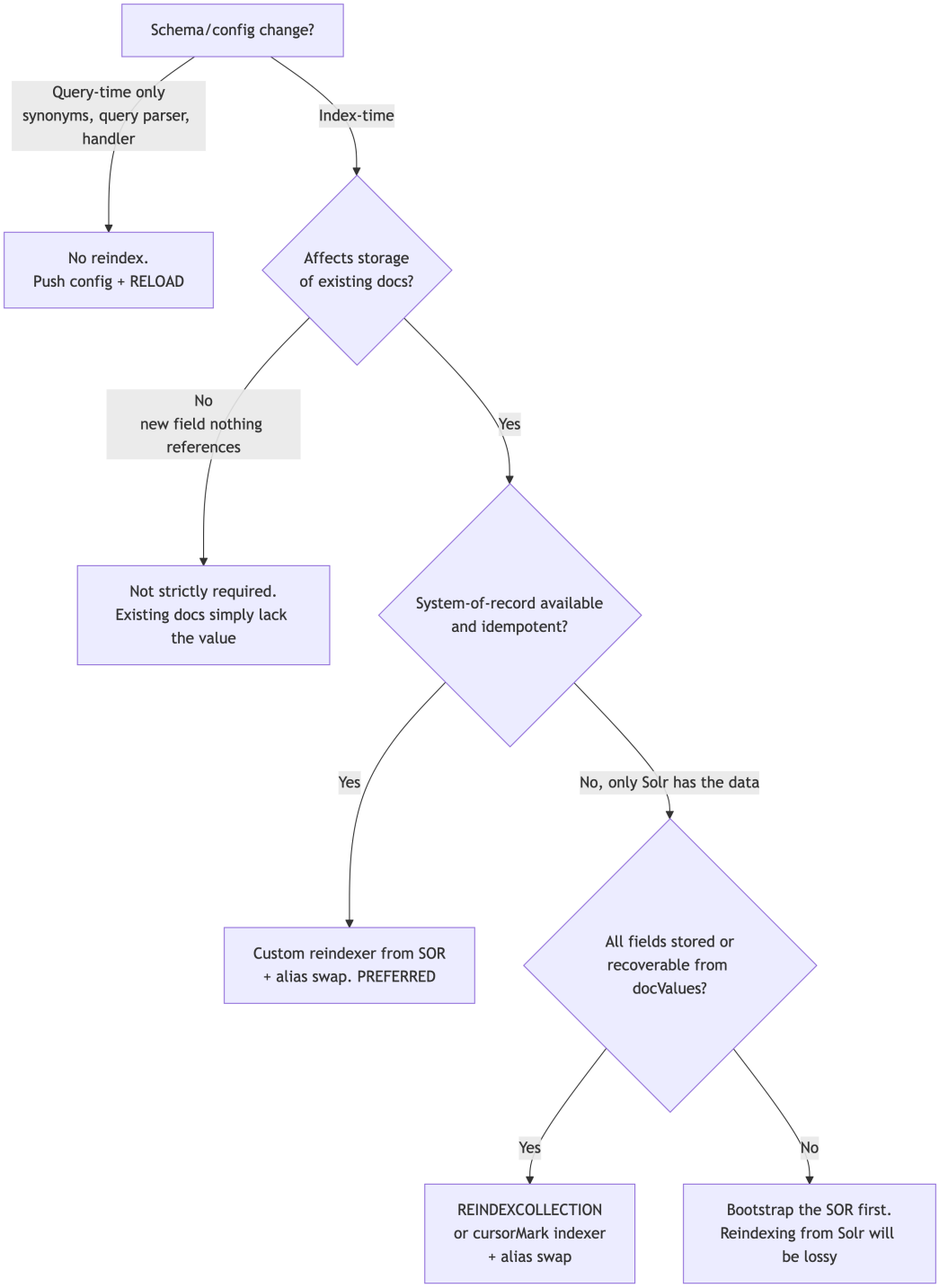

Figure 11: Decision tree for whether a schema or config change requires a full reindex, and which reindex strategy to reach for when it does.

The operational habits that keep you out of trouble:

1. Always read/write through an alias, never a collection name. Costs nothing at steady state, saves you on the first emergency.
2. Keep a one-command rollback ready. Blue/green's whole point is that createAlias(alias, oldCollection) undoes everything in milliseconds.
3. Reindex from your system of record whenever possible. Solr is a search index, not a system of record. Treat it as derived data and you'll sleep better.
4. Verify before you swap. Counts, top-N overlap, facet histograms, field coverage. Five queries' worth of code save five days' worth of incident response.

2.10 Commits: the most over-tuned, least understood knob in Solr

Solr distinguishes two commit kinds:

- Hard commit — fsync segment files to disk; close the current transaction log and open a new one. Durable. Optionally opens a new searcher.
- Soft commit — open a new searcher so recent docs become visible. Does not fsync, does not roll the tlog. Cheap-ish.

Configured in solrconfig.xml:

```
<updateHandler class="solr.DirectUpdateHandler2">
  <autoCommit>
    <maxTime>60000</maxTime>       <!-- hard commit every 60s -->
    <maxDocs>50000</maxDocs>
    <maxSize>512m</maxSize>
    <openSearcher>false</openSearcher>   <!-- do NOT open a searcher; soft commit does
 ↪  that -->
  </autoCommit>
  <autoSoftCommit>
    <maxTime>5000</maxTime>        <!-- new docs become visible every 5s -->
  </autoSoftCommit>
  <updateLog>
    <str name="dir">${solr.ulog.dir:}</str>
  </updateLog>
</updateHandler>
```

The canonical pattern: hard-commit on a long interval with openSearcher=false, soft-commit on a short interval. The hard commit makes data durable; the soft commit makes it visible. They are decoupled.

> Production tip — never send commit=true from a client. It triggers a cluster-wide hard commit that destroys cache warmups and trashes performance. If a third-party indexer insists on it, configure IgnoreCommitOptimizeUpdateProcessorFactory in your update chain to silently drop commit requests.

Why this matters

If you…	You get…
Soft-commit every 100 ms	Cache thrash; warmup queries never finish; latency spikes
Hard-commit on every add	I/O storm; merge thrash; tlog rolls every second
Never commit at all	Tlog grows unboundedly; recovery on restart takes hours
Hard-commit every 60s + soft-commit every 5s	Stable indexing; ~5s visibility lag (NRT)

2.11 Segment merging and "optimize"

A Lucene index is composed of immutable segments. Every soft commit either flushes a new segment or not; every hard commit definitely produces (or merges) segments. Background threads run a MergePolicy (default TieredMergePolicy) that consolidates many small segments into fewer large ones. Deleted documents stay on disk until the segment that contains them is merged away.

A manual optimize (or commit(optimize=true)) forces all segments into one (or a configured number). It is expensive — re-writes the entire index — and almost always wrong on a live cluster. Solr 10 adds LatestVersionMergePolicyFactory to ensure index format compatibility with the next major version during upgrades; that is a legitimate use of forced merging. For "performance tuning", let the default merge policy do its job.

Exercises

1. Design a schema for a domain you know. Pick a real domain — TV shows, library books, software bug tickets, anything. Write the managed-schema XML for it. For each field, justify your choice of indexed, stored, docValues, and multiValued. Now: which two fields would you sort on? Which three would you facet on? Did your schema enable that, or did you need to add docValues=true after the fact?

2. Trace an atomic vs. in-place update. Take any field in your schema from exercise 1 and decide: would a Solr {"id": "...", "fieldname": {"set": NEW}} update on this field go through the in-place path or fall back to atomic? Walk through the five criteria in §2.8.2. Then send a real request with update.partial.requireInPlace=true and verify your prediction.
3. Plan a zero-downtime reindex. Imagine your existing schema has description as string (a real mistake — should be text_en). Document the steps to fix this in production without any read-side downtime, using the blue/green alias swap in §2.9.3. List: what you create, what you populate, what you verify before the swap, and what your rollback command is.
4. Spin up Testcontainers. Write a JUnit 5 test that starts a Solr 10 container with a custom configset, indexes 100 documents from a CSV, and asserts that a faceted query returns the expected bucket counts. Aim for the whole test to run in under 15 seconds.

2.12 Chapter summary

- Schemas are explicit contracts. Use managed-schema, disable schemaless in production, and choose indexed/stored/docValues/multiValued deliberately per field.
- The text analysis chain (Tokenizer + Filters) is the most important relevance lever you have; keep index/query symmetric except for query-time synonym expansion.
- Use CloudSolrClient for connections; batch with add(Collection) or use ConcurrentUpdateJettySolrClient for fire-and-forget throughput.
- Test against a real Solr. Use SolrContainer from Testcontainers with apache/solr:10.0.0. One static container per test class; clean the index in @BeforeEach rather than restarting.
- Partial updates come in three flavors: full replace (default), atomic (server-side modifier maps — still a full reindex underneath), and in-place (genuine docValues-only update for single-valued numeric fields). Use update.partial.requireInPlace=true in Solr 10 to fail fast when an in-place update would silently fall back to atomic.
- Optimistic concurrency via _version_ (-1 = must not exist, 1 = must exist, N = exact match) composes freely with all three update flavors.
- Reindex when index-time analysis, field types, codec, luceneMatchVersion, or copyField declarations change. Skip reindexing for query-time-only changes (synonyms!), new untouched fields, and solrconfig.xml knobs that don't affect storage.
- Production reindexing is blue/green with collection aliases: build into a new collection, validate, atomically swap the alias, keep the old around for rollback. Read from your system of record when possible; REINDEXCOLLECTION is convenient but lossy for non-stored fields.
- Decouple durability (hard commit, slow) from visibility (soft commit, fast). Never send commit=true from clients. Don't optimize.

Search

What this chapter covers and why it matters. This is where the user's experience is actually made or broken. We will walk the query lifecycle, the parsers, faceting, scoring (BM25 in honest detail), vector search with DenseVectorField and the knn query parser, hybrid retrieval, spell checking, and highlighting — all with concrete SolrJ code.

Prerequisites: Chapter 1 (inverted index, query types, cost model) and Chapter 2 (you need a schema and an index to query against).

You will learn: How a Solr query is composed from q, fq, fl, sort, and friends; why eDisMax is the right default parser for user-facing search; how to use the JSON Facet API for modern aggregations; how BM25 actually scores; how vector search and HNSW work, with a worked cosine-similarity example; how to combine lexical and vector retrieval via Reciprocal Rank Fusion; how to debug "why is this result first?" in production; and how to build query and document autocomplete.

What you should not skip: §3.5 (BM25 in honest detail) and §3.6 (vector search and hybrid retrieval — the future of every search bar).

3.1 Query anatomy

A Solr query is a set of HTTP parameters. The important ones:

Table 1: The core Solr query parameters every search request is built from, with what each one controls and a concrete example.

Param	Meaning	Example
q	The main query (scored)	q=stranger things
fq	Filter queries (not scored, cached)	fq=status:ongoing&fq=platforms:Netflix
fl	Field list to return	fl=id,title,rating,score
sort	Sort spec	sort=score desc, rating desc
start, rows	Pagination	start=0&rows=20
defType	Query parser	defType=edismax

Param	Meaning	Example
wt	Response writer	wt=json (default)
debugQuery	Show parsed query + scoring	debugQuery=true

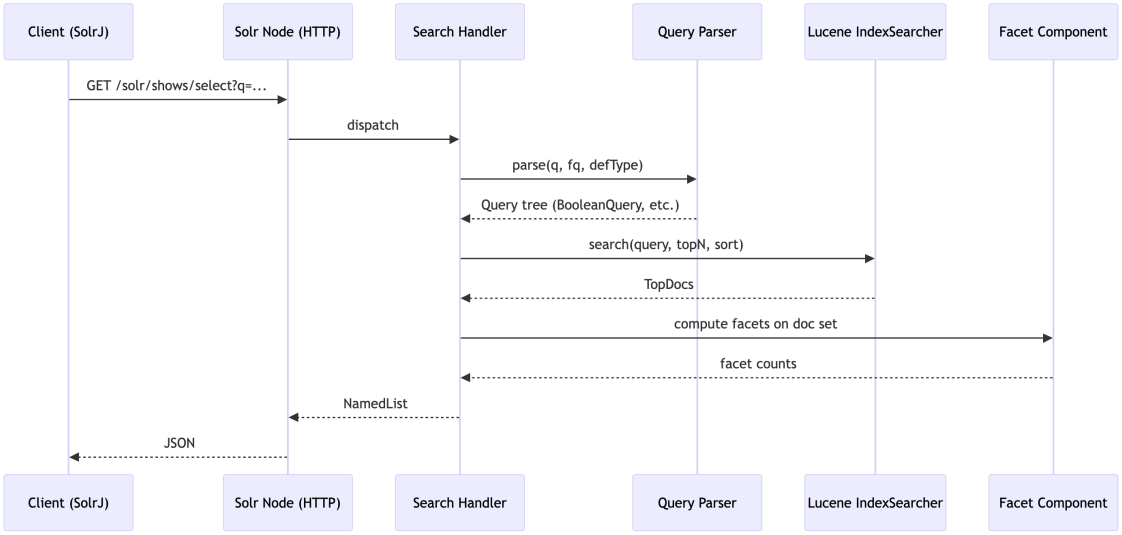

Figure 1: Lifecycle of a Solr query from the SolrJ client through the search handler, query parser, Lucene IndexSearcher, and facet component back to JSON.

3.2 Query parsers: Standard, DisMax, eDisMax

- Standard (Lucene) parser (defType=lucene, default): full Lucene syntax — fielded queries, boolean operators, ranges, fuzzy, etc. Powerful but surfaces parse errors to end users. Bad for raw user input.
- DisMax: simpler, forgiving syntax. Designed for user-facing search across multiple fields.
- eDisMax (extended DisMax): superset of DisMax. Default choice for any user-facing search bar (product, content, or otherwise).

Why eDisMax wins for production

eDisMax adds: support for full Lucene syntax inside the q string (so power users can do platforms:Netflix), proper escaping of bad input, phrase boosts on bigrams/trigrams (pf2, pf3), boost queries, and multiplicative boost functions.

Key eDisMax params:

Table 2: The eDisMax parameters that matter in production, covering per-field boosts, phrase boosts, slop, minimum-should-match, tie-breaking, and additive plus multiplicative boosting.

Param	What it does
qf	Query fields with per-field boosts: qf=title^3 description^1 cast^2
pf	Phrase boost: if the full query appears as a phrase in these fields, boost it
pf2, pf3	Phrase boost on bigrams / trigrams
ps, ps2, ps3	Phrase slop (positions allowed between phrase tokens)
mm	Minimum should match: mm=2<-1 5<80% (“for queries up to 2 terms all must match; for 5+ terms at least 80%”)
tie	Tie-breaker between dismax (0.0) and sum (1.0); typically 0.1
bq	Additive boost query: bq=status:ongoing^5
boost	Multiplicative boost function

A realistic shows-catalog query — “find a stranger-things-like show on Netflix, prefer newer and ongoing series”:

```
GET /solr/shows/select?
  defType=edismax
  &q=stranger things
  &qf=title^3 cast^2 description^1 text_all^0.5
  &pf=title^5
  &pf2=title^3 description^1
  &ps=2
  &mm=2<-1 5<75%
  &tie=0.1
  &bq=status:ongoing^5
  &boost=recip(ms(NOW,added_at),3.16e-11,1,1)
  &fq=platforms:Netflix
  &fq=release_year:[2015 TO 2026]
  &fl=id,title,rating,platforms,score
  &facet=true&facet.field=genres&facet.field=platforms
  &rows=20
```

The boost=recip(...) clause multiplies the score by a value that decays with document age — newer shows bubble up. 3.16e-11 is “per millisecond” for a 1-year half-life. The bq=status:ongoing^5 adds an additive nudge for shows that are still being made, since a search bar usually wants to surface fresh content.

SolrJ equivalent

```
import org.apache.solr.client.solrj.SolrQuery;
import org.apache.solr.client.solrj.response.QueryResponse;
import org.apache.solr.common.SolrDocumentList;

SolrQuery q = new SolrQuery();
q.setRequestHandler("/select");
q.setParam("defType", "edismax");
q.setQuery("stranger things");
q.setParam("qf",   "title^3 cast^2 description^1 text_all^0.5");
q.setParam("pf",   "title^5");
q.setParam("pf2",  "title^3 description^1");
q.setParam("ps",   "2");
q.setParam("mm",   "2<-1 5<75%");
q.setParam("tie",  "0.1");
q.setParam("bq",   "status:ongoing^5");
q.setParam("boost","recip(ms(NOW,added_at),3.16e-11,1,1)");
q.addFilterQuery("platforms:Netflix");
q.addFilterQuery("release_year:[2015 TO 2026]");
q.setFields("id", "title", "rating", "platforms", "score");
q.setRows(20);

QueryResponse resp = client.query("shows", q);
SolrDocumentList docs = resp.getResults();
```

3.3 Faceting

Solr exposes faceting in three ways:

1. Legacy field/range/query/pivot facets via facet.* params. Simple, flat.
2. JSON Facet API (recommended). Nested, composable, supports aggregations like avg, percentile, unique. Introduced in Solr 6.0, per Yonik Seeley’s original announcement

on yonik.com (April 17, 2015): “Solr 5 has a completely re-written faceted search and analytics module with a structured JSON API.”

3. Pivot facets for hierarchical multi-field facets (the JSON API supersedes most uses).

Legacy field facet

```
&facet=true&facet.field=genres&facet.field=platforms&facet.mincount=1&facet.limit=20
```

Range facet

```
&facet=true
&facet.range=release_year&facet.range.start=1990&facet.range.end=2026&facet.range.gap=5
```

JSON Facets — the modern API

```
{
  "query": "stranger things",
  "filter": ["status:ongoing"],
  "limit": 20,
  "facet": {
    "platforms": {
      "type": "terms",
      "field": "platforms",
      "limit": 10,
      "facet": {
        "avg_rating": "avg(rating)",
        "ongoing_count": { "type": "query", "q": "status:ongoing" }
      }
    },
    "year_buckets": {
      "type": "range",
      "field": "release_year",
      "start": 1990, "end": 2026, "gap": 5
    },
    "genres": { "type": "terms", "field": "genres", "limit": 20 }
  }
```

```
}
```

SolrJ:

```
import org.apache.solr.client.solrj.request.json.JsonQueryRequest;
import org.apache.solr.client.solrj.request.json.TermsFacetMap;
import org.apache.solr.client.solrj.request.json.RangeFacetMap;

TermsFacetMap platformFacet = new TermsFacetMap("platforms").setLimit(10)
    .withStatSubFacet("avg_rating", "avg(rating)");
RangeFacetMap yearRange = new RangeFacetMap("release_year", 1990, 2026, 5);

JsonQueryRequest req = new JsonQueryRequest()
    .setQuery("stranger things")
    .withFilter("status:ongoing")
    .setLimit(20)
    .withFacet("platforms", platformFacet)
    .withFacet("year_buckets", yearRange);

QueryResponse resp = req.process(client, "shows");
```

> Production tip — facet on docValues=true string fields. Faceting on a tokenized text field gives you stems instead of category names. Faceting on a non-docValues field allocates a giant uninverted field cache on heap.

3.4 Matching tools

Table 3: A cheat sheet of the eight common matching idioms — exact, prefix, wildcard, fuzzy, phrase, slop, range, boolean — with the literal query syntax for each.

Need	Syntax
Exact match	q=platforms:Netflix (string field)
Prefix	q=title:strang*
Wildcard	q=title:t?ings
Fuzzy (edit distance 2)	q=title:thinngs~
Phrase	q=title:"stranger things"
Phrase with slop	q=title:"stranger things"~2
Range	q=release_year:[2015 TO 2026} ({ = exclusive)

Need	Syntax
Boolean	q=stranger AND (things OR strange) NOT comedy

The execution costs of each of these are covered in detail in §1.4. The short version: wildcards and leading-wildcards are slow; fuzzy queries are slower. If you need spelling tolerance, prefer the Spell Checker (§3.7) or an n-gram field. If you need leading-wildcard, use ReversedWildcardFilterFactory at index time.

3.5 Relevance scoring — BM25 in honest detail

Solr 10 ships BM25Similarity as the default. BM25 became the default similarity in Solr 5.0.0 (SOLR-8270/SOLR-8271): "SchemaSimilarityFactory has been modified to use BM25Similarity as the default for fieldTypes that do not explicitly declare a Similarity. The legacy behavior of using ClassicSimilarity as the default will occur if the luceneMatchVersion for the collection is less than 5.0." TF-IDF (ClassicSimilarity) survives but is legacy.

The Lucene Javadoc for BM25SimilarityFactory defines the parameters explicitly: "k1 (float): Controls non-linear term frequency normalization (saturation). The default is 1.2 · b (float): Controls to what degree document length normalizes tf values. The default is 0.75."

The BM25 formula

In Chapter 1 we promised "BM25 in one paragraph, no math." Here is the actual formula — it's worth seeing once because every tuning decision and every weird scoring artifact you'll encounter traces back to it.

For a document D and a query Q made of terms $t_1, t_2, ..., t_n$:

$$\text{score}(D,Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{\text{tf}(t,D) \cdot (k1 + 1)}{\text{tf}(t,D) + k1 \cdot (1 - b + b \cdot |D|/\text{avgdl})}$$

What each piece means:

- tf(t,D) — how many times term t appears in document D.
- IDF(t) — inverse document frequency. Big for rare terms, small for common terms.
- |D| — length of document D (token count) in the scored field.
- avgdl — average document length across the corpus, per field.
- k1 — TF saturation knob, default 1.2.

- b — length-normalization knob, default 0.75.

The shape of the formula is the entire story:

- The numerator is roughly proportional to tf(t,D) but capped: as tf grows large, the contribution approaches a horizontal asymptote. Repeated mentions of a term yield diminishing returns. That's saturation, controlled by k1.
- The denominator contains (1 - b + b · |D|/avgdl). When the document is the average length, this term is 1. When the document is longer than average, the denominator grows and the score shrinks. That's length normalization, controlled by b. Set b=0 to disable it entirely; set b=1 for full normalization.
- IDF(t) is a per-term multiplier in front. A term appearing in 1% of documents gets a much larger IDF than a term appearing in 50%. This is why rare terms dominate scoring.

Intuition for the levers:

- k1=1.2 (default). Smaller k1 means TF saturates faster — repeated mentions barely help. Larger k1 rewards verbose matches. For short show titles where each token is rare and meaningful (Stranger Things, Severance), k1=1.2 is fine; for long synopsis fields you might lower it to 0.9 so a synopsis that mentions "running" five times doesn't dominate.
- b=0.75 (default). Higher b punishes long docs more. A shows catalog has wildly uneven description lengths (a one-line tagline vs. a 500-word recap) so b=0.75 is a good default; if you mix tweet-length and article-length content in the same field, tune b per field-type.

Override per field type:

```
<fieldType name="text_en" class="solr.TextField">
  <similarity class="solr.BM25SimilarityFactory">
    <float name="k1">1.0</float>
    <float name="b">0.6</float>
  </similarity>
  <analyzer>...</analyzer>
</fieldType>
```

Always tune by measuring (NDCG@10, MRR, click-through). Don't tune by intuition. For a deep dive on BM25's derivation and tuning, the canonical reference is Elastic's "The BM25 algorithm and its variables" — the engine is the same Lucene one Solr uses.

Function queries for custom scoring

Function queries compute a score from doc fields:

```
q=stranger things&boost=product(popularity, recip(ms(NOW,added_at),3.16e-11,1,1))
```

This multiplies score by popularity × (1 / (age_factor + 1)) — so a stale but popular show competes with a fresh-but-less-popular one. Useful for time-decay, popularity blending, regional preference. Common functions: sum, product, div, recip, log, pow, if, geodist, ms, query.

3.6 Vector search and hybrid retrieval

The end-to-end picture: two parallel pipelines

Before walking into the vector mechanics, it helps to see how lexical and vector retrieval sit side by side in the same Solr collection. At indexing time, every document feeds two completely separate index structures within the same segment:

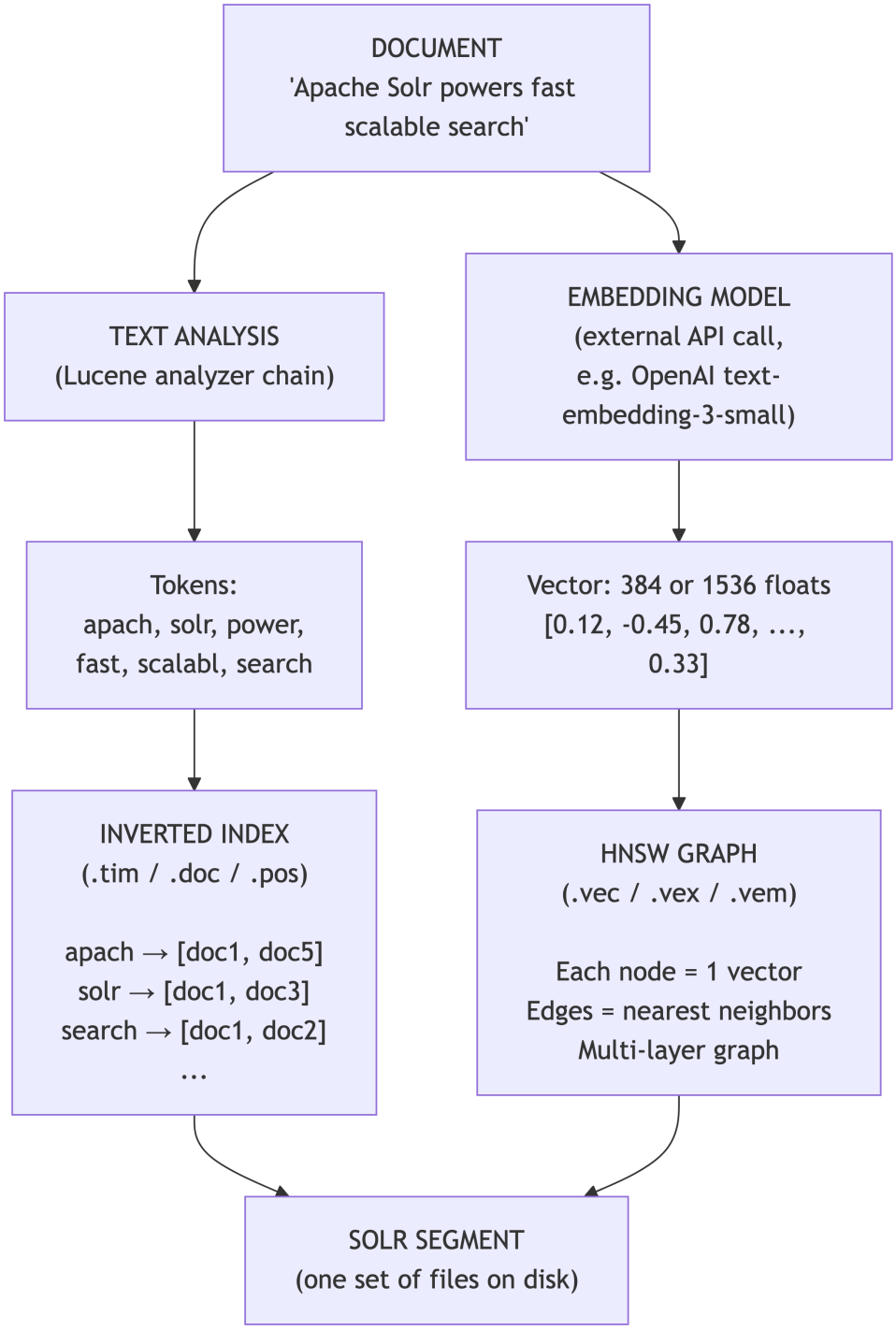

Figure 2: The two parallel indexing pipelines inside a single Solr segment: the analyzer builds an inverted index while the embedding model populates the HNSW vector graph.

The same SolrInputDocument you indexed in §2.6 populated both sides: the tokenized title and description fields built the inverted index; the embedding field built the HNSW graph. At

query time, two parallel paths read from those two structures and produce two ranked lists. Then a fusion step combines them.

DenseVectorField and the knn parser (Solr 9.0+)

Configured via the field type in §2.3. To search:

```
q={!knn f=embedding topK=20}[0.012, -0.183, ..., 0.044]
```

That syntax says: "find the top-20 nearest neighbors of the given query vector in the embedding field." Solr uses the Lucene HNSW (Hierarchical Navigable Small World) graph internally.

HNSW: how the graph walk actually works

HNSW is a layered proximity graph. Every vector you index becomes a node. Edges connect each node to its approximate nearest neighbors (up to M = hnswMaxConnections, default 16). The trick: not every node lives on every layer. A small fraction of nodes are randomly promoted to higher layers, where they form a sparser graph with longer edges. The top layer has only a handful of nodes; the bottom layer (layer 0) has every node.

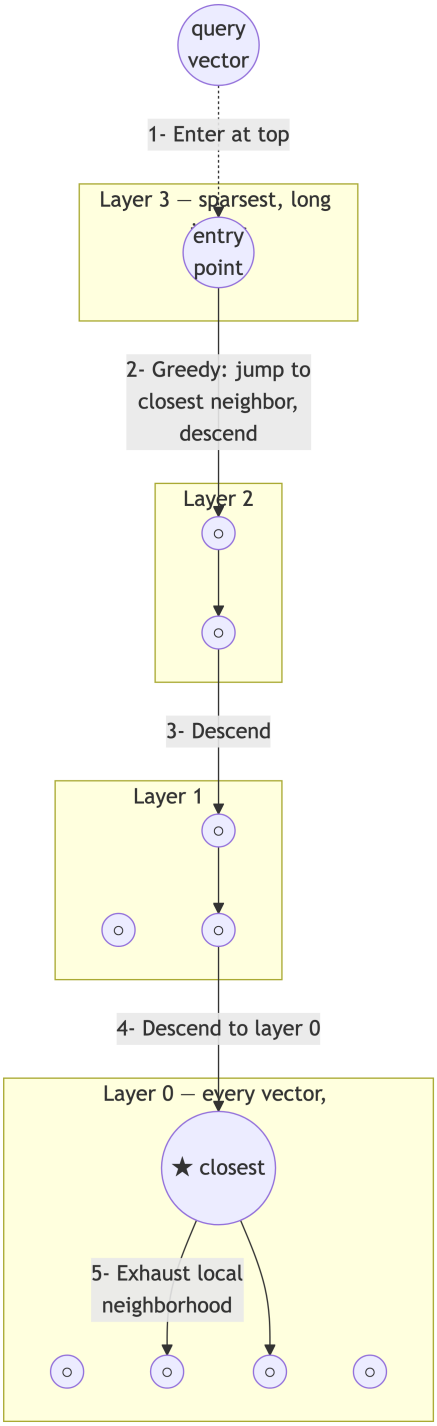

Figure 3: The HNSW layered greedy walk: enter at a sparse top layer, jump to the locally closest neighbor, descend layer by layer, and exhaust the dense bottom layer.

The walk has two phases — a greedy descent through the upper layers, then a wider sweep at layer 0:

1. Enter at a fixed entry point in the top layer.
2. Greedy step: look at all neighbors of the current node. If any neighbor is closer to the query than the current node, move to the closest one. Repeat until no neighbor improves distance. This finds the local minimum on this layer.
3. Descend to the next layer, using the local minimum as the entry point.
4. Repeat until you reach layer 0.
5. At layer 0, do an exhaustive search of a candidate pool of size efSearch (default = topK). Return the topK closest from that pool.

Why this is fast: each layer halves the search space. The top layers do long jumps over the vector space, like the express lanes of a skip list; the bottom layer does fine-grained local refinement. Visiting ~20 nodes out of 15,000 vectors is typical, vs. 15,000 brute-force distance computations.

Approximate, not exact — the algorithm trades a small amount of recall for orders of magnitude in speed.

The main knobs you have on HNSW are graph density at build time, build-time search effort, and query-time search effort. Defaults work well for most workloads; the Solr Dense Vector Search reference covers all of them. For the algorithm itself, the original paper by Malkov & Yashunin ("Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs") is the canonical reference.

Cosine similarity, with an example

The HNSW graph walk finds candidates close to the query, but "close" needs a distance metric. The most common for embedding vectors is cosine similarity: it measures the angle between two vectors, ignoring their magnitudes.

Two vectors pointing in the same direction → cosine 1.0. Perpendicular → 0.0. Opposite → -1.0. In practice with real text embeddings you almost always see values in roughly [0.0, 1.0] — strongly negative similarities are rare because nothing in natural-language space points the opposite direction of anything else.

A worked example with four documents. The numbers below are illustrative, not measured — they are the shape you should expect from a modern sentence-embedding model (e.g., sentence-transformers/all-MiniLM-L6-v2, OpenAI text-embedding-3-small). Exact values depend on the model, the corpus, and the prompt template; if you reproduce this exercise locally you will see different absolute numbers but the same ordering.

Query: "dark sci-fi thriller about memory and identity". Suppose we embed this query and the description of four shows from our seed catalog:

Document	Description (excerpt)	Cosine to query (illustrative)
SVR-001 (Severance)	“Workers undergo a procedure that surgically divides their memories between their work and personal lives.”	~0.85–0.95 — semantic bullseye
BM-001 (Black Mirror)	“Anthology series exploring a twisted, high-tech multiverse where humanity’s greatest innovations and darkest instincts collide.”	~0.70–0.80 — same concept space, different words
BRG-001 (Bridgerton)	“Wealth, lust, and betrayal set in the backdrop of Regency-era England.”	~0.05–0.15 — unrelated topic
MH-001 (Mindhunter)	“FBI agents interview imprisoned serial killers to understand their psychology.”	~0.30–0.45 — shares “dark” and “psychological” but not sci-fi

Three things worth noticing:

1. Direct keyword overlap helps (SVR-001 shares “memories”), but it’s not required. BM-001 has near-zero word overlap with the query yet scores well — that’s the semantic similarity working as advertised.
2. Polysemy is partially handled (MH-001). It is “dark” and “psychological” but not “sci-fi,” and the embedding model knows that. The score drops below BM-001 but stays above totally unrelated shows like BRG-001.
3. Magnitude doesn’t matter. A 6-word title and a 500-word synopsis that both genuinely talk about the same concept can score similarly — cosine ignores vector length. (Compare with BM25, which heavily penalizes long documents.)

similarityFunction in DenseVectorField

Solr exposes three similarity functions on the field type:

```
<fieldType name="knn_vector_384" class="solr.DenseVectorField"
        vectorDimension="384"
        similarityFunction="cosine"      <!-- options: dot_product, euclidean, cosine -->
        knnAlgorithm="hnsw"/>
```

Which one to choose:

- cosine — safe default. Works for any embedding model.
- dot_product — faster than cosine if you can guarantee your vectors are pre-normalized to unit length. Most modern embedding models (OpenAI's text-embedding-3-*, Sentence-Transformers' all-MiniLM-L6-v2, Cohere's embed-v3) return unit vectors by default. When they do, dot_product and cosine produce identical rankings but dot_product skips the normalization step.
- euclidean — only correct if your embedding model was trained with Euclidean distance. Most aren't. If in doubt, don't.

  Production tip. Confirm your embedding model's output is normalized once (compute the L2 norm of a sample vector — it should be very close to 1.0), then use dot_product. Free performance.

For the underlying math, see the Wikipedia article on cosine similarity or Pinecone's introduction to cosine similarity.

What's new in Solr 10 for vectors

The Solr 10.0 reference guide explicitly lists, as new features: "Scalar and binary quantized dense vectors are now supported for DenseVectorField (SOLR-17780, SOLR-17812). Quantization reduces memory consumption and can improve search performance at some cost to accuracy." And: "GPU-accelerated approximate nearest neighbor search is now available via the cuVS-Lucene pluggable codec (SOLR-17892)."

A binary-quantized field (1 bit per dimension instead of 32):

```
<fieldType name="binary_quantized_vector"
 ↪  class="solr.BinaryQuantizedDenseVectorField"
        vectorDimension="384"/>
<field name="embedding_bq" type="binary_quantized_vector" indexed="true"
 ↪  stored="true"/>
```

This reduces memory footprint roughly 32× at a measured accuracy cost — appropriate for very large vector corpora.

Hybrid search: combining keyword + vector

The two ranked lists you've now produced — BM25 results and KNN results — answer different questions and have incomparable score scales. A BM25 score of 12.5 means nothing in isolation;

a cosine score of 0.92 is on [-1, 1]. You cannot just average them or take a weighted sum without first re-scaling, and re-scaling is fragile across queries.

This is the entire reason Reciprocal Rank Fusion exists.

Two production-grade approaches:

1. Rerank. Use BM25 to retrieve a generous top-N candidate set; rerank the top-N with a vector query.
2. Reciprocal Rank Fusion (RRF). Run BM25 and KNN as two retrievers; fuse their ranked lists by 1 / (k + rank) and sort. Score-agnostic — only the rank position matters.

Rerank approach (works on Solr 10.0 today):

```
q=stranger things
&defType=edismax&qf=title^3 description^1
&rq={!rerank reRankQuery=$rqq reRankDocs=200 reRankWeight=2}
&rqq={!knn f=embedding topK=200}[...query vector...]
```

This says: “BM25 retrieves a large pool; rerank the top 200 by blending in a KNN-similarity score weighted 2×.”

RRF in honest detail, with a worked example

Two retrievers return ranked lists. For each document, sum 1/(k + rank_in_that_list) across all retrievers; documents not in a given list contribute 0. Sort descending by the fused score. The constant k = 60 is the canonical Cormack/Clarke/Büttcher (2009) value — small enough to give the top ranks real weight, large enough to keep noisy late ranks from dominating.

$$\mathrm{RRF}(d) = \sum_{i \in R} \frac{1}{k + \mathrm{rank}_i(d)}$$

where R = set of retrievers, k = 60
rank_i(d) = rank of doc d in retriever i (or ∞ if not present)

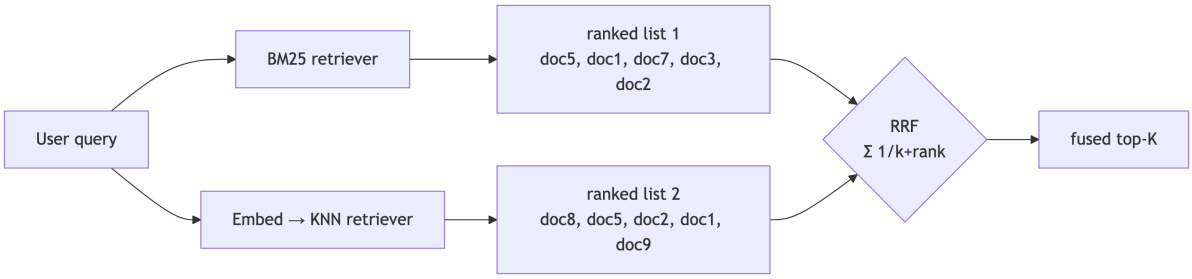

Figure 4: Reciprocal Rank Fusion combines a BM25 ranked list with a KNN vector ranked list by summing 1/(k+rank), making the score-scale mismatch irrelevant.

Suppose, for the query "dark sci-fi thriller about memory and identity" (continuing the example from §3.6's cosine table), the two retrievers return:

- BM25 ranks: MH-001 (#1, score 12.5), BB-001 (#2, 11.2), BM-001 (#3, 10.8), WW-001 (#4, 9.5), BRG-001 (#5, 8.3)
- KNN ranks: SVR-001 (#1, cos 0.92), MH-001 (#2, 0.89), BM-001 (#3, 0.85), BB-001 (#4, 0.81), WW-001 (#5, 0.78)

The raw scores live on totally different scales — BM25 is unbounded positive, cosine is [-1, 1]. Trying to add them directly would be nonsense. RRF throws away the scores and uses only ranks:

Document	BM25 rank	KNN rank	RRF calculation	RRF score
MH-001	1	2	1/(60+1) + 1/(60+2)	0.03253
BB-001	2	4	1/(60+2) + 1/(60+4)	0.03175
BM-001	3	3	1/(60+3) + 1/(60+3)	0.03175
SVR-001	—	1	0 + 1/(60+1)	0.01639
WW-001	4	5	1/(60+4) + 1/(60+5)	0.03101
BRG-001	5	—	1/(60+5) + 0	0.01538

Sorted final ranking:

1. MH-001 (Mindhunter) — strong in both lists. Wins decisively despite not being a sci-fi show, because both retrievers agree on its "dark psychological" character.
2. BB-001 (Breaking Bad) — good in both. Second. Same kind of cross-retriever agreement.
3. BM-001 (Black Mirror) — good in both, tied with BB-001.
4. WW-001 (Westworld) — present in both but ranked lower; still beats the vector-only hit.
5. SVR-001 (Severance) — vector-only hit. A pure semantic match the keyword retriever missed because the query didn't share many lexical tokens with the synopsis. This is exactly what RRF protects against in pure-BM25 systems: semantic matches that have no keyword overlap.

The key insight: documents that appear in both lists at reasonable ranks systematically beat documents that appear in only one list at the top rank. Cross-retriever agreement is the strongest signal you have for relevance. This is why RRF works so well in practice despite being almost trivially simple.

Important version note. Native Reciprocal Rank Fusion is tracked in Apache JIRA as SOLR-17319. The JIRA records: "Fix Version/s: 10.1, 9.11". RRF is not in Solr 10.0.0 — you must wait for Solr 10.1 (or Solr 9.11 on the 9.x line) for the native query-parser form. In the meantime, do RRF in your application layer (run a BM25 query and a KNN query in parallel, merge results client-side) or use the rerank pattern above.

```
// Client-side RRF in SolrJ (works on Solr 10.0)
record Ranked(String id, int rank) {}
List<Ranked> bm25 = runBm25(...);
List<Ranked> knn  = runKnn(...);
int k = 60;
Map<String, Double> fused = new HashMap<>();
for (Ranked r : bm25) fused.merge(r.id(), 1.0 / (k + r.rank()), Double::sum);
for (Ranked r : knn)  fused.merge(r.id(), 1.0 / (k + r.rank()), Double::sum);
List<String> finalRanking = fused.entrySet().stream()
    .sorted(Map.Entry.<String,Double>comparingByValue().reversed())
    .map(Map.Entry::getKey).toList();
```

> Production tip — tune k only if you've measured. The canonical k=60 is robust across a wide range of retrievers and corpora. Lower k (e.g., 10) makes top-rank agreement more dominant; higher k flattens the curve and lets mid-ranks contribute more. Most teams that tune k are reaching for the wrong knob — usually the actual fix is in retriever quality (better embeddings, better analyzer chain).

3.7 Spell checking and suggestions

solrconfig.xml:

```
<searchComponent name="spellcheck" class="solr.SpellCheckComponent">
  <lst name="spellchecker">
    <str name="name">default</str>
    <str name="classname">solr.DirectSolrSpellChecker</str>
    <str name="field">text_all</str>
    <str name="distanceMeasure">internal</str>
    <float name="accuracy">0.5</float>
```

```
    <int name="maxEdits">2</int>
  </lst>
</searchComponent>

<requestHandler name="/select" class="solr.SearchHandler">
  <lst name="defaults"><str name="spellcheck.dictionary">default</str></lst>
  <arr name="last-components"><str>spellcheck</str></arr>
</requestHandler>
```

```
SolrQuery q = new SolrQuery("stragner thigns");      // user misspelled "stranger things"
q.setParam("spellcheck", true);
q.setParam("spellcheck.collate", true);
QueryResponse resp = client.query("shows", q);
resp.getSpellCheckResponse().getCollatedResults();    // returns "stranger things"
```

3.8 Highlighting

```
&hl=true&hl.fl=title,description&hl.fragsize=160&hl.tag.pre=<mark>&hl.tag.post=</mark>
```

Returns matched snippets per doc with HTML markers around hit terms. Use the unified highlighter (hl.method=unified) for performance on long fields with positions+offsets indexed.

3.9 Result grouping with the Collapse query parser

Show one row per franchise, not one row per show in the franchise. Show one row per creator, not one row per each of their shows. The Collapse query parser plus the Expand component is how you do it in Solr 10 without leaving the JSON Facet world or paying the cost of the legacy group=true Result Grouping feature.

The Solr 10 reference guide is explicit: "Collapse & Expand can together do what the older Result Grouping (group=true) does for most use-cases but not all… Generally, you should prefer Collapse & Expand."

How it works:

- {!collapse field=franchise_id} is a post-filter applied after BM25 scoring. For each value of franchise_id, only one representative document survives — by default the highest-scoring

one. You can override with min=release_year (the original), max=rating (best-rated of the franchise), or sort=... for arbitrary tie-breaking.

- The expand component then re-fetches the collapsed siblings for each surviving doc, so the UI can show "and 3 more shows in this franchise" without a second round-trip.

For this example, assume the schema has been extended with a franchise_id field that groups related shows (e.g. all three Star Wars shows in our seed catalog — TM-001, LK-001, AND-001 — share franchise_id = "STAR-WARS"; the Better Call Saul spinoff BCS-001 shares franchise_id = "BREAKING-BAD-UNIVERSE" with BB-001):

```
q=star wars
&fq={!collapse field=franchise_id max=rating}
&expand=true
&expand.rows=4
&fl=id,title,rating,franchise_id
```

From SolrJ:

```
SolrQuery q = new SolrQuery(userQuery)
    .setRequestHandler("/select")
    .addFilterQuery("{!collapse field=franchise_id max=rating}")
    .setFields("id,title,rating,franchise_id");
q.set("expand", "true");
q.set("expand.rows", "4");
QueryResponse rsp = solr.query("shows", q);
Map<String, SolrDocumentList> siblings = rsp.getExpandedResults();   // group_id →
 ↪  siblings
```

Schema requirement. The collapse field must be single-valued and either docValues="true" (recommended — used by default) or indexed="true" with string/numeric type. For SolrCloud, all members of a collapse group must live on the same shard — otherwise collapse happens per-shard and the merged result has duplicates. The standard way to guarantee co-location is the compositeId router's prefix-routing syntax, which uses your uniqueKey (the <uniqueKey> declared in the schema — by default the id field) to route documents:

```
uniqueKey:        id
schema:           <uniqueKey>id</uniqueKey>
indexed id value: "STAR-WARS!TM-001"
                  prefix    suffix
routing:          murmur3("STAR-WARS") % shardCount
```

The string before the ! in the id is hashed; everything after the ! is part of the unique identifier but ignored for routing. So STAR-WARS!TM-001, STAR-WARS!LK-001, and STAR-WARS!AND-001 all land on the same shard. The collapse field (franchise_id in the example) is a separate field whose value matches the prefix ("STAR-WARS") — you'd set both at index time.

> Note on this book's seed dataset. The 30-show table in §2.3 uses simple IDs like ST-001 without prefix routing, because the running examples assume a single-shard dev collection. If you adapt this schema for a multi-shard production deployment and want to collapse on franchise, switch the IDs to the FRANCHISE!showcode form at the start, never later.
>
> Production tip — routing matters. This pattern only works if you commit to it at collection-create time and at index time. Switching routers later requires a full reindex into a new collection (§2.9.3). If you anticipate ever needing collapse, design the id prefix scheme on day 1.
>
> Production tip — single-shard collections sidestep the issue. If your corpus fits in one shard (under ~50M docs), use a single-shard collection. Collapse works correctly with no routing gymnastics.

3.10 Debugging queries: document transformers and debug=true

When a customer reports "why is this result first?", the answer is one query parameter away.

fl=...,[explain] annotates each returned document with its full BM25 (or LTR) scoring derivation:

```
q=stranger things&fl=id,title,score,[explain style=nl]
```

Returns per-doc:

```
{ "id": "ST-001", "score": 12.83,
  "[explain]": {
    "value": 12.83,
    "description": "sum of:",
    "details": [
      {"value": 8.21, "description": "weight(title:stranger in 142) ..."},
      {"value": 4.62, "description": "weight(title:thing in 142) ..."}
    ]
  } }
```

Other transformers that earn their keep:

Table 4: The document transformers worth knowing in production, what each one adds to the response, and when reaching for it actually saves you time.

Transformer	What it adds	When to use
[explain] or [explain style=nl]	Per-doc score derivation	Triaging "why is this ranked here"
[shard]	The shard the doc came from	Triaging "why does this query miss in production but hit on a single-node dev cluster"
[docid]	Internal Lucene doc-id	Catching segment-merge artifacts; almost never useful in app code
[features store=v1]	LTR feature vector at request time	Logging features for offline model training (see Ch 6)
[child]	Nested children for block-join queries	Validating nested-document indexing
[elevated] / [excluded]	Boolean flag for QueryElevation hits	UI badges showing editorial overrides (see Ch 6)

The companion is debug=true (or finer: debug=query, debug=timing, debug=results), which adds a top-level debug block with the parsed query, per-component timing breakdown, and (with results) the explain text for every returned document. Same syntax across every Solr version since 4.x.

> Production tip. debug=timing is the single most underused operations tool in Solr. It shows the per-SearchComponent time (query, facet, highlight, debug, stats, mlt) — if your p99 latency just doubled, this tells you in one request which component is responsible.

3.11 Autocomplete and suggestions

Solr offers two distinct mechanisms for autocomplete, and they answer different questions. Pick by asking what the dropdown shows:

Table 5: How to choose between SuggestComponent and EdgeNGram instant search based on what the autocomplete dropdown actually shows the user.

The dropdown shows…	Use	Why
Query strings the user can click to run (”stranger”, ”stranger things”, ”strange new worlds”)	SuggestComponent	Ranked, weighted, low-latency phrase completion from a dictionary
Actual shows the user can click to open (a specific title, a specific actor)	EdgeNGram instant search	A regular query over a field indexed with prefix n-grams

For admin tooling that needs raw field values with a prefix lookup and no separate index — e.g., “list all genres starting with sci” — the older TermsComponent is still the right answer. Don’t use it for user-facing autocomplete: SuggestComponent is faster, ranks by weight, and supports infix matching.

3.11.1 SuggestComponent — query completions

What it is. A dedicated component (solr.SuggestComponent) that builds an in-memory or on-disk side structure (FST or Lucene infix index) from a dictionary field, and serves prefix/infix lookups against that structure at sub-millisecond latency.

Why you need it. A plain q=stranger* wildcard query against your main index returns documents, not query strings, and ranks them by BM25 — which is wrong for autocomplete. SuggestComponent returns ranked phrases weighted by whatever you choose (popularity, recency, manually curated weight). It’s the modern, recommended way to do query completions in Solr 10.

The pattern is to build a small _popular_queries collection — one document per suggestable phrase, weighted by aggregated user clicks (Chapter 5) — and point the suggester at that.

Schema (managed-schema):

```
<fieldType name=”text_suggest” class=”solr.TextField” positionIncrementGap=”100”>
  <analyzer>
    <tokenizer name=”standard”/>
    <filter name=”lowercase”/>
  </analyzer>
</fieldType>
```

```
<field name="suggest_phrase" type="text_suggest" indexed="true" stored="true"/>
<field name="suggest_weight" type="plong" stored="true" docValues="true"/>
```

Component + handler (solrconfig.xml):

```
<searchComponent name="suggest" class="solr.SuggestComponent">
  <lst name="suggester">
    <str name="name">queries</str>
    <str name="lookupImpl">AnalyzingInfixLookupFactory</str>
    <str name="dictionaryImpl">DocumentDictionaryFactory</str>
    <str name="field">suggest_phrase</str>
    <str name="weightField">suggest_weight</str>
    <str name="suggestAnalyzerFieldType">text_suggest</str>
    <str name="indexPath">suggester_dir</str>
    <str name="buildOnStartup">false</str>
    <str name="buildOnCommit">false</str>
  </lst>
</searchComponent>

<requestHandler name="/suggest" class="solr.SearchHandler" startup="lazy">
  <lst name="defaults">
    <str name="suggest">true</str>
    <str name="suggest.dictionary">queries</str>
    <str name="suggest.count">10</str>
  </lst>
  <arr name="components"><str>suggest</str></arr>
</requestHandler>
```

Example: build once, then query (SolrJ):

```
import org.apache.solr.client.solrj.impl.CloudSolrClient;
import org.apache.solr.client.solrj.request.QueryRequest;
import org.apache.solr.client.solrj.response.QueryResponse;
import org.apache.solr.client.solrj.response.Suggestion;
import org.apache.solr.common.params.ModifiableSolrParams;

try (var client = new CloudSolrClient.Builder(
        List.of("http://solr-1:8983/solr", "http://solr-2:8983/solr"))
        .withDefaultCollection("popular_queries").build()) {
```

```
    // 1. Build the suggester. Run this from a scheduler, not on every commit.
    ModifiableSolrParams build = new ModifiableSolrParams();
    build.set("qt", "/suggest");
    build.set("suggest.buildAll", "true");
    build.set("distrib", "true");                  // fan out to every shard
    var buildReq = new QueryRequest(build);
    buildReq.setPath("/suggest");
    client.request(buildReq);

    // 2. Look up suggestions for what the user typed.
    ModifiableSolrParams q = new ModifiableSolrParams();
    q.set("qt", "/suggest");
    q.set("suggest.q", "stra");            // matches "stranger things", "strange new worlds", ...
    q.set("suggest.count", "10");
    var qReq = new QueryRequest(q);
    qReq.setPath("/suggest");
    QueryResponse resp = qReq.process(client);

    for (Suggestion s : resp.getSuggesterResponse()
                            .getSuggestions().get("queries")) {
        System.out.printf("%s  (weight=%d)%n", s.getTerm(), s.getWeight());
    }
}
```

> Production tip — never buildOnCommit=true. A rebuild on every soft commit will starve query traffic under any non-trivial write load. Build on a schedule (cron / Kubernetes CronJob) at whatever cadence matches your dictionary churn.
>
> Production tip — build every shard. In SolrCloud the suggester data structure is per-replica. After a build, suggestions reflect only the replicas you built. Pass distrib=true (as above) or fan out explicitly.

Further detail — alternative lookup implementations (BlendedInfix, Fuzzy, FST), suggest.cfq context filtering, expression-based weights — is in the Solr Suggester reference.

3.11.2 EdgeNGram — instant document matches

What it is. A schema pattern where a field is indexed with EdgeNGramFilterFactory, breaking each token into prefix n-grams ("stranger" → "st", "str", "stra", "stran", "strang", "strange",

”stranger”). Querying the indexed prefixes with the user’s partial input becomes a regular term lookup — no wildcards, no separate component.

Why you need it. When the dropdown should show real matching shows or actors (not query strings), you need a regular /select query that matches partial input. EdgeNGram is the standard way to make that fast: the typed input matches indexed prefixes directly, no q=stra* wildcard expansion.

Schema rule that surprises everyone: apply EdgeNGramFilterFactory at index time only. Use a plain analyzer at query time. If you n-gram the query side too, ”stranger” matches anything with ”st”, recall explodes, and relevance collapses.

```
<fieldType name=”text_instant” class=”solr.TextField” positionIncrementGap=”100”>
  <analyzer type=”index”>
    <tokenizer name=”standard”/>
    <filter name=”lowercase”/>
    <filter name=”edgeNGram” minGramSize=”2” maxGramSize=”15”/>
  </analyzer>
  <analyzer type=”query”>
    <tokenizer name=”standard”/>
    <filter name=”lowercase”/>
  </analyzer>
</fieldType>

<field name=”title”          type=”text_general” indexed=”true” stored=”true”/>
<field name=”title_instant”  type=”text_instant” indexed=”true” stored=”false”/>
<field name=”cast_instant”   type=”text_instant” indexed=”true” stored=”false”/>
<copyField source=”title” dest=”title_instant”/>
<copyField source=”cast”  dest=”cast_instant”/>
```

Example — instant search with phrase boost (SolrJ):

```
import org.apache.solr.client.solrj.request.SolrQuery;
import org.apache.solr.client.solrj.response.QueryResponse;

SolrQuery q = new SolrQuery(userInput)
    .setRows(10)
    .setFields(”id”, ”title”, ”platforms”, ”rating”);
q.set(”defType”, ”edismax”);
q.set(”qf”, ”title_instant^1.0 cast_instant^0.7”);
q.set(”pf”, ”title^4.0”);          // phrase boost on the full (non-ngram) field
```

```
q.set("ps", "2");
q.set("timeAllowed", "150");      // typeahead must never block the UI

QueryResponse resp = client.query("shows_instant", q);
```

> Production tip — separate collection, tight timeout. Run instant-search as its own collection (shows_instant) with the EdgeNGram analyzer. The index grows substantially with (minGramSize, maxGramSize), so verify the size impact on a representative sample. Always set timeAllowed=150 (ms) — a partial result beats a slow one when the user is mid-keystroke.

The Solr reference covers EdgeNGramFilterFactory parameters in the Tokenizers and Filters page.

3.11.3 Combining them

A mature autocomplete UX runs both: SuggestComponent on /suggest returns a few popular query phrases at the top of the dropdown, and an instant-search query against shows_instant returns three or four matching shows below (with poster thumbnails, etc.). The two requests fire in parallel from the client; whichever returns first renders first. The popular-queries dictionary feeding SuggestComponent is built from the signals pipeline in Chapter 5 — that's how suggestions stay aligned with what users actually search for.

Exercises

1. Build an eDisMax query in three steps. Start with q=stranger things, defType=edismax, qf=title^3 description^1 cast^2. Run it against the seed catalog. Now add pf=title^5 and ps=2. Run it again with debug=true and find the phrase-boost contribution in the parsed query. Finally, add bq=status:ongoing^5 and boost=recip(ms(NOW,added_at),3.16e-11,1,1). Look at the [explain] for the top result and identify which term contributed what fraction of the score.
2. Decide between Collapse and Result Grouping. A streaming service extends its catalog with episode-level documents and wants the search results page to show one row per show (not per episode), with the highest-rated episode as the representative. Should they use {!collapse field=show_id max=rating} or legacy group=true? Justify using §3.9 and the routing requirements.
3. Hand-compute RRF. Given BM25 ranks [ST-001, BB-001, BM-001, MH-001, SVR-001] and KNN ranks [SVR-001, BM-001, ST-001, WED-001, BB-001], compute the RRF fused

ranking with k=60. Which shows win? Which only-in-one-list show made the top 5? Now change k=10 and recompute — does the ordering change?

4. Spot the slow query. A user complains that q=title:*ing (searching for any title ending in "ing") is slow. Without running it, identify (a) which Lucene data structure is being scanned, (b) why this is more expensive than q=title:stranger, and (c) what schema change you would make to support this access pattern at scale.

3.12 Chapter summary

- Default to eDisMax for any user-facing search. Master qf, pf, mm, tie, bq, boost.
- BM25 is the default similarity (since Solr 6); tune k1 and b only with measured relevance metrics in hand.
- The JSON Facet API (Solr 5+) is the modern way to compute facets and sub-aggregations.
- Vector search in Solr 10 is mature: HNSW with scalar/binary quantization and (optional) GPU acceleration via cuVS. Use the knn parser, tune efSearch.
- For hybrid retrieval today on 10.0.0, use the rerank pattern or client-side RRF. Native RRF lands in Solr 10.1 (SOLR-17319, fix versions 10.1 and 9.11).

SolrCloud and Sharding

What this chapter covers and why it matters. Solr 10 is the version where SolrCloud finally stopped sharing the spotlight: standalone mode still exists behind a --user-managed flag, but cloud is the default and the future. This chapter covers the architecture, ZooKeeper's role, sharding/replication, replica types, leader election, document routing, the Collections API, scaling, and Kubernetes deployment with the Apache Solr Operator.

Prerequisites: Chapter 2 (collections, schemas, configsets) and Chapter 3 (you should understand a distributed query before reading the distributed query flow).

You will learn: Why SolrCloud needs ZooKeeper and what ZK actually stores; the differences between collections, shards, replicas, and cores; the three replica types (NRT, TLOG, PULL) and when to use each; how compositeId routing works (referenced in §3.9 Collapse); how leader election survives node failures; how to issue Collections API commands from SolrJ; how scatter-gather distributed queries actually work; and how to deploy a production cluster on Kubernetes with the Apache Solr Operator.

What you should not skip: §4.4 (replica types — the most consequential design decision in a SolrCloud cluster) and §4.12 (Kubernetes deployment, including the honest assessment of the Operator's Solr 10 support).

4.1 The Solr 10 cluster picture

In Solr 10, "bin/solr start now defaults to starting Solr in SolrCloud mode. Use the –user-managed switch to start in standalone (user-managed) mode instead" (Major Changes in Solr 10). The community has been moving in this direction for years; the deprecation discussion on dev@solr.apache.org dates back to 2021.

Solr 10 also upgraded its underlying engine: the Solr 10 News page states "Powered by Apache Lucene 10.3, with numerous small and large improvements." The cluster runtime is on Jetty 12 and Jakarta EE 10, the metrics subsystem migrated to OpenTelemetry (replacing Dropwizard), and several long-deprecated modules (jaegertracer-configurator, analytics, hadoop-auth) have been removed.

Solr 9 differences — the platform shifted under your feet. The big ones if you're upgrading from 9.x: - Java requirement. Solr 10 server needs Java 21. Solr 9.x ran on Java 11+. (SolrJ on the client side still only needs Java 17 in 10.x.) - bin/solr start default. In Solr 9, the default was standalone mode, and SolrCloud required -c. In Solr 10, the default is SolrCloud, and standalone requires --user-managed. Scripts written against 9 may need updating either way. - HTTP layer. Jetty 12 (was 10) and Jakarta EE 10 (was javax). If you write custom plugins that touched servlet APIs, expect package renames (javax.servlet.* → jakarta.servlet.*). - Metrics. Dropwizard → OpenTelemetry. Metric names changed from dot-delimited (SEARCH.requestTimes.p95) to snake_case (search_request_times_p95). Existing Prometheus dashboards will break. - bin/solr delete no longer deletes a configset when you delete a collection — you must now explicitly pass --delete-config. In 9, a configset used by only one collection was deleted automatically. - Removed modules. jaegertracer-configurator, analytics, hadoop-auth. If you used any of these, you need a replacement.

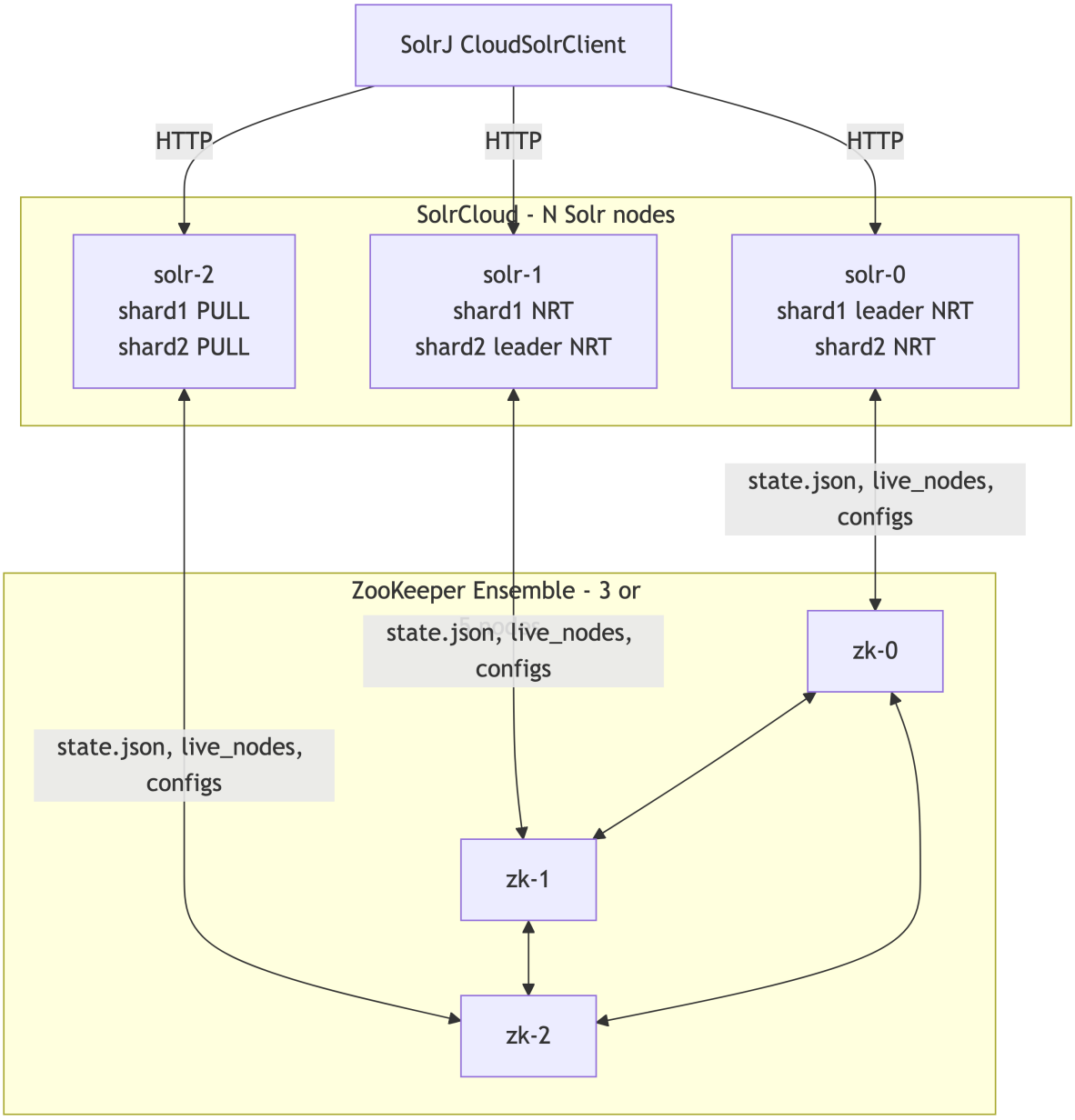

Figure 1: A SolrCloud cluster with three Solr nodes sharing cluster state via a ZooKeeper ensemble, with CloudSolrClient routing requests directly to any node.

4.2 What ZooKeeper stores

For each collection ZooKeeper holds:

- /live_nodes/ — ephemeral znode per running Solr node.
- /collections/<name>/state.json — shard map, replica list, leader pointer, replica states.

- /configs/<configset>/ — the configset (schema, solrconfig.xml, etc.) shared by collections.
- /overseer_elect/ — election znodes for the Overseer.
- /aliases.json, /clusterprops.json — cluster-wide metadata.

Solr is otherwise stateless: a node restart rebuilds its in-memory view from ZooKeeper.

> Production tip — run ZK on dedicated nodes with SSDs. ZK fsyncs every write. Co-locating ZK with Solr means GC pauses in Solr's JVM can starve ZK's election traffic and cause spurious leader churn.

Solr 10's optional Overseer-less mode

The Solr 10 release notes describe SOLR-17877: "The SolrCloud Overseer is disable-able, in lieu of a simpler distributed mode of cluster command and collection state processing." You enable it with the overseerEnabled cluster property or the SOLR_CLOUD_OVERSEER_ENABLED env var. "In Solr 11, the Overseer might cease to exist, in an effort to simplify SolrCloud and maintenance." For new deployments on Solr 10, stay with the Overseer for now — distributed mode is fine but the migration path is one-way.

4.3 Collections, shards, replicas

- Collection — a logical index. Has a schema, a number of shards, a configset, a router.
- Shard — a horizontal slice of the collection. A document belongs to exactly one shard.
- Replica — a physical copy of a shard, hosted on some node. One replica per shard is the leader; the others are followers.

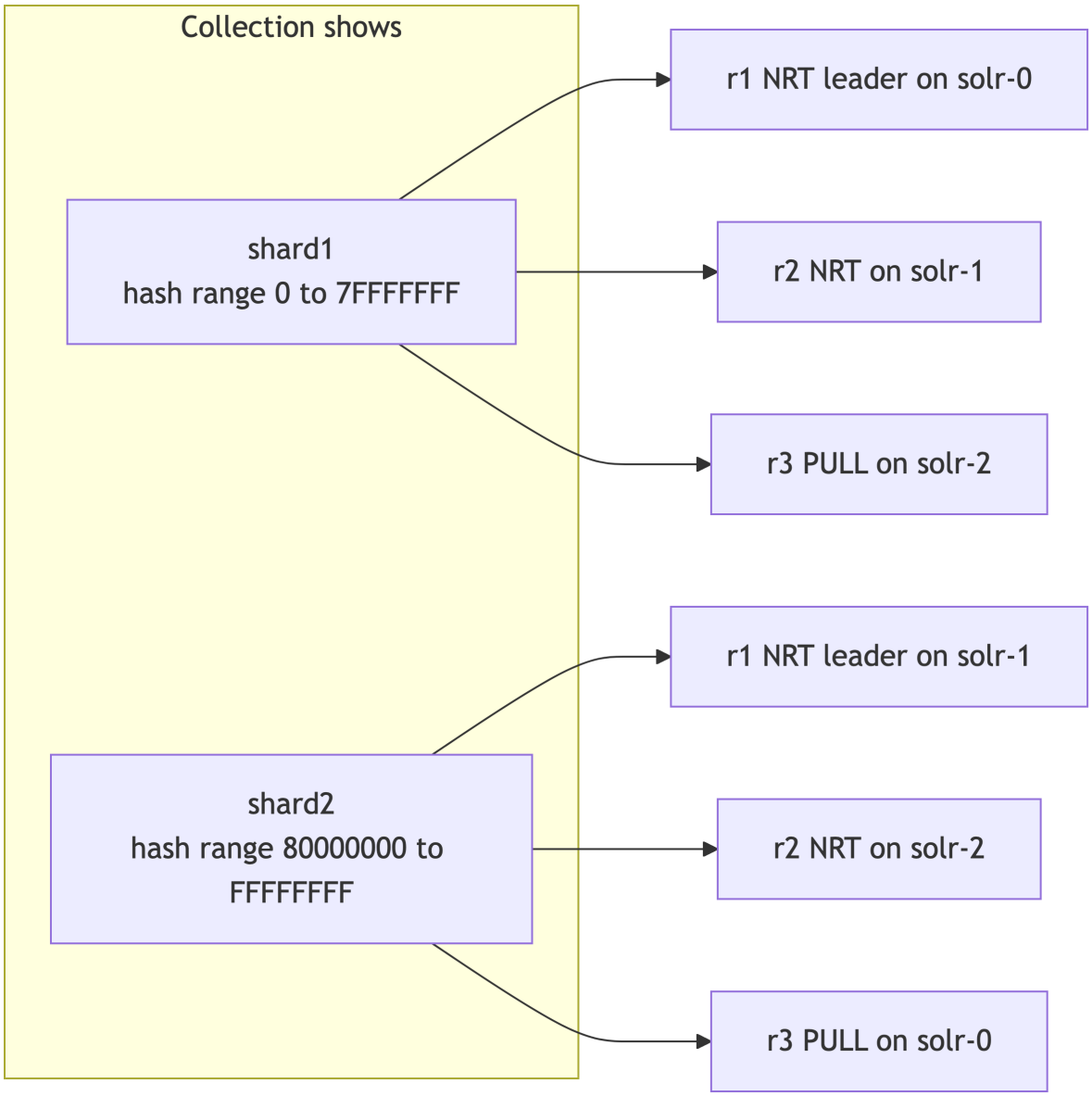

Figure 2: A two-shard collection with hash-range partitioning and a mix of NRT leaders, NRT followers, and PULL replicas spread across three Solr nodes.

4.4 Replica types: NRT, TLOG, PULL

The Solr reference guide defines them succinctly:

- NRT (Near Real Time) — "maintains a transaction log and updates its index locally. Replicas of type NRT support NRT (soft commits) and RTG. Any NRT replica can

become a leader." Indexes everything locally at write time; soft commits make new docs visible within seconds.

- TLOG — "maintains a transaction log but only updates its index via replication." The leader forwards docs to TLOG replicas, which append them to their own transaction log and pull segment files from the leader. They do not run the analyzer chain or build segments locally for incoming writes; that CPU cost is paid only by the leader. The tlog exists so a TLOG replica can become the leader by replaying its log up to the latest version (peer sync) if the current leader fails.
- PULL — "does not maintain a transaction log and only updates its index via replication. This type is not eligible to become a leader." Pure search replica. Cheapest replica type. Cannot become leader because it has no tlog to replay.

Table 1: Comparison of the three SolrCloud replica types — NRT, TLOG, PULL — across local indexing, transaction log, leader eligibility, and the workload each one is best for.

Replica type	Indexes from writes	Has tlog	Leader-eligible	Best for
NRT	yes (per-replica)	yes	yes	small clusters, near-real-time visibility
TLOG	no — pulls segments; tlog tracks writes for failover	yes	yes	high indexing volume, save CPU on followers
PULL	no — pulls segments only	no	no	scaling read throughput cheaply

The header "Indexes locally" in the old version of this table was easy to misread: TLOG replicas do write to local disk (their tlog and the pulled segments), they just don't run the analyzer pipeline on incoming writes. That CPU saving is the whole point of the type.

Common production layouts:

- All-NRT (3 replicas/shard). Simplest. Use when soft-commit visibility latency matters.
- TLOG leaders + PULL followers (1 TLOG + N PULL per shard). Best for write-heavy, read-very-heavy workloads where seconds-of-staleness on followers is OK.
- NRT + PULL. NRT for leader/indexing, PULL for additional query capacity. Caveat from the Solr user list: mixing NRT and PULL/TLOG can trigger full-index recoveries on PULL when an NRT leader changes, because NRT replicas of the same shard have differently-named segment files. Prefer pure TLOG+PULL or pure NRT layouts.

4.5 Document routing

Solr 10 supports two routers:

- compositeId (default) — murmur3(id) % shardCount decides the shard. Prefix routing (tenant1!doc-42) lets you co-locate a tenant on a known shard.
- implicit — you specify the shard at index time. You're on your own for routing logic. Useful when each shard represents a time bucket or a tenant.

  Production tip — pick a router up front and never change it. Switching routers requires a full reindex into a new collection.

4.6 Leader election and recovery

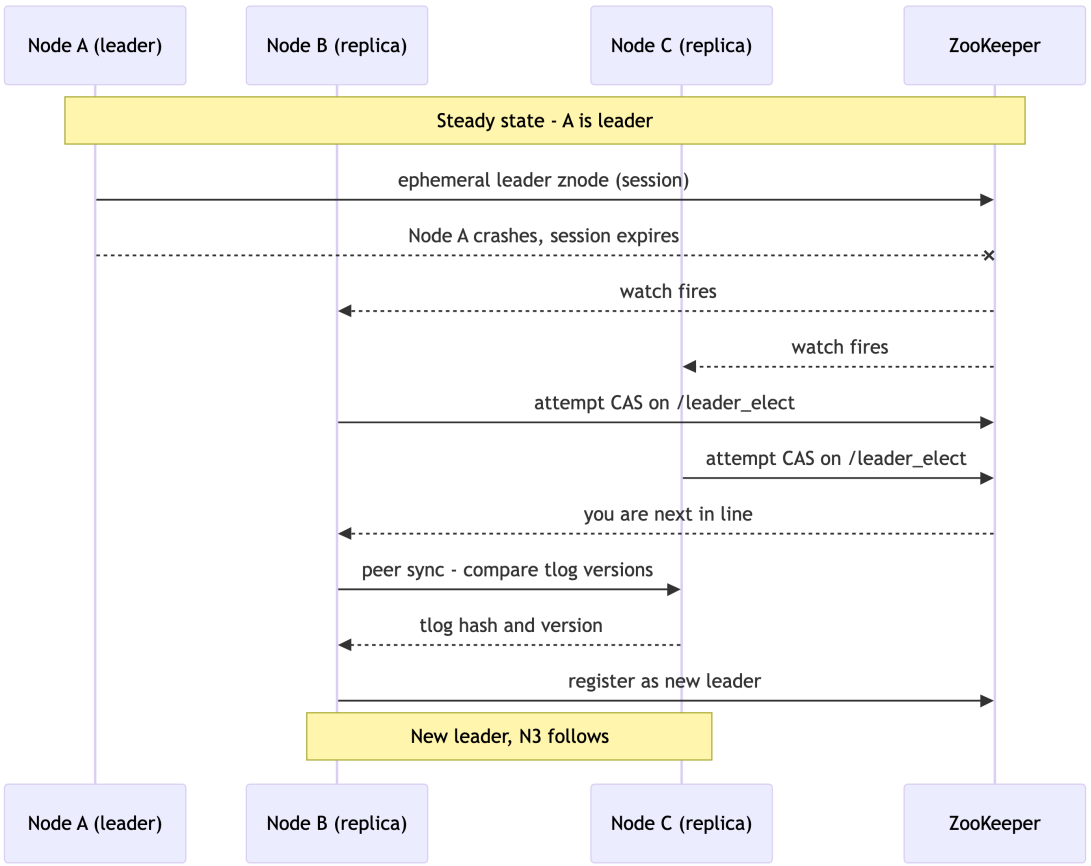

Figure 3: Leader election after a node crash: ZooKeeper watches fire, the lowest-sequenced follower wins the election znode race, peer-syncs with the survivor, and registers as the new leader.

Leader election is a ZooKeeper sequential ephemeral-node race. The candidate with the lowest sequence number becomes leader after running a peer sync (compare tlog versions with peers; if too far behind, do a full replication recovery from the new leader).

The replica recovery state machine:

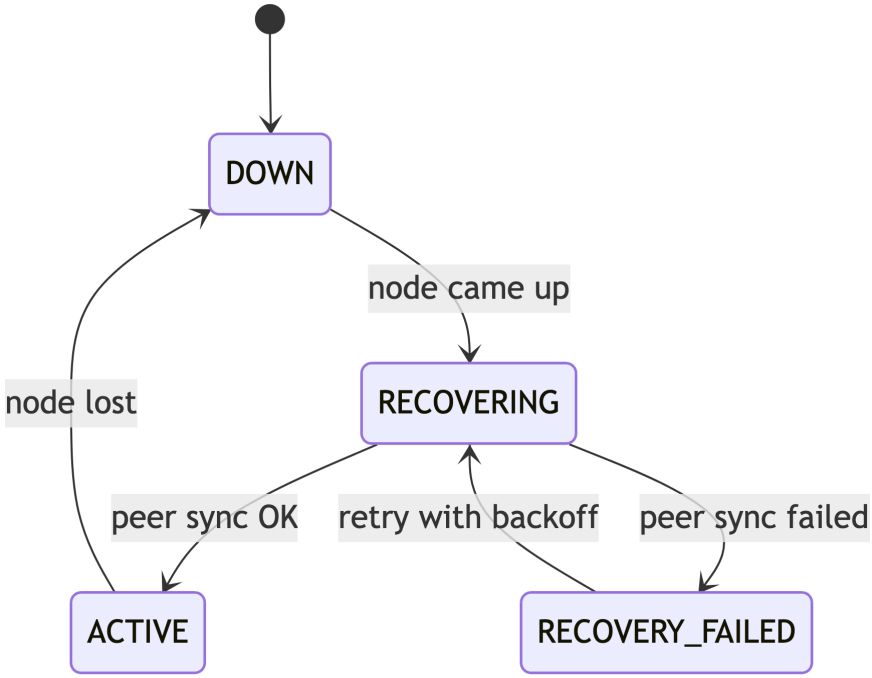

Figure 4: The replica recovery state machine: a node transitions from DOWN through RECOVERING to ACTIVE, falling back to RECOVERY_FAILED with retry on peer-sync failure.

4.7 The Collections API in SolrJ

```
import org.apache.solr.client.solrj.request.CollectionAdminRequest;
import org.apache.solr.client.solrj.response.CollectionAdminResponse;
import org.apache.solr.common.cloud.Replica;

// Create
CollectionAdminRequest.Create create =
    CollectionAdminRequest.createCollection("shows", "shows_configset", 4, 2);
create.process(client);

// Reload (after configset change)
```

```
CollectionAdminRequest.reloadCollection("shows").process(client);

// Split a shard
CollectionAdminRequest.splitShard("shows").setShardName("shard1").process(client);

// Add a replica
CollectionAdminRequest.addReplicaToShard("shows", "shard1")
    .setType(Replica.Type.PULL)
    .process(client);

// Delete replica
CollectionAdminRequest.deleteReplica("shows", "shard1", "core_node3").process(client);

// Delete collection
CollectionAdminRequest.deleteCollection("shows").process(client);
```

In Solr 10, "the bin/solr delete command no longer deletes a configset when you delete a collection. Previously if you deleted a collection, it would also delete its associated configset if it was the only user of it. Now you have to explicitly provide a –delete-config option." This decouples configset lifecycle from collection lifecycle — a welcome change for shared configsets.

4.8 ConfigSets

A configset is a directory of managed-schema.xml, solrconfig.xml, synonyms.txt, stopword files, etc. In SolrCloud it lives in ZooKeeper under /configs/<name>/. You upload one with bin/solr zk upconfig or the Configset API and reference it at collection-create time.

```
bin/solr zk upconfig -n shows_configset -d /path/to/local/configset -z zk-0:2181
```

> Production tip — version your configsets in git, deploy them with a CI pipeline that calls zk upconfig and then RELOAD COLLECTION. This gives you reviewable schema changes and atomic rollouts.

4.9 Distributed query flow

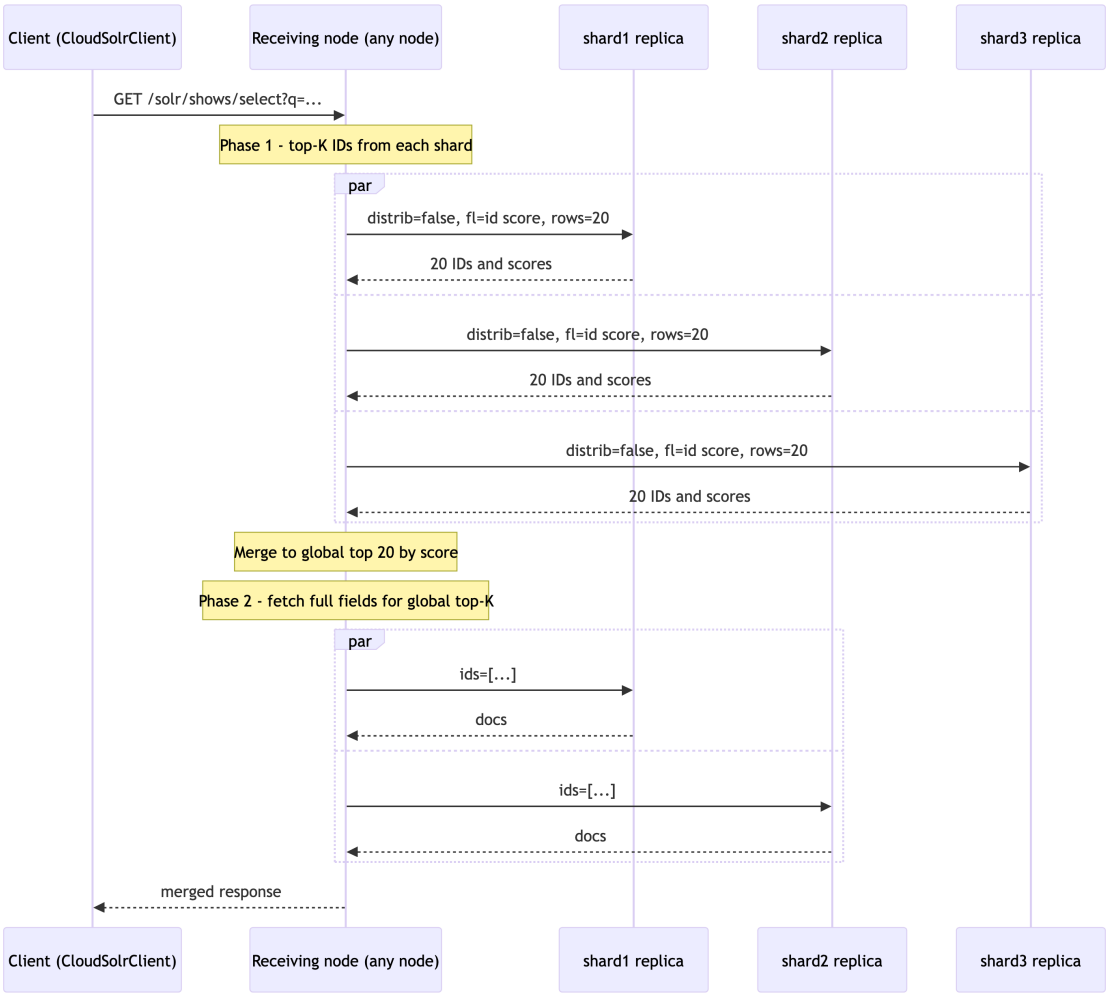

Figure 5: The two-phase distributed query: every shard returns top-K IDs in parallel, the aggregator merges to a global top-K, then refetches full documents only for the survivors.

This two-phase distributed search is a Lucene/Solr classic: phase 1 gets IDs+scores from every shard, phase 2 pulls the full documents only for the global top-K. Faceting is similarly two-phased with refinement to fix counts at shard boundaries.

> Production tip — the receiving node is the aggregator. Use CloudSolrClient so the client picks a node intelligently (it can prefer leaders for indexing or follow shards.preference for queries).

4.10 Scaling: when to add shards vs. replicas

Table 2: Decision matrix for whether a given scaling symptom calls for adding shards, adding replicas, or both.

Symptom	Add shards	Add replicas
Index too big for one node's disk / RAM	yes	no
Indexing throughput is the bottleneck	yes	depends
Query QPS is the bottleneck	no	yes
Query latency is high at low QPS	yes (parallelism)	no
Need higher availability	no	yes

Sharding has a cost: every distributed query fans out to every shard. Below ~50–100M docs per shard, additional shards usually hurt latency more than they help. The sweet spot in 2026 is typically 50M–200M docs per shard.

4.11 Replacement for the (removed) autoscaling framework

In Solr 9.0 the entire autoscaling framework was removed (SOLR-14656): "Autoscaling framework removed. This includes: Autoscaling, policy, triggers etc." It was widely regarded as too complex and unreliable.

What replaces it in Solr 9+ and continues into Solr 10:

1. Replica Placement Plugins. A pluggable API for where to put a replica on collection create / replica add. Stock implementations:
   - AffinityPlacementFactory — respects availability zones, replica types, disk usage. The default for production. As of Solr 9.1, default minimalFreeDiskGB is 20.
   - MinimizeCoresPlacementFactory — pure load balancing.
   - RandomPlacementFactory — testing.
2. Node Roles. A node can have role overseer, data, or both — letting you dedicate nodes to coordination vs. indexing/serving.
3. External orchestration. True auto-scaling (adding/removing nodes in response to load) is now an operational concern, not a Solr-internal one. On Kubernetes the Solr Operator (§4.12) plus HPA handles this. On VMs you script it.

Configure AffinityPlacementFactory once per cluster:

```
curl -X POST -H 'Content-Type: application/json' \
   'http://solr-0:8983/api/cluster/plugin' \
   --data '{
```

```
    "add": {
      "name": ".placement-plugin",
      "class": "org.apache.solr.cluster.placement.plugins.AffinityPlacementFactory",
      "config": {
        "minimalFreeDiskGB": 20,
        "prioritizedFreeDiskGB": 100
      }
    }
  }'
```

Then set the availability_zone and replica_type system properties on each Solr node (-Davailability_zone=us-east-1a -Dreplica_type=tlog) and the placement plugin honors them.

4.12 Kubernetes with the Apache Solr Operator

The Apache Solr Operator is the canonical way to run SolrCloud on Kubernetes. Per the Apache Solr homepage (solr.apache.org), the latest GA is "Apache Solr Operator™ v0.9.1 available (25.Mar)" — released March 25, 2025.

Solr 10 compatibility caveat. As of May 2026, no v0.10.x release with explicit Solr 10 support has been published. A v0.10.0-prerelease exists on the main branch (referenced in upgrade docs), and GitHub issue #821 (March 16, 2026) explicitly asks for clarification on whether v0.9.1 supports Solr 10. The official compatibility matrix still lists "Solr 9.4+". Running Solr 10 images under operator v0.9.1 is unsupported/experimental. For production: either pin Solr 9.10.x until operator v0.10.0 GA lands, or accept the experimental status and test thoroughly.

Install (current released v0.9.1)

```
helm repo add apache-solr https://solr.apache.org/charts
helm repo update
kubectl create -f
 ↪  https://solr.apache.org/operator/downloads/crds/v0.9.1/all-with-dependencies.yaml
helm install solr-operator apache-solr/solr-operator --version 0.9.1
```

Production SolrCloud CR

The CRD is still apiVersion: solr.apache.org/v1beta1 — beta-versioned, per the operator README: “The API Version is still beta (v1beta1), and minor versions can have backwards-incompatible API changes. However, the Solr Operator will always have upgrade paths that are backwards-compatible.”

```
apiVersion: solr.apache.org/v1beta1
kind: SolrCloud
metadata:
  name: prod-solr
  namespace: solr
spec:
  replicas: 3
  solrImage:
    repository: solr
    tag: "10.0.0"          # experimental under v0.9.1 - see caveat above
    pullPolicy: IfNotPresent

  # JVM
  solrJavaMem: "-Xms4g -Xmx4g"
  solrOpts:   "-XX:+UseG1GC -XX:MaxGCPauseMillis=200"

  # External (provided) ZK ensemble - recommended for production
  zookeeperRef:
    connectionInfo:
      externalConnectionString:
↪   "zk-0.zk-hs.zk.svc:2181,zk-1.zk-hs.zk.svc:2181,zk-2.zk-hs.zk.svc:2181"
      chroot: "/prod-solr"

  # Persistent storage per pod
  dataStorage:
    persistent:
      reclaimPolicy: Retain
      pvcTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          storageClassName: "ssd"
          resources:
            requests:
              storage: "100Gi"
```

```
customSolrKubeOptions:
  podOptions:
    resources:
      requests: { cpu: "2", memory: "6Gi" }
      limits:   { cpu: "4", memory: "8Gi" }
    affinity:
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
                - key: solr-cloud
                  operator: In
                  values: ["prod-solr"]
            topologyKey: kubernetes.io/hostname
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                  - key: solr-cloud
                    operator: In
                    values: ["prod-solr"]
              topologyKey: topology.kubernetes.io/zone

solrAddressability:
  podPort: 8983
  commonServicePort: 80
```

The Apache Solr Operator GKE walkthrough recommends, for production deployments, defining the ZookeeperCluster outside the SolrCloud CR and pointing to it via spec.zookeeperRef.connectionInfo. The pod anti-affinity above spreads pods across nodes (required) and prefers spreading them across zones (preferred). Pods carry the solr-cloud=<name> label automatically.

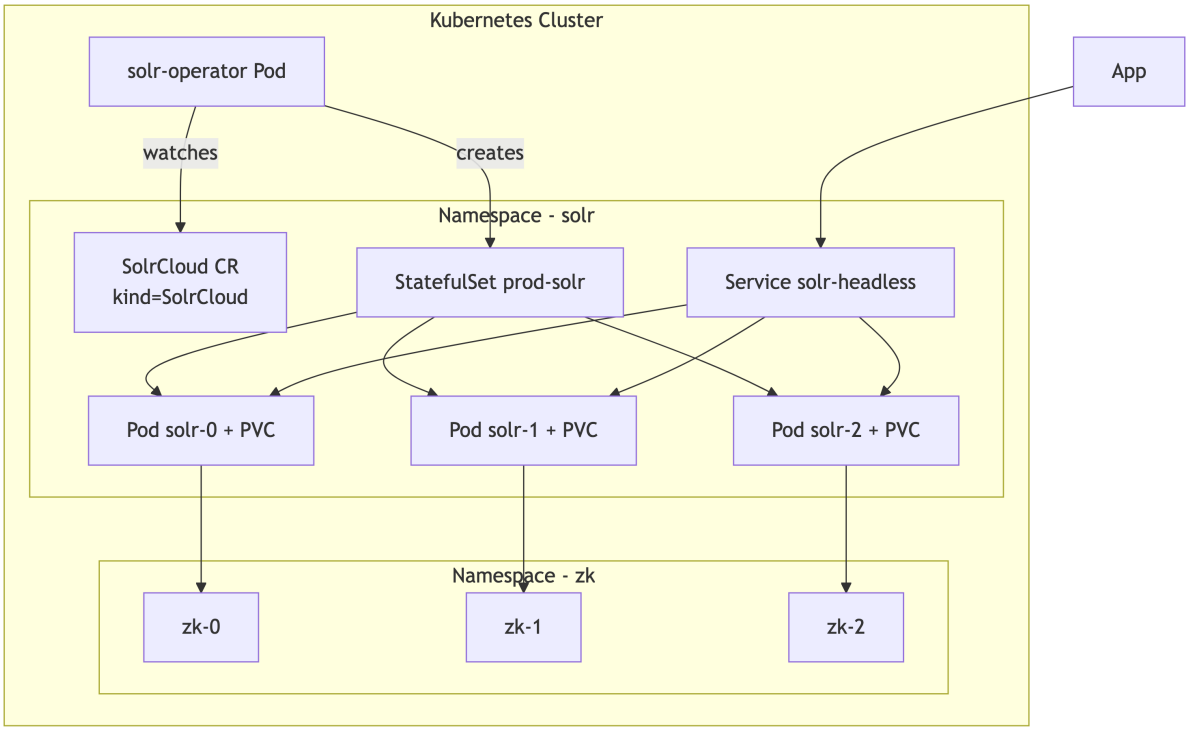

Figure 6: How the Apache Solr Operator turns a SolrCloud custom resource into a StatefulSet of Solr pods with PVCs, a headless service, and an external ZooKeeper ensemble.

The operator handles: StatefulSet creation, headless service, PVC templates, rolling restarts on config change, TLS cert mounting, optional Prometheus exporter sidecar, and (in newer versions) managed scale up/scale down with replica balancing on Solr 9.3+.

4.13 Monitoring

Solr 10 "migrated from Dropwizard metrics to OpenTelemetry (OTEL) for observability. This migration provides native Prometheus support, OTLP support, exemplar support for tracing correlation, and native attributes and labels on all metrics. All metrics have been migrated to snake-case metric names instead of dot-delimited format" (Major Changes in Solr 10).

The /admin/metrics API now defaults to Prometheus exposition format. Pipe it into Prometheus + Grafana, or push OTLP to your tracing backend. Standard dashboards to build:

- QPS per collection, p50/p95/p99 latency
- Indexing rate and tlog size
- Cache hit ratios (queryResultCache, filterCache, documentCache)
- GC pauses and heap usage

- Replica state (active/recovering/down)
- Circuit breaker trips (CPU load average, memory)

Exercises

1. Size a cluster. A streaming-content catalog has 80M documents (~3 KB average), receives 200 indexing writes/sec at peak, and serves 5,000 queries/sec at peak with p95 latency target 200 ms. Specify: number of shards, replica types and counts per shard, node count, JVM heap per node, and disk per node. Justify each number using §4.4 and §4.10.
2. Choose replicas for a workload. Three workloads: (a) a logging cluster ingesting 50K events/sec with hourly query traffic; (b) a product search bar serving 2,000 QPS with hourly bulk reindexes; (c) an internal admin search used by 50 employees. For each, choose between all-NRT, TLOG+PULL, or NRT+PULL, and explain the tradeoff you're making.
3. Trace a failover. Walk through the leader-election sequence (§4.6) when the current leader of shard1 of a 3-replica collection (1 NRT leader, 2 NRT followers) crashes. At each step, identify which ZooKeeper znode changes, which Solr node takes which action, and how long the collection is unavailable for writes.
4. Build the Kubernetes spec. Adapt the SolrCloud CR in §4.12 for a 5-node cluster spread across 3 availability zones with 200 GB SSD per pod and a separate ZK ensemble. Add an Ingress for the Admin UI. Validate that your anti-affinity rules prevent two replicas of the same shard from landing on the same node.

4.14 Chapter summary

- Solr 10 makes SolrCloud the default; standalone survives as --user-managed. Underlying engine: Lucene 10.3, Jetty 12, Java 21 server.
- ZooKeeper holds cluster state and configsets; keep it on dedicated nodes. Overseer-less mode is opt-in in Solr 10 (likely default in 11).
- Three replica types — NRT, TLOG, PULL — let you trade indexing latency vs. CPU and high-availability.
- The Collections API in SolrJ is your control plane for creating, splitting, reloading collections.
- The removed autoscaling framework is replaced by Replica Placement Plugins + external orchestration (Kubernetes Operator + HPA).
- Production Kubernetes deployment uses the Apache Solr Operator (current GA: v0.9.1; v0.10.x with first-class Solr 10 support is in development) with an externally-managed ZooKeeper ensemble, persistent storage, pod anti-affinity, and OpenTelemetry/Prometheus monitoring.

Relevance Engineering with Solr

What this chapter covers and why it matters. Chapter 3 taught you how Solr finds documents (BM25, eDisMax, vectors, JSON Facets). This chapter is about how you prove those documents are the right ones — and how you keep proving it as your corpus, your users, and your business change. In production, "relevance" is not an opinion; it is a measured loop: judge → retrieve → log → boost or rerank → A/B test → deploy. Every section below slots into that loop.

Prerequisites: Chapter 2 (schemas, atomic and in-place updates, aliases) and Chapter 3 (eDisMax, scoring, function queries). §5.2 reuses the in-place update mechanic from §2.8.2; §5.3 builds on the eDisMax boost parameter from §3.2.

You will learn: How to measure relevance with a judgment list, NDCG@10, and Quepid; how to log and aggregate user signals into a sibling collection; how to apply signals as boosts for head queries; how to use Solr's built-in ltr module for learning-to-rank reranking; how to use the relatedness() aggregation for query expansion and content discovery; and how to layer editorial overrides on top of all of the above.

What you should not skip: §5.1 (measurement — without numbers, the rest of the chapter is theatre) and §5.3 (signals boosting — the highest-ROI relevance technique most teams haven't shipped).

The first four sections compress the practical core of AI-Powered Search (Grainger, Turnbull, Irwin — Manning, 2025) into the minimum a Solr engineer needs to be dangerous. §5.5 covers Solr's built-in relatedness() aggregation (SOLR-9480) for ad-hoc relationship mining. §5.6 covers editorial overrides — the manual sibling of automated boosting.

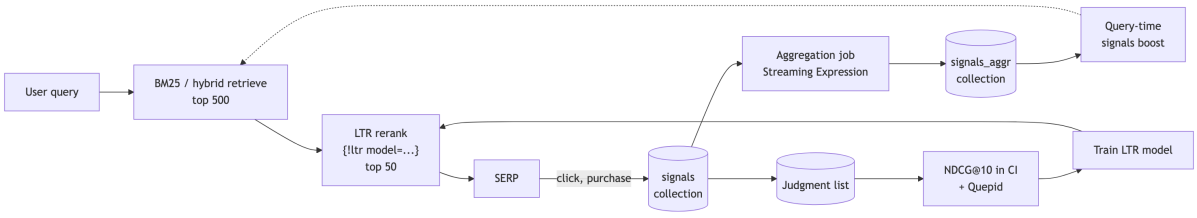

Figure 1: The end-to-end relevance loop: retrieve, rerank, log signals, aggregate, boost, judge, and feed model training back into the LTR reranker.

5.1 Measuring relevance: judgment lists, NDCG, and Quepid

A judgment list is a CSV of (query, doc_id, grade) rows — typically grades 0..3 — that defines "good" for your domain. With judgments you can compute offline metrics: NDCG@k (graded relevance, position-discounted, normalized to 1.0; the industry default), MRR (reciprocal rank of the first relevant result; use when there is one right answer), and MAP (mean of average precisions across queries; binary relevance, rewards finding all relevant docs early). Quepid is the open-source workbench from OpenSource Connections (Apache 2.0, github.com/o19s/quepid) that holds your judgment book, runs queries against any Solr/OpenSearch/Elasticsearch endpoint, and shows the metric delta as you tune.

Without numbers, every relevance argument becomes a HiPPO argument — Highest-Paid Person's Opinion. A 150-query judgment list with grades from two domain raters is enough to detect a 2–3 point NDCG@10 regression before it hits production. This is the only honest gate for shipping a schema change, a new analyzer, or an LTR model.

Implementation: NDCG@k in SolrJ

Score a judgment list against a live collection from Java. The implementation is intentionally explicit so you can drop it into a CI job.

```
public final class NdcgEvaluator {

    record Judgment(String query, String docId, int grade) {}

    public static void main(String[] args) throws Exception {
        List<String> solrUrls = List.of("http://solr-0:8983/solr","http://solr-1:8983/solr");
        try (var solr = new CloudSolrClient.Builder(solrUrls)
                .withDefaultCollection("shows").build()) {

            Map<String, List<Judgment>> byQuery = loadJudgments("judgments.csv")
                .stream().collect(Collectors.groupingBy(Judgment::query));

            double sum = 0;
            for (var e : byQuery.entrySet()) {
                var q = new SolrQuery(e.getKey()).setRows(10).setFields("id");
                List<String> ranked = solr.query(q).getResults().stream()
                    .map(d -> (String) d.getFieldValue("id")).toList();
                sum += ndcg(ranked, e.getValue(), 10);
            }
            System.out.printf("NDCG@10 = %.4f over %d queries%n",
```

```
                sum / byQuery.size(), byQuery.size());
        }
    }

    static double ndcg(List<String> ranked, List<Judgment> judged, int k) {
        Map<String,Integer> grade = judged.stream()
            .collect(Collectors.toMap(Judgment::docId, Judgment::grade, (a,b)->a));
        double dcg = 0;
        for (int i = 0; i < Math.min(k, ranked.size()); i++) {
            int g = grade.getOrDefault(ranked.get(i), 0);
            dcg += (Math.pow(2, g) - 1) / (Math.log(i + 2) / Math.log(2));
        }
        List<Integer> ideal = judged.stream().map(Judgment::grade)
            .sorted(Comparator.reverseOrder()).limit(k).toList();
        double idcg = 0;
        for (int i = 0; i < ideal.size(); i++) {
            idcg += (Math.pow(2, ideal.get(i)) - 1) / (Math.log(i + 2) / Math.log(2));
        }
        return idcg == 0 ? 0 : dcg / idcg;
    }
}
```

Production tip — wire this into CI. Run the evaluator in Jenkins/GitHub Actions on every PR that touches schema, analyzers, or solrconfig. Fail the build on a >1% NDCG@10 regression on a frozen "regression suite" of 100–300 queries; allow larger swings on a separate "exploration suite." Two suites, two thresholds — otherwise every legitimate relevance experiment looks like a regression.

Production tip — point Quepid at staging. Quepid issues queries on every keystroke as raters work. Running it against prod doubles your search traffic and skews your own signals (§5.2). Always run Quepid against a staging Solr endpoint with q.op and default parameters matching production.

AIPS reference. Chapter 4 ("Crowdsourced relevance") for the philosophy of judgments; Chapter 10 §10.2 for how the same list feeds Learning to Rank; Chapter 11 §11.1 for implicit judgments derived from clicks.

5.2 The signals plane: collection schema and aggregation

A signal is any logged user event tied to a document — query, click, add_to_cart, purchase, dwell_ms. By convention (Lucidworks Fusion, AI-Powered Search Chapter 8, and most production Solr deployments) signals live in a sibling collection named <primary>_signals, and aggregated signals — one row per (query, doc, action) with a count and a weight — live in <primary>_signals_aggr. Aggregation is a periodic job, not a real-time process: typically a Streaming Expression that runs every 15 minutes to an hour.

Raw clicks are too sparse and too noisy to query at search time. The aggregation collection is the materialized view that the signals boosting (§5.3) and LTR feature logging (§5.4) stages both depend on. Without §5.2, §5.3 and §5.4 cannot happen.

Schema for the _signals collection

```
<field name="id"          type="string"  indexed="true"  stored="true" required="true"/>
<field name="type"        type="string"  indexed="true"  stored="true"/>
                                <!-- "click" | "query" | "purchase" | "add_to_cart" -->
<field name="query_s"     type="string"  indexed="true"  stored="true"
 ↪  docValues="true"/>
<field name="doc_id"      type="string"  indexed="true"  stored="true"
 ↪  docValues="true"/>
<field name="user_id"     type="string"  indexed="true"  stored="true"
 ↪  docValues="true"/>
<field name="position_i"  type="pint"    indexed="true"  stored="true"
 ↪  docValues="true"/>
<field name="timestamp"   type="pdate"   indexed="true"  stored="true"
 ↪  docValues="true"/>
<field name="filters_ss"  type="strings" indexed="true"  stored="true"
 ↪  docValues="true"/>
```

Logging from your search API in SolrJ

```
public void logClick(SolrClient solr, String query, String docId,
                     String userId, int position, List<String> filters)
        throws Exception {
    SolrInputDocument s = new SolrInputDocument();
    s.addField("id",          UUID.randomUUID().toString());
```

```
    s.addField("type",        "click");
    s.addField("query_s",     query.toLowerCase(Locale.ROOT).trim());
    s.addField("doc_id",      docId);
    s.addField("user_id",     userId);
    s.addField("position_i", position);
    s.addField("timestamp",   Instant.now().toString());
    s.addField("filters_ss", filters);
    solr.add("shows_signals", s);          // autoCommit handles durability
}
```

Aggregation via Streaming Expression — two passes

Run hourly. There are two passes: the rollup produces raw counts, and a second pass enriches each document with a weight_d that downstream consumers actually use.

The _signals_aggr schema:

```
<field name="id"         type="string" indexed="true"  stored="true"  required="true"/>
<field name="query_s"    type="string" indexed="true"  stored="true"  docValues="true"/>
<field name="doc_id"     type="string" indexed="true"  stored="true"  docValues="true"/>
<field name="count_i"    type="pint"   indexed="true"  stored="true"  docValues="true"/>
<field name="weight_d"   type="pdouble" indexed="false" stored="false"
 ↪   docValues="true"/>
<!-- weight_d is indexed=false stored=false docValues=true so the in-place update
     path (§2.8.2) can rewrite it without reindexing the whole document. -->
```

Pass 1 — roll up raw counts. A select Streaming Expression with a rollup produces one row per (query, doc) pair. The id is computed deterministically so subsequent runs upsert instead of duplicating:

```
update(shows_signals_aggr, batchSize="500",
  select(
    rollup(
      search(shows_signals,
             q="type:click AND timestamp:[NOW-7DAYS TO NOW]",
             fl="query_s,doc_id",
             sort="query_s asc, doc_id asc",
             qt="/export"),
      over="query_s,doc_id",
```

```
    count(*)),
  query_s, doc_id,
  count(*) as count_i,
  concat(fields="query_s,doc_id", delim="|") as id))
```

Pass 2 — enrich with weight_d as an in-place update. Apply your dampening function. The cleanest approach is to compute the weight in your application code and issue per-row in-place updates against _signals_aggr. Because weight_d is indexed=false stored=false docValues=true, this is a true in-place update (§2.8.2) — no reindex, no analyzer chain, microseconds per row:

```
// Pass 2 in Java, run after pass 1 commits.
QueryResponse rows = solr.query("shows_signals_aggr",
    new SolrQuery("*:*").setRows(50_000)
        .setFields("id,query_s,count_i")
        .addSort("query_s", SolrQuery.ORDER.asc)
        .addSort("count_i", SolrQuery.ORDER.desc));

// Group by query_s to compute per-query max for normalization.
Map<String, Long> maxByQuery = new HashMap<>();
for (SolrDocument d : rows.getResults()) {
    maxByQuery.merge((String) d.get("query_s"),
                     ((Number) d.get("count_i")).longValue(), Math::max);
}

List<SolrInputDocument> batch = new ArrayList<>(1_000);
for (SolrDocument d : rows.getResults()) {
    long count = ((Number) d.get("count_i")).longValue();
    long max   = maxByQuery.get((String) d.get("query_s"));
    double w   = Math.log1p(count) / Math.log1p(max);

    SolrInputDocument upd = new SolrInputDocument();
    upd.addField("id", d.get("id"));
    upd.addField("weight_d", Map.of("set", w));   // in-place update
    batch.add(upd);
    if (batch.size() == 1_000) { solr.add("shows_signals_aggr", batch); batch.clear(); }
}
if (!batch.isEmpty()) solr.add("shows_signals_aggr", batch);
solr.commit("shows_signals_aggr");
```

After both passes, each _signals_aggr document has the shape {id, query_s, doc_id, count_i, weight_d}, and the §5.3 boosting code can read weight_d directly.

> Production tip — normalize on write. Always lowercase + trim query_s on the way in. Half of the "head query" tail you'll see in week one is just casing variants of "stranger things" vs. "Stranger Things" vs. " STRANGER THINGS ". Also store position_i — without it, you can't apply any position-bias correction later.
>
> Production tip — tune commit policy for write-heavy, read-rare. On _signals, set autoCommit with openSearcher=false plus a separate autoSoftCommit of ~60s. Hard-commit every 15 minutes. You're write-heavy and read-rare on this collection; don't pay the NRT visibility cost on every click.

AIPS reference. Chapter 4 ("Crowdsourced relevance") for the signals taxonomy; Chapter 8 §8.1 ("Basic signals boosting") for the aggregation contract.

5.3 Signals boosting: head queries and the index-time vs. query-time tradeoff

Take the aggregated weights from §5.2 and use them to push popular documents up for the specific query that produced the popularity. AIPS frames this as the lowest-cost, highest-ROI lever in the entire relevance toolbox, because for head queries — the ~1% of unique queries that drive ~50% of traffic — you already have enough signal to dominate BM25.

A correctly-tuned signals boost typically delivers a 5–15% NDCG@10 lift on head queries with zero ML infrastructure. It is the single best thing you can ship before LTR.

Query-time boost via the API layer

The most flexible pattern: compute boosts in your search API with one fast lookup against _signals_aggr, then pass them as the boost parameter (eDisMax multiplicative boost).

```
QueryResponse aggr = solr.query("shows_signals_aggr",
    new SolrQuery("query_s:\"" + ClientUtils.escapeQueryChars(userQuery.toLowerCase())
     ↪  + "\"")
       .setFields("doc_id,weight_d")
       .setRows(50));

String boostExpr = aggr.getResults().stream()
    .map(d -> String.format("if(termfreq(id,'%s'),%.3f,0)",
          d.get("doc_id"),
```

```
            ((Number) d.get("weight_d")).doubleValue()))
    .collect(Collectors.joining(",", "sum(", ")"));

SolrQuery q = new SolrQuery(userQuery)
    .setRequestHandler("/select");
q.set("defType", "edismax");
q.set("qf", "title^3 cast^2 description^1");
q.set("boost", boostExpr);                    // multiplicative
QueryResponse rsp = solr.query("shows", q);
```

This pattern scales because: (a) the _signals_aggr lookup is cheap and cacheable; (b) boost is evaluated only against the documents BM25 already matched; (c) you can A/B test new boost expressions without redeploying Solr.

Query-time boost via {!join} (smaller collections only)

For small _signals_aggr collections, you can skip the API-layer lookup and join directly:

```
q.set("bq",
  "{!join from=doc_id to=id fromIndex=shows_signals_aggr score=max}" +
  "query_s:\"" + ClientUtils.escapeQueryChars(userQuery.toLowerCase()) + "\"" +
  "^{!func}weight_d");
```

This works but doesn't scale past a few million signal docs. Stick with the API-layer pattern for production.

Index-time alternative

Materialize per-(query, doc) boost into the main shows collection as a multi-valued payload field, e.g. signals_boost:"stranger things|2.3 stranger|1.1". Use Solr's DelimitedPayloadTokenFilterFactory and the payload() function in a bf. This eliminates the per-request lookup but forces a reindex whenever weights change — fine for nightly batches, painful for hourly.

The operational rule of thumb: if your aggregation cadence is faster than your reindex cadence, stay query-time. Move only the long-stable head (queries that haven't changed their top-10 in two weeks) to index-time.

> Production tip — cap the magnitude. Cap any single signal boost at roughly 2× the BM25 max-score for the query. Otherwise one viral show owns the SERP and

> you create a feedback loop that buries new releases. AIPS calls this "the Matthew effect"; engineers call it "the popularity death spiral."

AIPS reference. Chapter 8 in full — especially §8.1 ("Basic signals boosting"), §8.3 ("Fighting signal spam"), and §8.6 ("Index-time vs. query-time boosting: balancing scale vs. flexibility").

5.4 Learning to Rank with Solr's ltr module

The ltr module ships with Solr; enable with -Dsolr.modules=ltr -Dsolr.ltr.enabled=true. It runs a machine-learned reranker against the top N documents from your BM25/hybrid retrieve. There are three moving parts:

- A feature store (managed JSON describing features — Solr queries, field values, external inputs).
- A model store (managed JSON containing the trained model — typically LinearModel or MultipleAdditiveTreesModel).
- The {!ltr} rerank parser, invoked via the rq= query parameter.

BM25 is one signal. LTR lets you combine 30+ signals — text similarity, recency, IMDb rating, popularity from §5.2, "watched-something-similar" personalization vectors — into a single learned scoring function. The rerank pattern means you pay the LTR cost on only the top ~100–500 docs per query (typically 5–20 ms), not on the full corpus.

Enabling LTR in solrconfig.xml

```
<queryParser name="ltr" class="org.apache.solr.ltr.search.LTRQParserPlugin"/>

<transformer name="features"

          ↪  class="org.apache.solr.ltr.response.transform.LTRFeatureLoggerTransformerFactory"/>

<cache name="QUERY_DOC_FV"
      class="solr.search.CaffeineCache"
      size="4096"
      initialSize="1024"
      autowarmCount="0"
      regenerator="solr.NoOpRegenerator"/>
```

Defining a feature store

POST to /schema/feature-store:

```
[
  { "name": "originalScore",
    "class": "org.apache.solr.ltr.feature.OriginalScoreFeature",
    "store": "v1" },
  { "name": "titleMatch",
    "class": "org.apache.solr.ltr.feature.SolrFeature",
    "params": { "q": "{!field f=title}${user_query}" },
    "store": "v1" },
  { "name": "popularity",
    "class": "org.apache.solr.ltr.feature.FieldValueFeature",
    "params": { "field": "popularity" },
    "store": "v1" },
  { "name": "rating",
    "class": "org.apache.solr.ltr.feature.FieldValueFeature",
    "params": { "field": "rating" },
    "store": "v1" },
  { "name": "isOngoing",
    "class": "org.apache.solr.ltr.feature.SolrFeature",
    "params": { "fq": ["status:ongoing"] },
    "store": "v1" }
]
```

Uploading a model

POST to /schema/model-store. Start with LinearModel — it's debuggable. Graduate to MultipleAdditiveTreesModel (LambdaMART-style) once you have 10k labeled examples:

```
{ "store": "v1",
  "name": "shows_v1",
  "class": "org.apache.solr.ltr.model.LinearModel",
  "features": [
    { "name": "originalScore" },
    { "name": "titleMatch" },
    { "name": "popularity" },
    { "name": "rating" },
    { "name": "isOngoing" }
```

```
 ],
 "params": {
  "weights": {
    "originalScore": 0.5,
    "titleMatch":    1.2,
    "popularity":    0.3,
    "rating":        0.4,
    "isOngoing":     1.0
  }
 } }
```

Reranking from SolrJ

```
SolrQuery q = new SolrQuery(userQuery)
    .setRows(20);
q.set("rq", "{!ltr model=shows_v1 reRankDocs=200 efi.user_query=\""
           + userQuery + "\"}");
q.setFields("id,title,score,[features store=v1 efi.user_query=\"" + userQuery + "\"]");
QueryResponse rsp = solr.query("shows", q);
```

Feature-logging-only mode

For extracting training data, log features without actually reranking by using a no-op model or reRankDocs=1:

```
q=stranger&rq={!ltr model=shows_v1 reRankDocs=1}&fl=id,score,[features
 ↪   efi.user_query=stranger]
```

Combine the resulting feature vectors with the implicit judgments from your signals collection to assemble an LTR training set.

> Production tip — reRankDocs is per-shard. With 4 shards and reRankDocs=200, you actually rerank 800 docs. Watch the latency curve carefully; past ~500 effective docs reranked with a 50-tree model, p99 latency hockey-sticks.
>
> Production tip — reload after upload. The feature store and model store are managed resources. Per the Solr 10 reference guide, changes are not applied to active components until the collection is reloaded. Script the reload step into your model-deploy pipeline.

> Production tip — always include originalScore. It anchors the model to BM25 and prevents catastrophic degradation when features are missing (e.g., a brand-new doc with no popularity signal yet).

AIPS reference. Chapter 10 (“Learning to rank for generalizable search relevance”) — §10.3 (“Feature logging and engineering”), §10.5 (“Training and testing the model”), §10.6 (“Upload a model and search”). Solr reference: query-guide/learning-to-rank.html.

5.5 The relatedness() aggregation — Solr’s built-in Semantic Knowledge Graph

Per the Solr 7.4 release notes: “SOLR-9480: A new ‘relatedness()’ aggregate function for JSON Faceting to enable building Semantic Knowledge Graphs.” The contribution from Trey Grainger and Chris M. Hostetter added a JSON Facet aggregation that scores how strongly any term — or facet bucket — is associated with a foreground document set versus a background set. Effectively, every inverted-index term becomes a node, and edges materialize on demand from corpus statistics. A knowledge graph for free, with no extra index.

This is the only crowdsourced-relevance technique that lives inside Solr itself, with zero external infrastructure. Use it for query expansion (“what genres co-occur with stranger things but not with everything?”), “more like this” widgets, “if you liked X, you might like Y” recommendations, and faceted content discovery beyond simple counts.

Implementation

For our shows catalog: “what genres are most characteristic of shows that share a creator with Stranger Things (i.e., the Duffer Brothers)?” The foreground set is shows created by the Duffer Brothers; the background is everything.

```
POST /solr/shows/query
{
  "query": "*:*",
  "limit": 0,
  "facet": {
    "related_genres": {
      "type":  "terms",
      "field": "genres",
      "limit": 10,
      "sort":  "r desc",
      "facet": {
```

```
        "r": "relatedness($fg,$bg)"
      }
    }
  },
  "params": {
    "fg": "creator:\"The Duffer Brothers\"",
    "bg": "*:*"
  }
}
```

Each bucket returns {count, r: {relatedness, foreground_popularity, background_popularity}}. Sort descending on r.relatedness and you have, in one query, the genres most distinctive to Duffer-Brothers shows relative to the entire catalog (sci-fi, horror, supernatural will rank high; period, romance will not).

A more useful production query — "what other shows are most associated with viewers who watch Netflix sci-fi shows?":

```
{
  "query": "*:*",
  "limit": 0,
  "facet": {
    "related_shows": {
      "type":  "terms",
      "field": "cast_str",
      "limit": 20,
      "sort":  "r desc",
      "facet": { "r": "relatedness($fg,$bg)" }
    }
  },
  "params": {
    "fg": "platforms:Netflix AND genres:sci-fi",
    "bg": "*:*"
  }
}
```

From SolrJ via JsonQueryRequest:

```
var req = new JsonQueryRequest()
    .setQuery("*:*")
```

```
    .setLimit(0)
    .withParam("fg", "creator:\"The Duffer Brothers\"")
    .withParam("bg", "*:*")
    .withFacet("related_genres", Map.of(
        "type",  "terms",
        "field", "genres",
        "limit", 10,
        "sort",  "r desc",
        "facet", Map.of("r", "relatedness($fg,$bg)")));
QueryResponse rsp = req.process(solr, "shows");
```

> Production tip — filter rare terms. Add "min_popularity": 0.001 to the relatedness() call to drop ultra-rare terms whose Z-score is statistically meaningless (added in SOLR-12581, shipped in Solr 7.7).
>
> Production tip — docValues="true" required. relatedness() requires docValues="true" on the faceted field. If your field is indexed-only, you'll get a runtime error or silently wrong counts on multi-shard collections.

AIPS reference. Chapter 5 ("Knowledge graph learning") — the entire chapter is built on this primitive. Original paper: Grainger, AlJadda, Korayem, Smith, "The Semantic Knowledge Graph," IEEE DSAA 2016 (arXiv:1609.00464).

5.6 Editorial overrides with the Query Elevation Component

Every commercial streaming service eventually needs the "when the user types stranger things, the new season of Stranger Things is result #1, full stop, and we are not relying on BM25 to figure that out" lever. LTR cannot do this reliably; signals boosting only gets you most of the way. The QueryElevationComponent (solr.QueryElevationComponent) is Solr's built-in answer: an editorial pin/exclude list keyed on the query string, applied after all other scoring.

Configuration in solrconfig.xml

```
<searchComponent name="elevator" class="solr.QueryElevationComponent">
  <str name="queryFieldType">string</str>
  <str name="config-file">elevate.xml</str>
</searchComponent>
```

```
<requestHandler name="/select" class="solr.SearchHandler">
  <lst name="defaults">
    <str name="echoParams">explicit</str>
  </lst>
  <arr name="last-components">
    <str>elevator</str>
  </arr>
</requestHandler>
```

elevate.xml in the configset

```
<elevate>
  <query text="stranger things">
    <doc id="ST-001" />
  </query>
  <query text="the office">
    <doc id="TO-001"  />            <!-- Pin the US version first -->
    <doc id="FLG-001" />            <!-- UK version second -->
  </query>
  <query text="star wars">
    <doc id="AND-001" />            <!-- Editorial: critically acclaimed -->
    <doc id="TM-001"  />
    <doc id="LK-001"  />
  </query>
</elevate>
```

Runtime override and UI badging

You can also elevate or exclude at request time without changing elevate.xml:

```
q=stranger things&elevateIds=ST-001&excludeIds=MH-001
```

And surface the override in your UI using document transformers:

```
fl=id,title,rating,[elevated],[excluded]
```

[elevated] returns true for documents pinned by the elevator — perfect for a “Featured” or “Sponsored” badge.

> Production tip — store editorial rules outside the configset. elevate.xml in the configset means every editorial change forces a configset reload. For a fast-moving editorial team, host the rules in a small admin app that writes elevateIds/excludeIds directly on each request from a database — same outcome, no Solr restart.

Solr in Action reference. Chapter 16 §16.3 (boost-based overrides — the modern API is the runtime elevateIds= form documented here).

Exercises

1. Build a judgment list. Pick 20 head queries for any domain you care about (try TV shows: "comedy", "sci-fi", "british crime drama", "new on netflix", etc.). For each, run them against a real Solr instance (or any search engine), and grade the top 5 results 0–3. Compute NDCG@5 using the formula in §5.1. Now change one piece of the schema (e.g., add a phrase boost on title) and recompute. Did it move?
2. Aggregate signals end to end. Write the Streaming Expression that takes 30 days of click events from shows_signals and produces a shows_signals_aggr collection with count_i and weight_d. Use the two-pass pattern from §5.2. Verify that weight_d is in [0, 1] and that the heaviest-clicked show for each query has weight_d = 1.0.
3. Cap the boost. Write the API-layer code (§5.3) that loads signals for a query, but applies a cap so no single document's boost exceeds 2× the BM25 max-score returned by a probe query. Why is the unbounded version dangerous in a content recommendation context?
4. Pick LTR features. For a TV-shows search service, propose five LTR features beyond originalScore: two text features (SolrFeature), two field-value features (FieldValueFeature), and one query-dependent feature using efi.user_query. Justify each.

5.7 Chapter summary

You can run the entire relevance loop on stock Solr 10 with no external ML platform:

- Measure with a small judgment list and NDCG@10 in CI; use Quepid for the human rating workflow.
- Log every query and click into <collection>_signals; aggregate hourly into <collection>_signals_aggr with Streaming Expressions.
- Boost head queries query-time first, index-time only when stable. Cap boost magnitude to avoid feedback loops.
- Rerank the top 200 with the ltr module. Start with LinearModel plus originalScore as an anchor feature.

- Discover related terms and content with the relatedness() aggregation — no extra service required.
- Override with QueryElevationComponent when editorial trumps algorithm.

The non-obvious lesson: every box above is a Solr feature or a thin job over a Solr collection. The relevance loop does not require a separate vector DB, a separate feature store, or a separate ML platform. Build the loop first; only then ask which boxes deserve to graduate to a dedicated system.

Solr MCP — Exposing Solr to LLM agents

What this chapter covers and why it matters. A search engine is only as useful as the questions you can ask it. Chapters 2–3 taught you how to write Solr queries by hand; Chapter 5 taught you how to make the answers better. This chapter is about removing the hand altogether: standing up an MCP (Model Context Protocol) server in front of Solr so an LLM agent — Claude, GitHub Copilot, Cursor, JetBrains AI Assistant, any MCP client — can call Solr operations as tools using natural language. The shows collection you've built across the book becomes a thing your AI assistant can search, index into, and reason over.

Prerequisites: Chapter 2 (you must understand what a collection, schema, and field type are), Chapter 3 (the parser and parameter surface the MCP server's search tool wraps), and Chapter 4 (the SolrCloud topology the MCP server connects to). Chapter 5 is helpful but not required — the MCP server doesn't yet expose LTR or signals tools, though §6.10 sketches what those would look like.

You will learn: What MCP is and why Solr is a natural fit; the architecture of the Apache Solr MCP Server (Spring AI MCP + SolrJ); the four MCP primitives — @McpTool, @McpResource, @McpPrompt, @McpComplete — and what each one is for; the STDIO vs HTTP transport tradeoff; how to wire the server into Claude Desktop, Claude Code, VS Code, Cursor, and JetBrains IDEs; how to layer OAuth2 authentication and input-validation hardening on top of HTTP mode (and why STDIO needs none); how to get traces, logs, and metrics with the LGTM observability stack; how the server ships as both a JVM image (via Jib) and a GraalVM native image; and how to think about designing your own MCP primitives for LLM consumption.

What you should not skip: §6.5 (the shows-collection quick start — the smallest end-to-end loop), §6.3 (the four primitives — the entire user-visible API of any MCP server), and §6.10 (tool design — the difference between "an MCP server that exists" and "an MCP server an LLM uses well").

6.1 What MCP is and why Solr fits it

The Model Context Protocol (MCP) is an open standard from Anthropic, first published in November 2024 and adopted by Claude, OpenAI, Google, Microsoft, JetBrains, and others by mid-2025. It defines a simple JSON-RPC contract by which an LLM client (the host) can discover and invoke four kinds of primitives exposed by an external server:

- Tools — callable functions with typed arguments and structured responses.
- Resources — URI-addressable read-only data the client can fetch without invoking a tool.
- Prompts — reusable, parameterised prompt templates that surface in clients as slash-commands.
- Completions — typeahead suggestions for prompt arguments and resource-URI variables.

Think of it as a USB-C port for AI: one protocol, many capabilities, no per-vendor adapter. The specification, reference clients, and the protocol Inspector all live at modelcontextprotocol.io.

For Solr specifically, the protocol is a near-perfect fit:

- Tools map cleanly to Solr operations. “Search a collection,” “list collections,” “get the schema,” “index a batch of documents,” “create a collection” — every one is a verb-noun pair the protocol already knows how to express. The LLM doesn’t need to know that the search verb under the hood becomes a /select?q=...&fq=...&facet=true URL; it sees a search tool with named parameters.
- Resources map cleanly to Solr metadata. solr://collections returns the list of collections. solr://shows/schema returns the schema for the shows collection. The MCP client (and through it, the LLM) can read these without invoking a tool — a cheap, side-effect-free way to ground the model in the corpus.
- Solr’s strengths line up with where LLMs are weak. LLMs hallucinate facets; Solr returns real counts. LLMs guess at synonyms; Solr’s analyzer chain knows them. LLMs invent IDs; Solr enforces a uniqueKey. Putting Solr behind an MCP tool lets the LLM stop guessing and start retrieving.

The natural-language gain is concrete. Instead of writing:

```
q=title:”stranger things” AND genres:sci-fi
&fq=platforms:Netflix
&fq=release_year:[2015 TO 2026]
&facet=true&facet.field=genres
&sort=rating desc
&rows=10
```

a user can ask their assistant:

> "Find Netflix sci-fi shows with 'stranger things' in the title released since 2015, break down by genre, and sort by rating."

The LLM, using the Solr MCP Server, picks the right tool, fills in the parameters, runs the query, and presents the results — citing the actual documents. The user never sees the URL.

6.2 The Apache Solr MCP Server: architecture

The reference implementation we cover here is the Apache Solr MCP Server, an Apache 2.0 project that ships a small Spring Boot application exposing Solr operations as MCP tools, resources, prompts, and completions. The code is laid out by Solr concern — each subpackage owns its primitives:

```
org.apache.solr.mcp.server
  Main.java                       // Spring Boot entry point, profile-driven
  collection/CollectionService    // list-collections, create-collection, completions
  indexing/IndexingService        // index-json / -csv / -xml, index-file-document
  schema/SchemaService            // get-schema, add-fields, add-field-types
  search/SearchService            // search
```

The component graph:

Figure 1: Three-layer Solr MCP architecture: MCP clients talk to a Spring AI MCP server over STDIO or HTTP, which proxies through SolrJ to an existing Solr cluster.

Three layers, three responsibilities:

1. Clients speak MCP over either STDIO (stdin/stdout JSON-RPC) or HTTP (Streamable HTTP / SSE). They discover the tool list at connect time, then call tools by name with typed arguments. The client is responsible for showing tool calls and results to the user.
2. Server is a Spring Boot application built on Spring AI MCP (the protocol implementation), with each primitive implemented as a method on a Spring @Service annotated @McpTool / @McpResource / @McpPrompt / @McpComplete. Spring AI handles JSON-RPC, transport, schema generation, and serialisation. The server uses SolrJ (the same client library from §2.6) to talk to Solr.
3. Solr is your existing cluster — the one from Chapter 4. The MCP server is a separate process; it talks to Solr over HTTP just like any other application.

The MCP server adds zero Solr-side state. It's a pure facade. You can run it as a sidecar to your Solr cluster, as a separate deployment in the same Kubernetes namespace, or — for local development — on your laptop talking to a Docker Compose Solr.

6.3 The four MCP primitives

Every MCP server is built from four primitives. The Spring AI MCP server reference maps one annotation to each one:

Table 1: The four MCP primitives — tool, resource, prompt, completion — with the Spring AI annotation, what the LLM sees, and what the human user sees in the client UI.

Primitive	Annotation	What the LLM sees	What the user sees
Tool	@McpTool	A callable function with typed args and a structured return. The LLM picks it based on the description.	The model invokes it; client surfaces the call + result inline.
Resource	@McpResource(uri = "…")	A URI-addressable read-only blob. Read without a tool call, cheap, side-effect-free.	The client can read it or attach it as context.
Prompt	@McpPrompt(name = "…")	A reusable, parameterised prompt template.	Surfaces as a slash-command or "prompt picker" in the client UI.
Completion	@McpComplete(prompt = … | uri = …)	Typeahead suggestions for a prompt arg or a resource-URI variable.	Powers the dropdown when the user types into a prompt arg or URI picker.

Tools are the only primitive the LLM invokes. Resources and prompts can be invoked or surfaced for the user to choose. Completions are pure UX glue — they exist to make the other three pleasant to use from a typeahead picker. Pin this table; it's the conceptual surface of every MCP server you'll meet.

6.3.1 Tools — @McpTool

The Solr MCP Server exposes the operations a Solr admin or app developer would otherwise drive through curl, the Admin UI, or SolrJ. Pin this table; it's the entire tool surface:

Table 2: The thirteen tools exposed by the Apache Solr MCP Server, what each does, the Solr API it maps to, and the read-only / destructive / idempotent behavior hint attached at registration.

Tool	What it does	Maps to (Solr)	Behavior hint
list-collections	Return all collection names in the cluster.	Collections API: LIST	read-only
get-collection-stats	Index stats, query performance, cache hit ratios, handler throughput for one collection.	/admin/metrics filtered to the collection	read-only
check-health	Status + doc count + responsiveness — one-call "is this healthy?"	/admin/ping plus a numDocs query	read-only
create-collection	Create a new collection (name, numShards, numReplicas, configset).	Collections API: CREATE	destructive, not idempotent
get-schema	Field defs, field types, dynamic fields, copy fields, unique key.	Schema API: GET /schema	read-only
add-fields	Append <field/> definitions to a collection's schema.	Schema API: POST add-field	destructive, not idempotent
add-field-types	Append <fieldType/> definitions.	Schema API: POST add-field-type	destructive, not idempotent
search	Full search: query, filterQueries, facetFields, sortClauses, start, rows.	/select via SolrJ SolrQuery	read-only

Tool	What it does	Maps to (Solr)	Behavior hint
index-json-documents	Bulk-index a JSON array string. Batches of 1,000, per-doc retry on failure.	/update/json/docs via SolrJ add(Collection)	destructive, idempotent (if docs carry stable IDs)
index-csv-documents	Bulk-index a CSV string (first row = headers).	/update/csv	destructive, idempotent (same caveat)
index-xml-documents	Bulk-index a Solr-style <add><doc>...</doc></add> XML string.	/update	destructive, idempotent (same caveat)
index-file-document	Bulk-index a file upload (JSON/CSV/XML) without the LLM having to ship the bytes inline.	Mode-dispatches to the three index tools above	destructive, idempotent (same caveat)
find-ref-guide-url	Look up the canonical Apache Solr Reference Guide URL for a given concept or parameter.	Apache reference guide manifest (no Solr call)	read-only

The behavior-hint column is not folklore — those are real MCP tool annotations (readOnlyHint, destructiveHint, idempotentHint) attached at registration time so clients can show distinct UI ("this will modify state, confirm?") and the host LLM can reason about safety. They were added in May 2026 to bring the server in line with the MCP 2025-06-18 spec revision.

A minimal Solr-style tool implementation:

```
@Service
public class SearchService {

    private final SolrClient solr;
    public SearchService(SolrClient solr) { this.solr = solr; }

    @McpTool(
        name = "search",
```

```
        description = "Search a Solr collection with query, filters, facets, sorting, and
↪   pagination.",
        annotations = @McpToolAnnotations(readOnlyHint = true)
    )
    public SearchResponse search(
        @McpToolParam(description = "Solr collection to query", required = true)
            String collection,
        @McpToolParam(description = "Solr query string (q parameter). Defaults to *:*")
            Optional<String> query,
        @McpToolParam(description = "Filter queries (fq parameter)")
            List<String> filterQueries,
        @McpToolParam(description = "Fields to facet on")
            List<String> facetFields
        // ... etc.
    ) throws SolrServerException, IOException {
        validateCollectionName(collection);              // §6.7.3
        SolrQuery q = new SolrQuery(query.orElse("*:*"));
        filterQueries.forEach(q::addFilterQuery);        // also validated, §6.7.3
        facetFields.forEach(f -> { q.addFacetField(f); q.setFacet(true); });
        QueryResponse rsp = solr.query(collection, q);
        return SearchResponse.from(rsp);
    }
}
```

Spring AI generates the JSON-RPC schema from the parameter annotations, validates inputs against it, and serialises the response. You write Java; the framework handles MCP.

The search tool, in detail

This is the tool an LLM calls most. The parameter shape:

```
search(
  collection:    string,       // required — e.g., "shows"
  query:         string,       // optional — Solr q parameter; default "*:*"
  filterQueries: string[],     // optional — fq parameters (validated, §6.7.3)
  facetFields:   string[],     // optional — facet.field (validated)
  sortClauses:   [{item: string, order: "asc"|"desc"}],
  start:         int,          // optional — pagination offset (bounded, §6.7.3)
  rows:          int           // optional — result count; default 10 (bounded)
) → {
```

```
  numFound: int,
  start:    int,
  maxScore: float,
  documents: [...],
  facets:   { fieldName: { value: count, ... } }
}
```

What it does not expose: defType (parser selection), qf/pf/mm/bq/boost (eDisMax tuning from §3.2), rq (LTR rerank from §5.4), fl (field list), and debugQuery. The choice is deliberate: the server's job is to be a stable, narrow surface that an LLM can reliably call. Letting the LLM drive eDisMax tuning per request is a footgun. If you need that flexibility, configure the parser defaults in solrconfig.xml and let them apply universally — the LLM never has to know.

> Production tip — narrow tools, broad config. Resist the urge to expose every Solr knob as a tool parameter. Each parameter you expose is a chance for the LLM to set it incorrectly. Keep tool surfaces minimal and shape behavior via the collection's solrconfig.xml defaults. The model can still get great results — it just can't break things in novel ways.

6.3.2 Resources — @McpResource

A resource is a URI-addressable, read-only blob. Resources let an MCP client ground the LLM in cheap, side-effect-free context without spending a tool call.

Resource URI	Content	Annotation
solr://collections	JSON list of collection names.	@McpResource(uri = "solr://collections")
solr://{collection}/schema	JSON schema dump for {collection}.	@McpResource(uri = "solr://{collection}/schema")

URI templates use RFC 6570 syntax. Spring AI binds the path variables to method parameters by name:

```
@McpResource(
    uri = "solr://{collection}/schema",
    description = "JSON schema dump for the named collection."
)
public SchemaResponse schemaFor(String collection) throws Exception {
    return schemaService.getSchema(collection);
```

```
}
```

The collection URI variable is what powers the @McpComplete typeahead in §6.3.4. The client never has to guess collection names; it asks the server.

> Tool vs Resource — when each wins. Use a tool when the LLM needs to invoke an action with structured arguments and reason about the result (search, index-json-documents). Use a resource when the client can fetch the data ahead of time, attach it to the conversation, or surface it to the user in a side panel — without involving the model. Schema introspection is the canonical resource: cheap, side-effect-free, frequently needed as context.

6.3.3 Prompts — @McpPrompt

A prompt is a reusable, parameterised prompt template that surfaces in clients as a slash-command. The user types /setup-collection in Claude Desktop or MCP Inspector; the server returns a prompt that points the LLM at the right tools in the right order.

The Solr MCP Server exposes six prompts, one per common workflow:

Table 3: The six built-in MCP prompts, the arguments each one takes, and the multi-tool workflow each one directs the LLM to follow.

Prompt	Args	What it scripts
explore-collections	—	"List the collections; for each, fetch its schema and stats; summarise the cluster."
setup-collection	name, purpose (optional)	"Create a collection with the configset and shard/replica defaults the server uses, then add the fields that match the purpose."
view-schema	collection	"Read the solr://{collection}/schema resource and explain the field types, multi-valued fields, and unique key."

Prompt	Args	What it scripts
design-schema	collection, datasetDescription, sampleDocument (optional)	"Given this dataset description, propose add-fields and add-field-types calls; explain the trade-offs."
index-data	collection, format (json/csv/xml), sample (optional)	"Walk the user through index-{format}-documents, including batching, dry runs, and verification."
search-collection	collection, question	"Translate the user's natural-language question into a search tool call; cite the documents returned."

Each prompt returns a String that the framework wraps as a single user-role PromptMessage. The templates don't do the work themselves — they instruct the LLM which tools to call, in what order, with which arguments. They are the seams that let a single user gesture (/index-data shows json) compose into a sequence of MCP tool calls.

```
@McpPrompt(name = "setup-collection",
        description = "Create a collection and walk through field setup for a given purpose.")
public String setupCollection(
   @McpArg(description = "Collection name") String name,
   @McpArg(description = "What this collection is for") Optional<String> purpose
) {
   return """
      Create a Solr collection named '%s' using the default configset '%s', \
      %d shards, %d replicas. Then propose `add-fields` calls suitable for: %s. \
      After each step, confirm before proceeding.
      """.formatted(name, DEFAULT_CONFIGSET, DEFAULT_NUM_SHARDS,
                DEFAULT_REPLICATION_FACTOR, purpose.orElse("general use"));
}
```

The body interpolates the live DEFAULT_CONFIGSET / DEFAULT_NUM_SHARDS / DEFAULT_REPLICATION_FACTOR constants so the prompt text cannot drift from the tool's defaults — a real bug class when prompt text duplicates tool config.

> Tool vs Prompt — when each wins. A tool is what the LLM calls. A prompt is what the user invokes (typically as a slash-command) to direct the LLM to call the right tools. If you find yourself writing a tool whose body composes several other

tools in a specific order, that's a prompt: lift the orchestration up so the user can trigger it directly and the LLM can pick the steps.

6.3.4 Completions — @McpComplete

A completion is a typeahead provider. It does not invoke anything; it suggests values for a prompt argument or a resource-URI variable as the user types. Clients with prompt pickers or resource pickers (MCP Inspector is the canonical one) hit completion/complete and render the suggestions in a dropdown.

The Solr MCP Server has one completion today, bound to the solr://{collection}/schema resource template:

```
@McpComplete(uri = "solr://{collection}/schema")
public List<String> completeCollectionName(CompleteRequest.CompleteArgument arg)
        throws SolrServerException, IOException {
    if (!"collection".equals(arg.name())) return List.of();
    String prefix = arg.value() == null ? "" : arg.value().toLowerCase(Locale.ROOT);
    return listCollections().stream()
        .filter(name -> name.toLowerCase(Locale.ROOT).startsWith(prefix))
        .sorted()
        .limit(MAX_COMPLETION_RESULTS)        // 100 — MCP completion budget
        .toList();
}
```

A handful of design points worth pinning:

- Guard on the argument name. The URI template has one variable (collection), but a future template might have more. Match on arg.name() so completion only fires for the variable you mean.
- Filter by prefix, sort stably, cap the response. The MCP spec budgets a small list of completions per request; sending 10,000 collection names is both wasteful and unusable. 100, sorted, lowercase-prefix-filtered is the sweet spot.
- Re-use the underlying tool's data path. The completion above calls the same listCollections() that the list-collections tool calls. One data path, two surfaces; no separate cache, nothing to invalidate.

A completion can also be bound to a prompt argument: @McpComplete(prompt = "search-collection") powers the dropdown when the user types into the collection arg of the /search-collection slash-command. The handler signature is the same.

> Tool vs Completion — when each wins. A tool is for the LLM. A completion is for the client's typeahead UI. They can be backed by the same underlying call; the distinction is who's consuming the result. If only your LLM ever needs the data, skip completion — list-collections already covers that. Add completion when a human user types into a picker.

That's the entire user-visible API: 13 tools, 2 resources, 6 prompts, 1 completion. No LTR tool, no signals tool, no relatedness() tool — yet. §6.10 covers how you'd add them.

6.4 Transport modes: STDIO vs HTTP

The server supports both MCP transports. They are not interchangeable; pick by deployment shape:

Table 4: Side-by-side comparison of the STDIO and HTTP MCP transport modes across deployment shape, authentication, network exposure, observability, and lifecycle.

	STDIO	HTTP
How it works	Server runs as a child process of the client; communicates via stdin/stdout JSON-RPC	Server runs as a long-lived web service on port 8080; clients connect over HTTP/SSE
Best for	Local AI clients on a developer laptop (Claude Desktop, VS Code, JetBrains)	Multi-client, remote-team, or in-cluster deployments; MCP Inspector debugging
Authentication	OS-level process isolation — the client launches the server, so the server inherits the user's identity	OAuth2 with JWT validation (Auth0, Keycloak, Okta, any OIDC provider)
Network exposure	None — never listens on a port	Listens on 8080; behind a firewall and TLS in production
Observability	Not supported (stdout is the protocol stream)	OpenTelemetry → LGTM stack (Grafana, Tempo, Loki, Mimir)
Lifecycle	Lives as long as the client process	Long-lived; scale horizontally
Native-image variant	Available — sub-second startup, no JVM	Available — same

Use STDIO when your Solr cluster is local or in the same private network and a single developer is the only consumer. The client owns the server's lifecycle, so there's no orphan process to manage.

Use HTTP when you have multiple users sharing one Solr cluster (a team's internal search assistant), when you need OAuth2, when you need traces and metrics, or when you want to deploy the MCP server in Kubernetes alongside the SolrCloud cluster from §4.12.

> Why no observability on STDIO? The MCP STDIO protocol uses stdout as the wire. Anything else printed to stdout — log lines, trace exporters, anything — corrupts the protocol stream. The server suppresses all stdout output in STDIO mode. This is also why Docker images for the server are built with Jib instead of Spring Boot Buildpacks (§6.9): buildpacks emit build banners to stdout that would break STDIO.
>
> Debugging tip — point MCP Inspector at HTTP mode. Inspector is the protocol-level browser for MCP: it lists every tool, resource, prompt, and completion the server exposes, shows the JSON schemas, lets you call them with arbitrary inputs, and dumps the JSON-RPC traffic. Always exercise a new tool/prompt/completion in Inspector against HTTP mode before touching a real client — you'll see the schema and error responses without an LLM in the loop.

6.5 Quick start with the shows collection

End-to-end: from "Solr is running" to "Claude is searching shows" in about two minutes. Assumes a SolrCloud cluster from Chapter 4 reachable at http://localhost:8983/solr/ with the shows collection populated using the seed data from §2.3.

Step 1 — Run the MCP server in STDIO mode via Docker. The server runs as a child of the MCP client, so you don't actually invoke docker run directly; you configure the client to do so.

Step 2 — Configure Claude Desktop. Edit ~/Library/Application Support/Claude/claude_desktop_config.jso
on macOS (or %APPDATA%\Claude\claude_desktop_config.json on Windows):

```
{
  "mcpServers": {
    "solr-shows": {
      "command": "docker",
      "args": [
        "run", "-i", "--rm",
        "-e", "SOLR_URL=http://host.docker.internal:8983/solr/",
```

```
        "ghcr.io/apache/solr-mcp:latest"
      ]
    }
  }
}
```

The -i flag keeps stdin open for the JSON-RPC stream. host.docker.internal reaches the host machine from inside the container; Linux users add --add-host=host.docker.internal:host-gateway to the args.

For a local development build (cloned apache/solr-mcp, ran ./gradlew build), point at the JAR instead of the Docker image:

```
{
  "mcpServers": {
    "solr-mcp": {
      "command": "java",
      "args": ["-jar", "/path/to/solr-mcp/build/libs/solr-mcp-1.0.0-SNAPSHOT.jar"],
      "env": { "SOLR_URL": "http://localhost:8983/solr/" }
    }
  }
}
```

The java -jar form is faster on cold start (no container layer) and lets you iterate on the server's Java code without rebuilding an image — the right shape for contributors and for plugging in a snapshot build.

Restart Claude Desktop. The Solr tools appear in the tool drawer.

Verifying the server with MCP Inspector

Before sending the first query through Claude, it's worth confirming the server is exposing the tools you expect. @modelcontextprotocol/inspector launches the same STDIO command Claude Desktop will and renders the protocol surface in a browser:

```
npx @modelcontextprotocol/inspector \
  java -jar /path/to/solr-mcp/build/libs/solr-mcp-1.0.0-SNAPSHOT.jar
```

It opens on localhost:6274 showing the connected server, the full tool list, and a form for each tool that lets you call it directly and see the raw JSON response — exactly the surface the LLM will see.

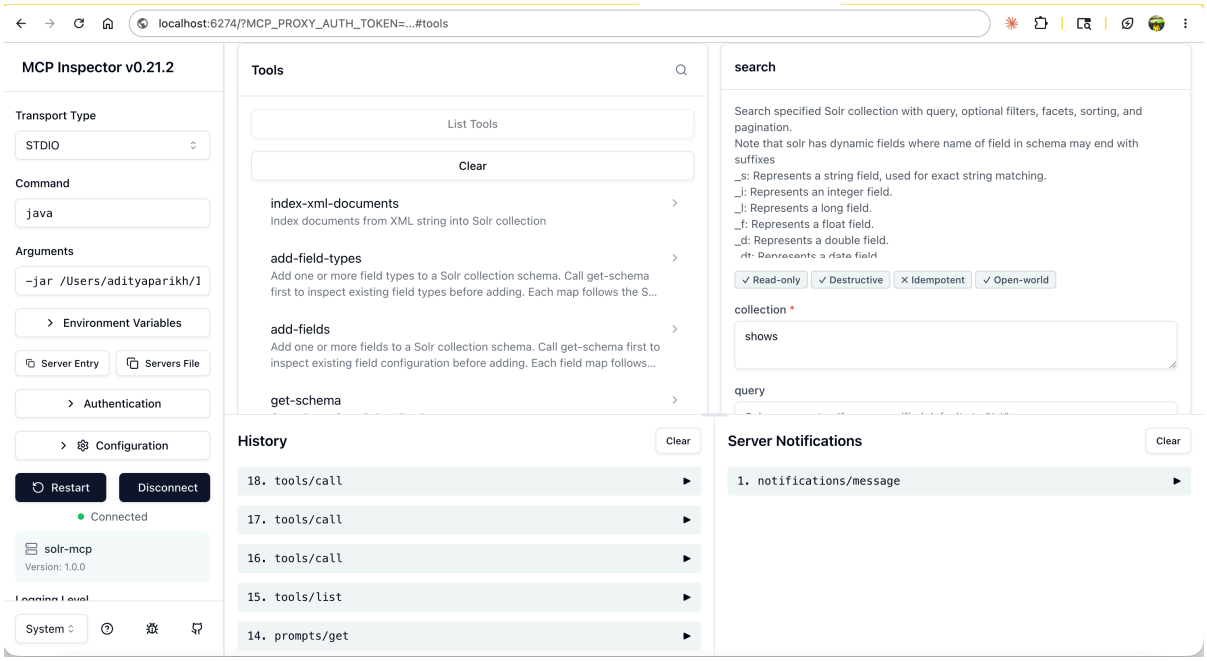

Figure 2: MCP Inspector v0.21.2 connected to the local solr-mcp server. Left: STDIO transport, the java -jar command that launches the server, and the connection state (Connected, server name solr-mcp, version 1.0.0). Middle: the tool list (index-xml-documents, add-field-types, add-fields, get-schema, …) and the recent-call history. Right: the form for the search tool, populated with collection=shows, plus the tool annotations (Read-only , Destructive , Idempotent , Open-world) that drive client-side safety prompts.

The Inspector is the fastest way to debug a misbehaving tool or to validate annotation changes (e.g., flipping Destructive to true for a new write tool) before they reach an LLM.

Step 3 — Try it out. In the Claude Desktop chat:

> "What collections are available in Solr?" → Claude calls list-collections, gets ["shows", "shows_signals", "shows_signals_aggr"], and lists them.
>
> "Show me the schema for the shows collection." → Claude reads the solr://shows/schema resource and summarises: title (text_en), platforms (string, multi-valued), release_year (pint), etc.
>
> "Search shows for Netflix sci-fi released after 2015. Sort by rating. Top 5." → Claude calls search with collection="shows", query="genres:sci-fi", filterQueries=["platforms:Netflix", "release_year:[2015 TO *]"], sort-

Clauses=[{item:"rating", order:"desc"}], rows=5. Returns Stranger Things, Black Mirror, Wednesday, Squid Game, The Witcher.

"Now break that down by genre." → Claude reissues the same query with facetFields=["genres"] and reads the bucket counts.

"Add this show to the catalog: 3 Body Problem, Netflix, sci-fi, 2024, ongoing." → Claude composes a JSON document and calls index-json-documents. The new doc is queryable within seconds.

A real session against the live shows collection looks like Figure 3. Note how the LLM doesn't get it right on the first try — its initial filter platform:"Amazon Prime" returns zero results, so it issues a facet query against the platform field, discovers the actual value is "Amazon Prime Video", and re-runs with the correct filter. The corrective loop is exactly what well-designed MCP tools enable: when a query returns nothing, the LLM has the tools (facets, schema reads) to figure out why and recover.

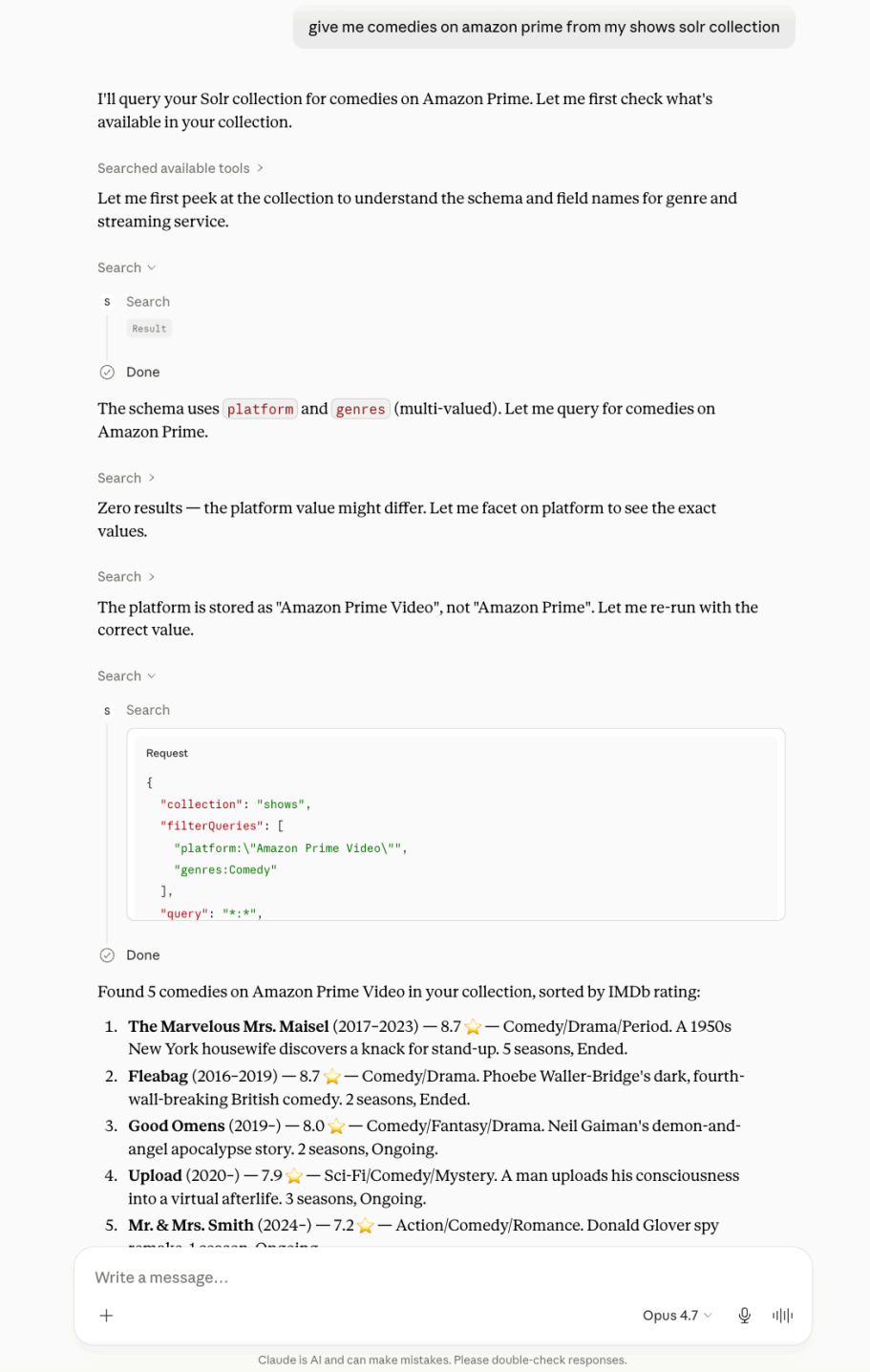

Figure 3: Claude Desktop turning a natural-language ask — “give me comedies on amazon prime from my shows solr collection” — into a chain of search tool calls. Claude first peeks at the schema, retries when its initial platform:”Amazon Prime” filter returns zero results, facets to discover the field value is actually ”Amazon Prime Video”, then re-runs with filterQueries=[”platform:\”Amazon Prime Video\””, ”genres:Comedy”] and renders the five matches sorted by IMDb rating.

The user never saw a Solr URL. The LLM never wrote q= or fq=.

6.6 Client integration: Claude Desktop, Claude Code, VS Code, Cursor, JetBrains

The MCP protocol is the same across clients; the configuration format isn't. Each client has its own config file or settings UI. The patterns repeat — command + args for STDIO, type: "http" + url for HTTP — so the table below is a quick reference and the per-client subsections below give the exact JSON.

Table 5: Where each major MCP client (Claude Desktop, Claude Code, VS Code, Cursor, JetBrains) expects its STDIO config file and how it expresses an HTTP connection.

Client	STDIO config file	HTTP config
Claude Desktop	claude_desktop_config.json	Same file, via mcp-remote wrapper
Claude Code	.mcp.json (project) or CLI claude mcp add	Same
VS Code (Copilot)	.vscode/mcp.json (workspace) or settings.json (user)	"type": "sse" URL
Cursor	.cursor/mcp.json (project) or Settings UI	URL-only config
JetBrains IDEs	.junie/mcp.json (project) or Settings → AI Assistant → MCP Servers	SSE transport

Claude Code (.mcp.json)

The most common shape; put this at the root of your project:

```
{
  "mcpServers": {
    "solr-shows": {
      "type": "stdio",
      "command": "docker",
      "args": ["run", "-i", "--rm",
               "-e", "SOLR_URL=http://host.docker.internal:8983/solr/",
               "ghcr.io/apache/solr-mcp:latest"]
    }
```

```
  }
}
```

For HTTP mode, swap the contents:

```
{
  "mcpServers": {
    "solr-shows": {
      "type": "http",
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

VS Code (.vscode/mcp.json)

```
{
  "servers": {
    "solr-shows": {
      "type": "sse",
      "url": "http://localhost:8080/mcp"
    }
  }
}
```

Cursor and JetBrains

Cursor accepts a top-level mcpServers block in .cursor/mcp.json; JetBrains accepts the same block in .junie/mcp.json once the AI Assistant plugin is installed. Both clients handle OAuth2 automatically when present.

> Production tip — separate STDIO and HTTP server instances per team. If your developers each connect via STDIO on their laptops and you also expose an HTTP MCP server for your CI bots and dashboards, run them as two separate deployments. They have different security postures (process isolation vs. OAuth2) and different observability surfaces (none vs. full LGTM). Don't dual-purpose one binary.

6.7 Security

MCP has two transports with two completely different threat models. Treat them as different products that happen to share a codebase.

6.7.1 STDIO: zero-auth by design, and that's correct

In STDIO mode the server is a child process of the MCP client. The client launches it, pipes JSON-RPC over stdin/stdout, and reaps it on exit. There is no socket, no listener, no port. Three properties fall out of that:

- No network attacker can reach the server. It is not on the network. The only way to send it bytes is to be the parent process — which means being the user who launched the client.
- The server runs as the user. The OS process model is the auth: the server inherits the user's UID/GID, file permissions, environment, and the kernel's process isolation. The user can already do anything the server can do; authenticating "the user" to "their own subprocess" would be theatre.
- No CORS, no CSRF, no rate limit needed. Those defend HTTP endpoints. STDIO has none.

That's why the chapter shows STDIO config like docker run -i --rm … with no token, no API key, no header. The MCP client (Claude Desktop, Claude Code, VS Code, Cursor, JetBrains) is the trust boundary. If a malicious extension can spawn a Docker container as the user, you've already lost — not because of MCP, but because untrusted code is running locally.

The one STDIO-specific failure mode worth knowing: anything the server prints to stdout corrupts the protocol stream. Logs, banners, accidental System.out.println calls — they all become unparseable JSON-RPC and the client disconnects. The server suppresses stdout in STDIO mode by routing all logging to stderr, and Docker images are built with Jib (§6.9) rather than Buildpacks precisely because Buildpacks emit banners to stdout on first container start. STDIO doesn't need auth, but it does need stdout hygiene.

6.7.2 HTTP: OAuth2 with JWT validation

In HTTP mode the server becomes a network-exposed service: a listener on port 8080, addressable from any client that can reach it. That changes everything. The server now needs authentication, input validation, transport encryption, and CORS — i.e., all the things a regular web service needs, applied through the lens of the MCP protocol.

The Solr MCP Server uses OAuth2 with JWT validation, layered on by Spring Security and the Spring AI Community MCP Security module. Any OIDC-compliant provider works: Auth0, Keycloak, Okta, AWS Cognito, Google Identity Platform.

Enable it with three environment variables:

```
PROFILES=http
SECURITY_ENABLED=true
OAUTH2_ISSUER_URI=https://your-provider.example.com/
```

The server then:

1. Exposes its OAuth2 metadata at /.well-known/oauth-authorization-server (per RFC 8414).
2. Rejects unauthenticated requests with 401 Unauthorized and a WWW-Authenticate header pointing to the metadata endpoint.
3. Validates incoming JWTs against the issuer's JWK set on every request — signature, iss, aud, exp, nbf. The JWK set is fetched once and cached.

Modern MCP clients (Claude Desktop via mcp-remote, Claude Code, VS Code, Cursor, JetBrains, MCP Inspector) all handle the OAuth2 flow transparently: when they hit the 401, they discover the authorization server, open a browser for user consent, exchange the authorization code for a token, and attach the token as Authorization: Bearer ... on every subsequent MCP request.

The flow, end to end:

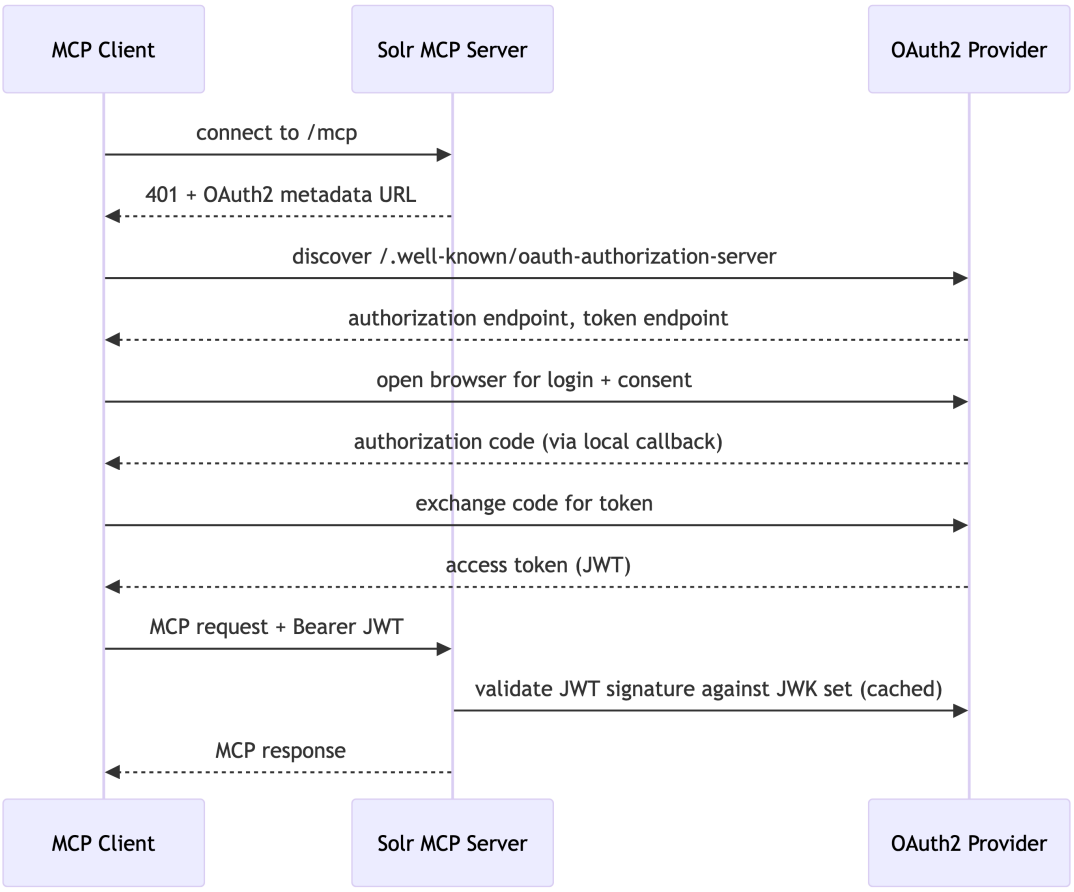

Figure 4: The OAuth2 discovery and JWT-validation flow an MCP client follows to authenticate against the HTTP-mode Solr MCP server through any OIDC provider.

Production tip — pin callback URLs in your IdP. Each MCP client uses a different callback URL on http://localhost:<port>/oauth/callback. MCP Inspector uses port 6274; the mcp-remote bridge used by Claude Desktop uses 3334; the server's own direct flow uses 8080. Register all three patterns in your OAuth2 application's allowed callback list, or only the client you configured first will work.

Production tip — even though it's local, TLS in production. The --allow-http flag for mcp-remote exists for development. In production, the MCP server sits behind TLS just like any other HTTP service, and the callback URLs are https://.... The flag is forgiving but the security model isn't.

6.7.3 Input validation: defending the tool surface

Authentication confirms who is calling. It does not confirm what they're allowed to send. An authenticated user — or an LLM acting on their behalf — can still craft a search tool call whose

query parameter contains a Solr local-param that escalates privilege, or whose filterQueries reach into _query_ or _val_ blocks. Both transports need this layer; HTTP mode needs it more because the caller might not be running on the same machine as the data.

The server enforces input validation at each tool boundary. Four representative classes:

- Local-param injection in q/fq. Solr's q={!parser ...}value syntax lets the first characters of a query select an arbitrary parser. A query of {!terms f=ssn}123,456,... would dump SSN values regardless of what the tool's name says it does. The server rejects any q/fq that begins with {! unless the parser is on a small allowlist (typically lucene, edismax, dismax).
- Field-name allowlisting for sortClauses and facetFields. Sorting on an arbitrary field can hit docValues=false fields and OOME the server. Faceting on the wrong field type can do the same. The server validates each name against the live schema and rejects unknowns with a clear MCP error rather than passing through to Solr.
- Pagination bounds. start 0, rows 1000, start + rows 50000. Deep pagination is a known Solr footgun (§3.10) and an LLM that asks for rows=1_000_000 should get a polite refusal, not a multi-gigabyte response.
- Collection-name format. Every tool that names a collection validates it against Solr's actual naming rules (alphanumeric + _, max length, no path traversal) before issuing any Solr call. Centralising this check means a path-traversal attempt in get-schema fails the same way as one in index-csv-documents.

These checks raise typed exceptions that Spring AI MCP translates into structured JSON-RPC errors. The client sees code: -32602, message: "invalid sort field 'private_field'" instead of a stack trace, and the LLM can recover gracefully.

6.7.4 CORS, on HTTP mode

The HTTP transport supports browser-based MCP clients (early versions of Inspector, future web hosts). That means CORS matters. Defaults out of the box:

- Access-Control-Allow-Origin: http://localhost:* only. The localhost wildcard covers Inspector on 6274, mcp-remote on 3334, and dev hosts on any port.
- Configurable via MCP_CORS_ALLOWED_ORIGINS env var as a comma-separated list when you need to grant a specific deployed host.

Defaulting to * (or omitting CORS entirely) on a service that holds an OAuth2 access token would let any web page in any tab silently call your MCP server with the user's token. The default of localhost-only is deliberate and conservative.

> Production tip — defence in depth, not authentication-or-validation. OAuth2 + JWT confirms who. Input validation + CORS confirms what. You need both. A signed JWT does not make a malicious query safe; a validated query does not make an unauthenticated caller safe. The server applies both gates to every HTTP request.

6.8 Observability with the LGTM stack

In HTTP mode, the server emits OpenTelemetry data — traces, metrics, logs — to any OTLP collector. The reference setup uses the Grafana LGTM stack: Loki for logs, Grafana for the UI, Tempo for traces, Mimir for metrics. Everything bundled in one grafana/otel-lgtm Docker container, perfect for local development; in production you'd point the same OTLP exporters at your existing observability backend.

What gets traced automatically:

- Every MCP tool invocation — one root span per call, with attributes for the tool name and arguments.
- Every outgoing SolrJ HTTP call — one child span per Solr request.
- Every incoming HTTP request to the server itself.
- (In HTTP mode) the OAuth2 token validation cycle.

What this lets you debug:

- "Why is the LLM's search slow?" — Open the trace, see the SolrJ child span, find which Solr collection was slow, correlate with Solr's own metrics from §4.13.
- "Did the LLM call the tool I think it called?" — Filter traces by tool name; the arguments are span attributes.
- "Are we paying for too many tool calls?" — Mimir metrics give you tool invocation rate per minute.

Standard config:

```
OTEL_TRACES_URL=http://localhost:4317          # OTLP gRPC endpoint
OTEL_SAMPLING_PROBABILITY=1.0                    # 100% sampling in dev
```

In production, drop sampling to 0.05–0.1 (5–10%) and point at your real OTLP collector. The server uses @Observed annotations (Micrometer) on every service method, plus Spring Boot's auto-instrumentation for HTTP and SolrJ — you don't have to add tracing code yourself.

6.9 Implementation: Spring AI MCP, Gradle, Jib, GraalVM, Testcontainers

For the Java engineer reading this book, the how it's built matters as much as the how it's used. Five implementation choices anchor the project; each one is worth understanding because each one would have been wrong if chosen carelessly.

6.9.1 Spring AI MCP for the protocol

Spring AI MCP (Spring AI 1.1+) is the protocol implementation. Each primitive is a Spring-bean method annotated with one of the four @Mcp… annotations from §6.3:

- @McpTool + @McpToolParam for tools (schema generated from method signature).
- @McpResource(uri = "…") for resources (URI templates, path-variable binding).
- @McpPrompt(name = "…") + @McpArg for prompts.
- @McpComplete(prompt = "…" \| uri = "…") for completions.

Spring AI handles the JSON-RPC envelope, generates the JSON schemas from parameter annotations, validates incoming arguments against them, serialises responses, and bridges to the chosen transport (STDIO over stdin/stdout, HTTP via Servlet or WebFlux). You write Java; the framework handles MCP. The same source tree compiles to both transports; the runtime profile picks which one starts up. The Spring AI MCP server reference is the authoritative API doc.

6.9.2 Gradle with the Kotlin DSL

The project's build is a single build.gradle.kts. Three things make it tractable:

- Version catalog (gradle/libs.versions.toml). Single source of truth for Spring Boot, Spring AI, SolrJ, Jib, GraalVM, Testcontainers versions. Renovate / Dependabot PRs touch exactly one file.
- Profile property -Pnative flips between JVM and native builds. Without it, the build produces a JVM jar and Jib publishes a JVM image. With it, the GraalVM Native Build Tools plugin is applied and Buildpacks switches to the Paketo native-image buildpack. One property, both worlds.
- Profile property -Pprofile=stdio|http picks which transport the native image bakes in. The native image's reflection metadata is profile-specific, so the build emits two binaries.

6.9.3 Jib for clean JVM images

Jib builds the JVM Docker images. The choice is not incidental: the standard Spring Boot Buildpacks (Paketo) emit build banners to stdout during image creation, and those banners survive into the first container start. For a process whose stdout is the wire protocol (STDIO mode), one such banner breaks the JSON-RPC handshake the first time the client launches it. Jib produces silent, reproducible, layer-cached images with no first-run pollution, plus multi-arch (amd64 + arm64) as a side effect. Buildpacks are still used for the native image (§6.9.4) because the Paketo native-image buildpack is the supported path and native binaries don't have the same first-run-stdout problem.

6.9.4 GraalVM native image for sub-second startup

The server compiles to a GraalVM native image via the GraalVM Native Build Tools Gradle plugin. The motivation is concrete: in STDIO mode the server is launched per session by the MCP client, so JVM startup is user-perceived latency. A native binary cuts cold start from ~2.5 s to under 200 ms.

Two binaries are produced — one per transport:

```
./gradlew nativeCompile -Pnative -Pprofile=stdio
./gradlew nativeCompile -Pnative -Pprofile=http
```

For container images, the same bootBuildImage task that produces JVM images via Jib produces native images via the Paketo native-image buildpack (selected automatically when -Pnative is set):

```
./gradlew bootBuildImage -Pnative -Pprofile=stdio
./gradlew bootBuildImage -Pnative -Pprofile=http
```

Two practical wrinkles to know:

- Reflection metadata. Spring AI's JSON-RPC layer reflects on @McpTool/@McpResource methods at startup. The project ships META-INF/native-image/reachability-metadata.json so the native image actually finds them. Without it, the server starts but exposes zero tools.
- The dockerIntegrationTest task runs the native image under Testcontainers. This is how the project catches reflection-metadata regressions: if a new tool is added without its types being declared, the test fails when MCP Inspector can't list it.

6.9.5 Testcontainers for real-Solr integration tests

Testcontainers runs the integration tests against a real Solr (§2.7), with the version pinned via system property so the server is validated against Solr 9.x and 10.x in the same test suite:

```
./gradlew test -Dsolr.test.image=apache/solr:10.0.0
./gradlew dockerIntegrationTest                          # JVM image vs Solr
./gradlew dockerIntegrationTest -Pnative -Pprofile=stdio   # native STDIO image vs Solr
./gradlew dockerIntegrationTest -Pnative -Pprofile=http    # native HTTP image vs Solr
```

That tested-version matrix is the only credible way to know whether the MCP server actually works against the Solr you run in production — and the only way to catch native-image breakage before it ships, since reflection regressions only surface at runtime.

6.10 Designing primitives for an LLM to use well

The Solr MCP Server's 13 tools, 6 prompts, 2 resources, and 1 completion are a starting set, not a ceiling. If you're building your own primitives (which you should: this is the gap chapter for the MCP book in the series), the design rules vary by primitive type. The first three rules below apply to tools — they're the highest-stakes surface because the LLM picks them autonomously. The fourth and fifth cover prompts and completions.

1. One verb per tool, predictable shape. search is one verb. search-and-then-summarise is two and an LLM will get the second one wrong. Decompose. Let the host LLM compose by calling several tools in sequence — or, better, capture that composition as a prompt (rule 4) so the user can trigger it directly.

2. Constrain the input space. Every free-text parameter is a chance for the model to hallucinate an invalid value. Where possible, enumerate. Instead of query_parser: string, take query_parser: "edismax" | "lucene" | "dismax". Instead of field: string, expose a resource the client can preload, or a completion the client's typeahead can use (rule 5). The tool descriptions and parameter descriptions are part of the prompt to the LLM — write them as if you were writing to a thoughtful junior engineer who has never used Solr. And declare behaviour hints (readOnlyHint, destructiveHint, idempotentHint) so the client can show appropriate confirmation UI.

3. Return structured data the LLM can quote, not narrative. A search tool that returns {numFound, documents:[...], facets:{...}} lets the LLM cite document IDs verbatim. A tool that returns "I found 42 sci-fi shows, the most popular is..." forces the LLM to paraphrase your paraphrase. The structured response is shorter, more accurate, and cheaper in tokens.

4. Lift orchestration into prompts, not into a "mega-tool". When you find yourself wanting a tool whose body composes three other tools in a specific order — say, "list collections, pick one, fetch its schema, propose fields" — that's a prompt, not a tool. A prompt has the LLM compose the steps and the user trigger the workflow. A tool has the LLM hide the steps from the user and pick them silently. The prompt is always the better default: the user sees what's happening and can intervene.

5. Add completions only when a human user types. Completions are for typeahead UIs, not for LLMs. If only the LLM consumes a value space, list-… tools cover it (the model can fetch once and remember). Add a completion when there is a prompt argument or a resource-URI variable a human user will actually type into — and remember to guard on the argument name, sort the response, and cap it at ~100 entries.

What you'd build next, on top of the primitives the server already exposes:

- recommend-similar-shows(id, k) — uses §3.6 vector search under the hood. The LLM doesn't need to know about embeddings; it asks for "more like Severance" and gets ["BM-001", "MH-001", "WW-001"].
- find-related-genres(seed_query) — wraps the §5.5 relatedness() aggregation. "What genres co-occur with creator:'The Duffer Brothers'?" returns a ranked list.
- personalised-search(user_id, query) — runs the §5.4 LTR rerank with a per-user feature vector. Same search surface, but with personalisation baked in.
- pin-result(query, doc_id) — wraps §5.6 query elevation. Useful when an editor uses the AI assistant to curate the search experience.

Each of these is one more @McpTool-annotated method on a Spring @Service. The hard work — the schema, the analyser chain, the relevance loop — is already in the book's prior chapters. The MCP server is the thin layer that makes it all callable from a chat box.

> Production tip — log every tool call with the user query that triggered it. When something goes wrong in production, you want to be able to answer "what did the user ask, what tool was called, what arguments were passed, what came back, and how long did it take?" The OpenTelemetry instrumentation in §6.8 covers most of this for free; the part you must add yourself is the user prompt that led to the tool call, which lives in the host LLM's conversation history and is not visible to the server unless the client forwards it. Some MCP clients support a _meta field on tool calls for exactly this purpose — use it.

6.11 Chapter summary

- MCP is the standard interface between LLM clients (Claude, Copilot, Cursor, JetBrains AI) and external systems. It defines four primitives: typed tools, URI-addressable re-

sources, parameterised prompts, and typeahead completions, all over JSON-RPC.

- Solr is a natural fit for MCP because Solr operations (search, index, facet, list collections, get schema, modify schema) map cleanly to MCP tools and resources, and Solr is the thing that lets LLMs stop guessing and start retrieving.
- The Apache Solr MCP Server is a Spring Boot application built on Spring AI MCP and SolrJ. Today it exposes 13 tools (search, indexing in JSON/CSV/XML/file, schema introspection and modification, collection management, health, stats, ref-guide URL lookup), 2 resources (solr://collections, solr://{collection}/schema), 6 prompts (explore-collections, setup-collection, view-schema, design-schema, index-data, search-collection), and a @McpComplete typeahead bound to the schema resource template.
- The four @Mcp… annotations — @McpTool, @McpResource, @McpPrompt, @McpComplete — are how Spring AI MCP makes you implement the four primitives. Tools are for the LLM to invoke; resources are for cheap grounding context; prompts let users trigger workflows as slash-commands; completions power typeahead pickers.
- STDIO mode needs no auth by design — the client launches the server as a subprocess, so the OS process model is the trust boundary. The only STDIO-specific hazard is stdout hygiene (logs must go to stderr, banners must not exist), which is why images are built with Jib rather than Buildpacks.
- HTTP mode is a network-exposed service and needs the full web-security stack: OAuth2 with JWT validation (any OIDC provider — Auth0, Keycloak, Okta), input validation (local-param injection, field-name allowlisting, pagination bounds), and CORS locked to localhost by default. Authentication confirms who; validation confirms what — you need both.
- Observability uses OpenTelemetry → LGTM (Grafana, Tempo, Loki, Mimir). Every tool invocation and every SolrJ call is traced; instrumentation is free via Spring Boot auto-config and @Observed annotations. STDIO mode cannot have observability because stdout is the wire.
- The server ships in two image flavours. A JVM image built with Jib for production deployments, and a GraalVM native-image variant (per transport) for sub-second cold start — critical for STDIO, where the server is launched per session.
- Testcontainers plus a Gradle version matrix is how the project knows it works against the Solr versions you run, in both JVM and native flavours.
- When designing your own primitives, prefer one verb per tool, constrain inputs with enums/resources/completions, return structured data the LLM can quote, lift multi-step orchestration into prompts, and add completions only where a human user will type. Always exercise new primitives in MCP Inspector against HTTP mode before pointing a real LLM client at them.
- The shows collection from this book becomes a first-class thing an LLM can reason over — without your users ever seeing a Solr URL.

Alternatives — Elasticsearch and OpenSearch

What this chapter covers and why it matters. You will be asked, repeatedly, "why Solr and not Elasticsearch?" This chapter gives you the honest, current answer, with the licensing history that actually drives the decision in 2026.

Prerequisites: Chapters 1–4. To compare engines fairly you need to understand the engine you started with.

You will learn: The five-year licensing churn (Elastic SSPL/ELv2 relicense, OpenSearch fork, AGPLv3 restoration, Linux Foundation transfer) and what it means in May 2026; the architectural differences between Solr, OpenSearch, and Elasticsearch; how API surfaces compare with side-by-side examples; an honest feature-by-feature comparison; and a decision framework for picking the right engine.

Skip if: You've already chosen Solr and are not being asked to justify the choice. This chapter is also useful for migration assessments and architecture reviews — return when you need it.

7.1 The family tree

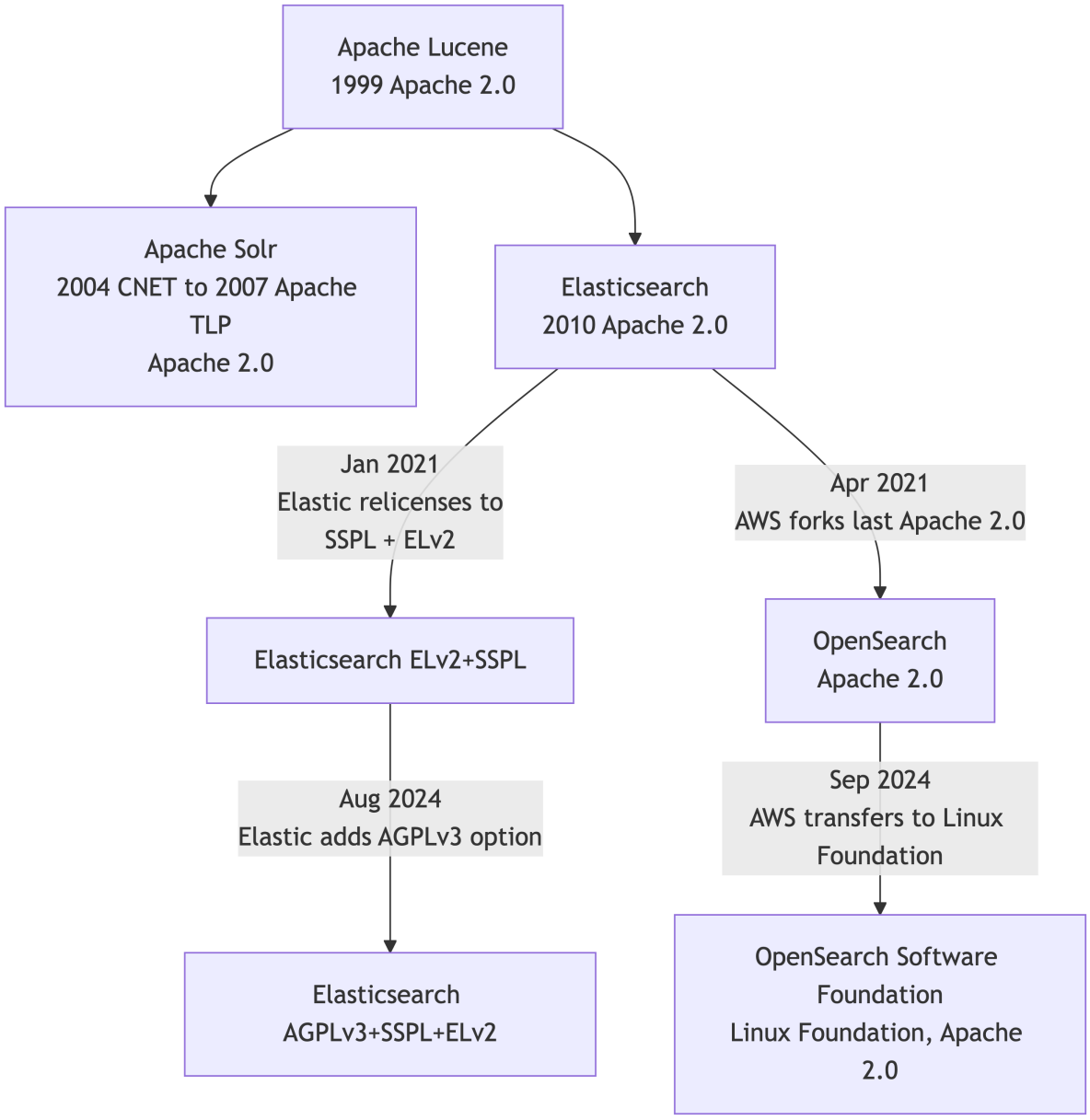

Figure 1: The Lucene-Solr-Elasticsearch-OpenSearch family tree, including the 2021 Elastic relicense, the OpenSearch fork, and the 2024 AGPL restoration and Linux Foundation transfer.

The key dates:

- January 2021 — Elastic relicensed Elasticsearch and Kibana from Apache 2.0 to dual SSPL/Elastic License v2 to block AWS from offering a managed service. The Open

Source Initiative does not consider either license open-source.

- April 2021 — AWS forked the last Apache 2.0 commit and called it OpenSearch, with Kibana → OpenSearch Dashboards.
- August 29, 2024 — Elastic added the GNU AGPLv3 (an OSI-approved open-source license) as a third option for Elasticsearch and Kibana source. As Elastic CTO Shay Banon wrote: "Elasticsearch and Kibana can be called Open Source again."
- September 16, 2024 — AWS transferred OpenSearch to the Linux Foundation. Per the Linux Foundation press release: "The OpenSearch Software Foundation launches with support from premier members AWS, SAP, and Uber and general members Aiven, Aryn, Atlassian, Canonical, DigitalOcean, Eliatra, Graylog, NetApp Instaclustr, and Portal26." (IBM subsequently joined as a Premier Member.)

7.2 Licensing snapshot (May 2026)

Table 1: May 2026 licensing snapshot for Solr, OpenSearch, and Elasticsearch — source license, distribution terms, and any cloud-service restrictions.

Product	Source license(s)	Distribution	Cloud-service restrictions
Apache Solr	Apache 2.0	Apache 2.0	None
OpenSearch	Apache 2.0	Apache 2.0	None
Elasticsearch (source)	AGPLv3 or SSPL 1.0 or ELv2	"Elastic License" binaries	ELv2 prohibits managed-service offerings of the product

For an on-prem deployment of search infrastructure with strict OSS licensing requirements, Solr or OpenSearch are clean choices. For a SaaS company that needs to host its own product on top of a search engine, the Elastic License complications matter; Apache 2.0 (Solr/OpenSearch) removes ambiguity.

7.3 Architectural differences

Table 2: Architectural differences between Solr and the OpenSearch/Elasticsearch family across coordination, schema philosophy, query DSL, sharding, replicas, time-series support, vector fields, and SQL.

Dimension	Solr	OpenSearch / Elasticsearch
Cluster coordination	ZooKeeper (external)	Internal (raft-like, via cluster manager)
Schema	Explicit, managed-schema	Mapping (dynamic by default)
Query DSL	URL params + JSON Request API	JSON DSL only
Sharding	compositeId or implicit router	_id hash; index templates
Replicas	NRT / TLOG / PULL	primary + replicas (plus newer search/data tiers)
Time-series indices	manual / aliases	first-class via index lifecycle management (ILM/ISM)
Vector field	DenseVectorField (Solr 9+)	dense_vector (ES) / knn_vector (OS)
Default similarity	BM25	BM25
Streaming / SQL	Streaming Expressions, Parallel SQL	ES SQL / OpenSearch SQL plugin

7.4 API differences

Solr exposes request handlers at URL paths (/select, /update, /admin/collections) and accepts a mix of query parameters and JSON request bodies. The v2 API (REST-ful) has been maturing through the 9.x and 10.x lines.

OpenSearch/Elasticsearch expose a uniform REST API with a JSON query DSL:

```
POST /shows/_search
{
  "query": {
    "bool": {
      "must":   [ { "match": { "title": "stranger things" } } ],
      "filter": [ { "term":  { "status": "ongoing" } } ]
    }
  },
  "aggs": {
```

```
    "platforms": { "terms": { "field": "platforms.keyword" } }
  }
}
```

Equivalent JSON Request API in Solr 10:

```
POST /solr/shows/query
{
  "query": "stranger things",
  "filter": ["status:ongoing"],
  "facet":  { "platforms": { "type": "terms", "field": "platforms" } }
}
```

Both are JSON-native today; the surface syntax differs but the underlying primitives (boolean queries, terms aggs, filters) map 1:1.

7.5 Feature comparison

Table 3: Feature-by-feature comparison of Solr 10, OpenSearch 3.x, and Elasticsearch 9.x across scoring, vectors, RRF, LTR, security, observability, and Kubernetes tooling.

Feature	Solr 10	OpenSearch 3.x	Elasticsearch 9.x
BM25 default	yes	yes	yes
Faceting / aggregations	yes (JSON Facet API)	yes	yes
Vector search (HNSW)	yes	yes (FAISS/NMSLIB/Lucene)	yes
Scalar/binary vector quantization	yes (Solr 10)	yes	yes
GPU vector search	experimental (cuVS pluggable codec, Solr 10 — new, not battle-tested)	experimental (k-NN plugin)	experimental
Native RRF	Solr 10.1 / 9.11 (not 10.0)	yes	yes
Learning to Rank	yes (built-in)	yes (plugin)	yes (paid tier)
Highlighting	yes (unified/fastvector)	yes	yes

Feature	Solr 10	OpenSearch 3.x	Elasticsearch 9.x
Spell check / suggesters	yes	yes	yes
SQL interface	yes (Parallel SQL)	yes	yes (paid features)
Streaming / pipeline	yes (Streaming Expressions)	no	no
ILM / time-series tiers	partial (aliases)	yes (ISM)	yes (ILM)
Security (TLS, RBAC, audit)	yes free	yes free	partial in basic, full in paid
Built-in ML / anomaly detection	partial	yes free	yes paid
Observability stack	Prometheus, OTEL (Solr 10)	OpenSearch Dashboards, OTEL	Kibana, APM (paid)
Kubernetes operator	Apache Solr Operator (ASF)	OpenSearch Operator	Elastic Cloud on Kubernetes (ECK)

7.6 Performance reality

Independent and vendor benchmarks both exist and contradict each other depending on the workload. Honest summary:

- For lexical search at scale, the three are within a small constant factor of each other; Lucene is the engine under all of them.
- For vector search, OpenSearch with FAISS or NMSLIB engines sometimes outperforms the pure Lucene HNSW used by Solr, especially at very high dimensions. Solr 10's cuVS-Lucene codec is new (SOLR-17892) and reports promising numbers on synthetic benchmarks, but it has not yet seen broad production deployment as of May 2026 — treat it as experimental and benchmark on your own corpus before committing.
- For faceting and analytics over high-cardinality fields, Solr's docValues + JSON Facet API has historically been very competitive.
- For time-series / log ingestion, OpenSearch and Elasticsearch are more polished (ISM/ILM, rollovers, frozen tiers).

Benchmark your own workload. Vendor numbers are marketing.

7.7 When to pick what

Table 4: A decision matrix for picking between Solr, OpenSearch, and Elasticsearch based on the priority that dominates the deployment.

If your priority is…	Pick
Pure search relevance with deep tuning + on-prem control	Solr
You want the same engine for logs/observability and search	OpenSearch
You want an SaaS managed service with the broadest cloud-provider footprint	Elastic Cloud (ES) or Amazon OpenSearch Service
You need maximum vector-search throughput today	OpenSearch (FAISS) — or Solr 10 with cuVS if you have GPUs
Your legal team is allergic to anything non-OSI-approved	Solr or OpenSearch (both Apache 2.0)
You're already running ES and it's working	Stay; the AGPL option restored open-source legitimacy
You're a contributor / want to influence the codebase	Solr (ASF), OpenSearch (Linux Foundation) — both have open governance

7.8 Migration considerations

Solr OpenSearch/Elasticsearch migrations are non-trivial because:

- Schemas vs. mappings. Solr requires explicit fields; ES auto-creates them. Field types map mostly cleanly (text_general text, string keyword).
- Query syntax. eDisMax has no direct ES equivalent; you reconstruct it as a bool with multi_match and query_string.
- Routing. compositeId prefix routing has no ES native equivalent — typically modeled with custom routing on _routing.
- Tooling. SolrJ is a mature, Java-native client library; the ES Java client and the OpenSearch Java client are similar but evolving.
- Operational model. Operator-managed clusters in Kubernetes differ in subtle ways (StatefulSets, PVC layout, ZK vs. internal coordination).

Don't migrate "to chase features"; migrate when the cost of staying exceeds the cost of moving. If you contribute upstream to Solr, the contribution leverage is itself a strategic reason to stay.

7.9 AI tool integration: how ES and OpenSearch compare to Solr MCP

Chapter 6 covered the Apache Solr MCP Server — a Spring Boot facade that exposes Solr collections, schema, and queries as MCP tools an LLM can call. Both Elasticsearch and OpenSearch have followed the same path, and the comparison matters when you're choosing an engine for an AI-driven application.

The three MCP servers, side by side

Table 5: Side-by-side comparison of the Apache Solr, Elastic, and OpenSearch MCP servers across implementation language, license, transport, authentication, tool surface, distinctive features, and maturity.

	Apache Solr MCP Server	Elastic MCP Server	OpenSearch MCP Server
Project	Community / Apache	Elastic (official)	OpenSearch (official, Linux Foundation)
Repository	apache/solr-mcp	elastic/mcp-server-elasticsearch	opensearch-project/opensearch-mcp-server-py
Implementation	Java / Spring AI MCP	TypeScript / Node	Python
License	Apache 2.0	Elastic License v2	Apache 2.0
Transport	STDIO + HTTP (Streamable)	STDIO + HTTP	STDIO + HTTP / SSE
Authentication	OAuth2 (Auth0, Keycloak, any OIDC)	API keys, Elastic Cloud auth	OpenSearch security plugin (basic, JWT, SAML)
Tool surface (core)	search, index-json/csv/xml, list-collections, get-collection-stats, check-health, create-collection, get-schema	search, list_indices, get_index, get_mappings, esql (the Elasticsearch Query Language)	SearchIndexTool, IndexMappingTool, ListIndexTool, ClusterHealthTool, plus model-tool for ML inference

	Apache Solr MCP Server	Elastic MCP Server	OpenSearch MCP Server
Distinctive feature	OAuth2 across multiple IdPs; OTel/LGTM observability built in; clean Java integration	First-class **ES	QL** support — the LLM can write piped queries directly; deep tie-in with Elastic's AI Search features (semantic_text, ELSER)
Maturity (May 2026)	1.0; production-ready for HTTP/OAuth2; ~30 deployments referenced in the wild	1.x; widely deployed via Claude Desktop; one of the most-downloaded MCP servers	2.x; integrated with the OpenSearch Dashboards "Assistant"

All three follow the same pattern from §6.2: a thin facade that exposes the engine's REST API as MCP tools, with the LLM client doing the natural-language → tool-call translation.

The licensing wrinkle, again

The Elastic MCP Server inherits Elastic's licensing posture (§7.2). It's distributed under Elastic License v2, which prohibits offering it as a managed multi-tenant service. For an internal AI assistant inside your organisation this is fine; for a SaaS product that exposes "search-as-a-service" to customers it is not. The Apache and OpenSearch MCP servers are both pure Apache 2.0 and have no such restriction.

When the engine choice flips the MCP choice

A few practical patterns:

- You already run Solr → Solr MCP is the obvious fit. Same JVM stack, same SolrJ client library, same deployment topology, and the Java team that already operates Solr can operate the MCP server.
- You already run Elasticsearch → Elastic MCP gets you ES|QL. ES|QL (Elastic's piped query language, ~Spring 2024) is genuinely well-suited to LLM consumption — pipelines compose linearly, the LLM doesn't have to nest aggregations the way it must with Query DSL. The Elastic MCP Server is the only one that exposes this surface natively.

- You already run OpenSearch → OpenSearch MCP gets you neural search and ML Commons. If you've invested in OpenSearch's built-in model serving (k-NN with FAISS, ELSER-equivalent sparse retrieval, hybrid search via the search pipeline), the OpenSearch MCP Server exposes that machinery as tools the LLM can call directly.
- You're greenfield and the choice is open → the same Apache 2.0 / on-prem / first-class-faceting argument from §7.7 still applies, and Solr + Apache Solr MCP Server is the cleanest stack.

The shared challenges

The interesting observation is that all three MCP servers fight the same battles: how narrow should the tool surface be? How do you stop the LLM from running unbounded queries? How do you log tool calls without violating user privacy? How do you let the LLM read the schema without flooding the context window? These are not Solr-specific problems; they are MCP-server problems. §6.10 (designing tools for an LLM to use well) applies to all three. Read it before you stand up any of them in production.

> Production tip — pick the MCP server that matches your engine, not the engine that matches your MCP server. The MCP layer is thin (a few hundred lines of code per tool) and well-understood; the engine underneath is years of operational investment. Don't let "the MCP server is nicer in language X" drive an engine migration. If you're happy with your Solr cluster from Chapters 2–5, run the Solr MCP server. If you're happy with your OpenSearch cluster, run theirs. Engine migrations to chase MCP nicety are a category error.

Exercises

1. Argue both sides. Write a two-paragraph memo recommending Solr for a new on-prem enterprise-search deployment, and a separate two-paragraph memo recommending OpenSearch for the same hypothetical. What's the strongest argument for each? Which one would you actually pick if you had to deliver in 90 days?
2. Translate a query. Take the eDisMax example from §3.2 (the full one with qf, pf, mm, bq, boost) and write the equivalent Elasticsearch bool-with-multi_match query. Note what doesn't translate cleanly.
3. Audit the licensing. Your legal team asks: "Can we ship Solr inside a closed-source product we sell to customers?" Can we ship Elasticsearch the same way? OpenSearch? Answer each using §7.2 and cite the relevant license.
4. Pick the right MCP server. A streaming company already runs OpenSearch for log analytics and is considering using its catalog data for an AI assistant. They have no Solr or Elasticsearch experience. Which MCP server should they reach for? Justify using §7.9,

and note one concrete thing they'd gain (and one they'd give up) versus a greenfield Solr + Apache Solr MCP Server stack.

7.10 Chapter summary

- The 2021–2024 license churn is settled enough to choose confidently in 2026: Solr and OpenSearch are pure Apache 2.0; Elasticsearch restored an OSI-approved option (AGPLv3) in August 2024 alongside SSPL and ELv2.
- Architecturally, all three are Lucene-based; differences are in coordination (ZK vs. internal), schema philosophy (explicit vs. dynamic), and feature emphasis (Solr leans search; OpenSearch/Elasticsearch lean search + observability).
- OpenSearch matured fast under AWS and is now neutrally governed under the Linux Foundation (OpenSearch Software Foundation, September 16, 2024) with multiple maintainers and a technical steering committee.
- All three engines have first-class MCP servers in 2026 (§7.9). They follow the same architectural pattern; the licensing and tool-surface differences mostly trace back to the engine they sit on top of.
- Choose Solr when you want first-class faceting, on-prem control, full Apache 2.0, a JVM-aligned client library, and an MCP server in the same language as your application. Choose OpenSearch when you want a single stack for search + observability + neural search. Choose Elasticsearch when you're already in the Elastic ecosystem, ES|QL is appealing, and the AGPL/ELv2 mix is acceptable.

Back matter

Closing notes

This handbook took you from "what is an inverted index" through schema design, search, SolrCloud, relevance engineering, and finally — in Chapter 6 — exposing all of it to LLM agents via the Model Context Protocol. The chapters compose: the schema you designed in §2.3 is what an LLM reads through solr://shows/schema; the eDisMax tuning from §3.2 is what shapes the search tool's behaviour; the relevance loop from Chapter 5 is what the LLM is implicitly relying on when it claims a result is "the best"; and the SolrCloud topology from Chapter 4 is what the MCP server's HTTP mode actually deploys against in production.

If you're picking the order to revisit: §1.3 (the five data structures), §2.8 (partial updates), §3.6 (vector search and RRF), §6.5 (the MCP quick start), and §6.10 (designing tools for an LLM to use well) are the load-bearing sections. Everything else hangs off them.

The next moves, depending on where you're going:

- You're shipping Solr to production for the first time. Wire up the Apache Solr Operator (§4.12), turn on the relevance loop end-to-end (§5.2–§5.4), and only then bolt on the MCP server (§6.5). Skipping the relevance loop and going straight to MCP gives an LLM a search engine that returns the wrong things confidently.
- You're modernising an existing Solr deployment. The Solr 9 → 10 differences (Java 21, OpenTelemetry, SolrJ artifact split, bin/solr start default change) are flagged inline in green-bar callouts throughout. The blue/green reindex pattern in §2.9.3 is the safe way to migrate schemas.
- You're choosing an engine. Chapter 7 is the honest comparison. The new §7.9 covers how the MCP story plays out across all three engines.

Key version pins to remember

Component	Version	Notes
Apache Solr	10.0.0	Latest GA. SolrCloud default. Java 21 server.
Apache Lucene	10.3	Bundled with Solr 10.0.0
SolrJ	10.0.0	Java 17 minimum

Component	Version	Notes
Solr Operator	v0.9.1 (March 25, 2025)	v0.10.x for Solr 10 in development
Native RRF	Solr 10.1 / 9.11 (SOLR-17319)	NOT in 10.0.0
Last 9.x release	9.10.1	End of the 9.x line
Apache Solr MCP Server	1.0	Spring AI MCP + SolrJ. Apache 2.0. STDIO + HTTP with OAuth2.
Spring AI MCP	1.1.x	The MCP protocol implementation under the server
MCP protocol	2025-06-18 spec	The latest stable revision as of May 2026
OpenSearch	governed by Linux Foundation since Sep 16, 2024	Apache 2.0
Elasticsearch	AGPLv3 option added Aug 29, 2024	Triple-licensed (AGPLv3 / SSPL / ELv2)

References

References are grouped by type. Throughout the book, the first mention of any source is linked inline; this section gathers the canonical URL for each so the inline citations can stay brief and the long-term locations can be updated in one place.

Official Apache documentation

- Apache Solr Reference Guide (latest). https://solr.apache.org/guide/solr/latest/
- Apache Solr 10 Reference Guide (versioned). https://solr.apache.org/guide/solr/10_0/
- Apache Solr News and downloads. https://solr.apache.org/news.html
- Apache Solr Major Changes in 10. https://solr.apache.org/guide/solr/10_0/upgrade-notes/major-changes-in-solr-10.html
- Apache Lucene 10.3 Javadoc — org.apache.lucene.codecs.lucene103 package. https://lucene.apache.org/core/10_3_0/core/org/apache/lucene/codecs/lucene103/package-summary.html
- Apache Solr Reference: Learning to Rank. https://solr.apache.org/guide/solr/latest/query-guide/learning-to-rank.html
- Apache Solr Reference: Dense Vector Search. https://solr.apache.org/guide/solr/latest/query-guide/dense-vector-search.html
- Apache Solr Reference: Suggester. https://solr.apache.org/guide/solr/latest/query-guide/suggester.html
- Apache Solr Reference: Tokenizers and Filters. https://solr.apache.org/guide/solr/latest/indexing-guide/filters.html
- Apache Solr Operator. https://solr.apache.org/operator/

JIRA tickets cited in the text

- SOLR-8270 / SOLR-8271 — BM25Similarity becomes default (Solr 6).
- SOLR-9480 — relatedness() aggregation for JSON Faceting / Semantic Knowledge Graph.
- SOLR-12581 — min_popularity parameter for relatedness().
- SOLR-14656 — Autoscaling framework removed (Solr 9.0).
- SOLR-17161 — SolrJ artifact split (solr-solrj, solr-solrj-jetty, solr-solrj-zookeeper, solr-solrj-streaming).

References

- SOLR-17319 — Native Reciprocal Rank Fusion (fix versions 10.1 and 9.11).
- SOLR-17780, SOLR-17812 — Scalar and binary quantized dense vectors.
- SOLR-17877 — Optional Overseer-less SolrCloud mode.
- SOLR-17892 — GPU-accelerated KNN via cuVS-Lucene codec.
- SOLR-18005 — ConcurrentUpdateHttp2SolrClient refactor → ConcurrentUpdateJettySolrClient.

Browse: https://issues.apache.org/jira/projects/SOLR

Books

- Grainger, T., Turnbull, D., & Irwin, M. (2025). AI-Powered Search. Manning. https://www.manning.com/books/ai-powered-search
- Turnbull, D., & Berryman, J. (2016). Relevant Search: With applications for Solr and Elasticsearch. Manning.
- Grainger, T., & Potter, T. (2014). Solr in Action. Manning.

Courses

- Udemy — Apache Solr 8 course (slug learn-apache-solr-8). The published curriculum for this course was used as a coverage benchmark when auditing Chapter 2 against a typical introductory-Solr syllabus, to make sure the operator-focused material in this book also covers the foundational concepts a working engineer would otherwise pick up from such a course. https://www.udemy.com/course/learn-apache-solr-8/

Papers

- Malkov, Yu. A., & Yashunin, D. A. (2018). "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs." IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836. https://arxiv.org/abs/1603.09320
- Grainger, T., AlJadda, K., Korayem, M., & Smith, A. (2016). "The Semantic Knowledge Graph: A compact, auto-generated model for real-time traversal and ranking of any relationship within a domain." IEEE DSAA 2016. https://arxiv.org/abs/1609.00464
- Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods." SIGIR 2009. https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf

Blog posts and external references

- Seeley, Y. (April 17, 2015). "Solr Facet Functions and Analytics: a new JSON Facet API." yonik.com.
- Sease. "Exploring Solr Internals — The Lucene Inverted Index" (2015). https://sease.io/2015/07/exploring-solr-internals-lucene.html
- McCandless, M. "Changing Bits" blog. https://blog.mikemccandless.com/
- Blaszyk, J. "Exploring Apache Lucene — Part 1: The Index." https://j.blaszyk.me/tech-blog/exploring-apache-lucene-index/
- Elastic. "The BM25 algorithm and its variables." https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables
- Banon, S. (August 29, 2024). "Elasticsearch is Open Source, Again." Elastic blog.
- Linux Foundation (September 16, 2024). "OpenSearch Software Foundation launch" press release.

Model Context Protocol (Chapter 6)

- MCP specification. https://modelcontextprotocol.io/specification
- MCP GitHub organisation. https://github.com/modelcontextprotocol
- MCP Inspector. https://github.com/modelcontextprotocol/inspector
- Apache Solr MCP Server (repository). https://github.com/apache/solr-mcp
- Elastic MCP Server. https://github.com/elastic/mcp-server-elasticsearch
- OpenSearch MCP Server. https://github.com/opensearch-project/opensearch-mcp-server-py
- Spring AI MCP (Spring Boot integration). https://docs.spring.io/spring-ai/reference/api/mcp/mcp-overview.html
- Spring AI Community MCP Security (OAuth2 support). https://github.com/spring-ai-community/mcp-security
- Claude Desktop. https://claude.ai/download
- mcp-remote (the OAuth2-aware bridge used by STDIO clients to reach HTTP servers). https://github.com/geelen/mcp-remote

Tooling

- Quepid — relevance workbench. https://github.com/o19s/quepid
- Testcontainers Solr module. https://java.testcontainers.org/modules/solr/
- Apache Solr Docker image. https://hub.docker.com/_/solr
- Mermaid. https://mermaid.js.org/
- Jib (Docker images without a daemon). https://github.com/GoogleContainerTools/jib

References

- Grafana LGTM stack (Loki, Grafana, Tempo, Mimir). https://github.com/grafana/docker-otel-lgtm
- OpenTelemetry. https://opentelemetry.io/

Index

The index is alphabetical by topic; bold entries point to the section where a concept is defined, regular entries to where it is used. Section numbers are stable across editions; page numbers (in PDF) shift with edits.

A

B

C

D

E

F

G

H

I

J

K

L

Q

R

S

W

Z